



UNIVERSIDAD SIMÓN BOLÍVAR
DECANATO DE ESTUDIOS PROFESIONALES
COORDINACIÓN DE INGENIERÍA DE COMPUTACIÓN

EVALUACIÓN EMPÍRICA DEL CONSUMO ENERGÉTICO Y RENDIMIENTO EN UNA BASE DE DATOS DISTRIBUIDA

Por:

Lic. Francisco Javier Márquez Hernández

PROYECTO DE GRADO

Presentado ante la Ilustre Universidad Simón Bolívar
como requisito parcial para optar al título de
Ingeniero de Computación

Valle de Sartenejas, marzo de 2026



UNIVERSIDAD SIMÓN BOLÍVAR
DECANATO DE ESTUDIOS PROFESIONALES
COORDINACIÓN DE INGENIERÍA DE COMPUTACIÓN

EVALUACIÓN EMPÍRICA DEL CONSUMO ENERGÉTICO Y RENDIMIENTO EN UNA BASE DE DATOS DISTRIBUIDA

Por:

Lic. Francisco Javier Márquez Hernández

Realizado con la asesoría de:

Dr. Marlene Goncalves Da Silva

PROYECTO DE GRADO

Presentado ante la Ilustre Universidad Simón Bolívar
como requisito parcial para optar al título de
Ingeniero de Computación

Valle de Sartenejas, marzo de 2026



UNIVERSIDAD SIMÓN BOLÍVAR
DECANATO DE ESTUDIOS PROFESIONALES
COORDINACIÓN DE INGENIERÍA DE COMPUTACIÓN

EVALUACIÓN EMPÍRICA DEL CONSUMO ENERGÉTICO Y RENDIMIENTO EN UNA BASE DE DATOS DISTRIBUIDA

Por:

Lic. Francisco Javier Márquez Hernández

Realizado con la asesoría de:

Dr. Marlene Goncalves Da Silva

RESUMEN

Aunque la optimización de consultas es crítica, existen escasos estudios que evalúen conjuntamente el impacto del diseño físico y el volumen de datos sobre el consumo energético en bases de datos relacionales distribuidas. Para abordar esta brecha, este trabajo analiza los efectos del uso de índices (sugeridos por inteligencia artificial) y la reescritura de consultas sobre el gestor PostgreSQL en un entorno distribuido con Citus. La metodología contempló la ejecución de consultas del benchmark TPC-H sobre volúmenes de datos de entre uno y cincuenta gigabytes. El consumo energético se midió mediante Scaphandre, Prometheus e integración numérica, evaluando los planes de ejecución a través de correlaciones estadísticas no paramétricas (Kendall y Spearman). Los resultados prueban que se puede reducir el consumo energético sin dejar de lado una reducción del tiempo de ejecución si se disminuyen las operaciones de entrada/salida. Asimismo, los resultados también muestran que es posible una degradación del rendimiento y del consumo energético de una consulta si no se tiene en consideración las características de la misma al momento de hacer el diseño físico.

Palabras clave: índices, reescritura de consultas, rendimiento, TPC-H, consumo energético.

DEDICATORIA

Dedico este trabajo a: Mi madre, quien es incansable en su apoyo durante toda mi carrera.

Mi pareja, por haberme apoyado en los momentos donde mi propia voluntad flaqueaba.

Mis amigos, Cesar Méndez, Gaby Rodríguez, Andrea Delgado, Jesús Pacheco, Samjay Mustafa y Jesus González, quienes insistieron durante años en que culminara este proyecto, incluso cuando los múltiples desafíos de Venezuela se hacían presentes.

Mi tutora, Dr. Marlene Goncalves y la profesora Dr. Ana Aguilera, por apoyarme en el desarrollo de este proyecto.

Mi ex-coordinadora de carrera, Dra. Aivlé Cabrera y su asistente, Lic. Gina Martínez por tener fe en que iba a culminar mi proyecto.

Mis estudiantes y exestudiantes Andrea Ceballo, Bárbara Guedes, Franklin Camacho, Jean Vásquez, Wismar Pulido, Sofía Gámez, Juan Leandro y Sebastián Rodríguez por su apoyo durante momentos difíciles durante este proyecto.

AGRADECIMIENTOS

Mediante las siguientes líneas quisiera manifestar mi gratitud a:

Mi madre por su insistencia en mi culminación de este proyecto.

Mi pareja por su fe en mi éxito.

Mis profesores por darme la formación que permitió que llevase a cabo este proyecto sin más traspiés de los necesarios.

Mi coordinador por su labor administrativa en el desarrollo de la investigación.

Mi tutora y a la Prof. Ana Aguilera por apoyarme con su experiencia en mis experimentos.

Mis amigos por no permitir que me distrajera de la meta.

A todos ustedes, en varios idiomas:

Muchas gracias.

Thank you so much.

Merci beaucoup.

どうもありがとうございます。

Índice general

DEDICATORIA	3
AGRADECIMIENTOS	4
Índice general	7
Introducción	8
I. El problema	10
1.1. Planteamiento del problema	10
1.2. Antecedentes	12
1.2.1. Eficiencia de las consultas	13
1.2.2. Herramientas de medición de consumo	14
1.3. Objetivos	15
1.3.1. Objetivo general	15
1.3.2. Objetivos específicos	15
II. Marco teórico	17
2.1. <i>Benchmark</i> TPC-H	17
2.1.1. Diseño conceptual de la base de datos	17
2.1.2. Consultas	18
2.2. Medición del consumo energético	20
2.2.1. Potencia eléctrica	20
2.2.2. Consumo energético	20
2.3. Diseño físico de bases de datos	21
2.3.1. Índices	21
2.3.2. Reescritura de consultas	22
2.3.3. Expresiones de tabla común (CTE)	22
2.4. Gestión de memoria	23
2.4.1. Jerarquía de memoria y administración del <i>buffer pool</i>	23
2.4.2. Tormenta de accesos aleatorios	24
2.4.3. Mecanismos de paginación y fallos de página	24
2.5. Pruebas estadísticas	25
2.5.1. Prueba de Shapiro-Wilk	25
2.5.2. Pruebas no paramétricas: Spearman y Kendall	26
III. Marco tecnológico	28
3.1. Ambiente distribuido	28
3.1.1. PSQL	28
3.1.2. Citus	28
3.1.3. DBGen	29
3.2. Consumo	29
3.2.1. RAPL y Scaphandre	29

3.2.2.	Prometheus	30
3.3.	Herramientas suplementarias	30
3.3.1.	Coreutils	30
3.3.2.	Sed	31
3.3.3.	Jq	31
3.3.4.	Python	31
3.3.5.	R y R Studio	31
3.3.6.	Wget	31
IV.	Estudio experimental	32
4.1.	Arquitectura	32
4.2.	Metodología experimental	32
4.2.1.	Carga de datos	32
4.2.2.	Diseño físico	33
4.2.3.	Medidas	34
4.2.4.	Consumo energético	34
4.2.5.	Tratamiento de los datos	35
V.	Resultados y discusión	36
5.1.	Tamaño de la base de datos	36
5.2.	Tiempos de ejecución	37
5.2.1.	Resumen de tiempos	38
5.2.2.	Gráficos de barras: comparación por volumen de datos	38
5.2.3.	Resumen del impacto de la estrategia de diseño	44
5.3.	Consumo energético neto	47
5.3.1.	Resumen de consumo neto	48
5.3.2.	Gráficos de barras: consumo contra volumen de datos	49
5.3.3.	Consumo del nodo coordinador	54
5.4.	Revisión estadística	56
5.4.1.	Normalidad (Shapiro-Wilk)	56
5.4.2.	Correlaciones no paramétricas: Kendall y Spearman	57
	Conclusiones	60
	Referencias	61
A.	TPC-H	72
A.1.	Diseño lógico de la base de datos	72
A.2.	Consultas	72
B.	Instalación de herramientas	80
B.1.	Scaphandre	80
B.2.	Prometheus	80
B.3.	Jq	81
B.4.	Dbgen de TPC-H	81
B.5.	Configuración de clústeres	81
C.	Medias y desviaciones estándar del consumo energético por nodos	83
C.1.	Consumo nodo coordinador	83
C.2.	Consumo nodo trabajador 1	85
C.3.	Consumo nodo trabajador 2	87
D.	Consumo nodos trabajadores	89
E.	Gráficos de dispersión sin diseño	93
E.1.	Consumo neto	93

E.2. Consumo trabajador 1	95
E.3. Consumo trabajador 2	96
E.4. Consumo coordinador	98
F. Gráficos de dispersión con diseño	100
F.1. Consumo neto	100
F.2. Consumo trabajador 1	102
F.3. Consumo trabajador 2	103
F.4. Consumo coordinador	105

Introducción

En los últimos años, con el crecimiento vertiginoso del internet y la migración de los sistemas analógicos y físicos hacia sistemas virtuales, se ha incrementado el interés por el consumo energético de los centros de datos [1, 2], incluso centrado en las técnicas usadas en ellos [3, 4], y los sistemas gestores de bases de datos [5]. El estudio de la eficiencia de las consultas y su consumo energético ha estado presente desde el comienzo, aunque su enfoque ha sido más limitado hacia la eficiencia de las consultas sobre bases de datos centralizadas [6, 7, 8, 9, 10]. Para los fines de este estudio, el término «consulta» trasciende la mera sentencia SQL y se define como el proceso holístico e integral que se activa en el sistema: desde la recepción, análisis sintáctico y planificación hasta la entrega de los resultados.

Sin embargo, estos estudios aún requieren mucha más atención a los múltiples factores que afectan la eficiencia de las consultas y el consumo energético asociado a cada uno: el volumen de datos y el diseño físico. En los estudios disponibles, se destaca el efecto energético de las consultas en términos de las operaciones elementales de una base de datos (lectura, inserciones, eliminaciones y modificaciones). Asimismo, también se estudia cómo operaciones de agregación afectan el perfil energético [10, 11]. Sobre esa base, se han publicado resultados del consumo energético de tales consultas y el efecto que tienen diferentes optimizaciones [8, 11, 12, 13, 14, 15]. Los trabajos consultados coinciden en el estudio de las consultas sin diseño físico y con diseño físico, pero restringido sólo al uso de índices. Inclusive, algunos [16] ponen su enfoque en la manipulación del planificador de consultas —componente del gestor que se encarga de seleccionar los algoritmos y estructuras de ejecución [17, 18, 19, 20]—. Por el contrario, la literatura que aborda el impacto de la reescritura de consultas es escasa.

Las investigaciones preliminares reportan un fenómeno contraintuitivo: menores tiempos de ejecución pueden generar un mayor consumo energético [16, 21, 22, 23]. Es contraintuitivo porque la energía consumida por un sistema depende de la potencia —asumida constante— del mismo y del tiempo de trabajo. Así, el incremento del consumo energético por disminución del tiempo de ejecución está justificado por la intensificación en el uso del hardware y, por tanto, el incremento de la potencia instantánea durante la consulta. Al optimizar una consulta, el planificador suele migrar de algoritmos de acceso secuencial lentos a mecanismos de procesamiento paralelo de alto rendimiento en la CPU. Esta transición suprime los tiempos muertos de espera y fuerza a los núcleos del procesador a operar a su máxima frecuencia de reloj de forma sostenida. En consecuencia, aunque el intervalo de tiempo total (t) se reduce, la potencia eléctrica instantánea (P) demandada por los componentes de la circuitería se incrementa. Como el consumo energético (U) es la integral de la potencia en el tiempo, si la mejora en rendimiento (disminución del tiempo de ejecución) no supera al incremento de la potencia, el impacto energético por la mejora no puede ser menor al de la consulta sin tal cambio.

Para cuantificar estos perfiles de consumo, se ha recurrido a herramientas de software orientadas a la medición energética de bloques de código en los diferentes lenguajes de programación, de los cuales dichas herramientas suelen adoptar su nombre (CPPJoules, RJoules, entre otros) [24, 25, 26, 27]. No obstante, la medición de la eficiencia energética en arquitecturas de datos modernas impone el desafío de evaluar entornos distribuidos y coordinados, donde el consumo ya no depende de un único hilo de ejecución aislado, sino de la orquestación de un clúster de servidores.

En este sentido, la presente investigación enfoca su marco experimental en un ambiente distribuido regido por el gestor de bases de datos PostgreSQL, debido a su arquitectura altamente extensible [28] y a su robusto ecosistema de extensiones. Como pieza central de esta arquitectura se emplea Citus, una extensión que transforma a PostgreSQL en un sistema de bases de datos

distribuido mediante el particionamiento de tablas (*sharding*) y la fragmentación de consultas a través de un clúster de nodos trabajadores y un nodo coordinador [29, 30]. Es debido a la naturaleza distribuida del ambiente de trabajo que, la acepción del término «consulta» de este estudio incluye también planificación en el nodo coordinador de Citus incluyendo la fragmentación y ejecución distribuida en los nodos trabajadores, hasta la transferencia de red y la consolidación de los resultados en el nodo coordinador.

Para resolver el problema de la medición energética de esta capa distribuida, se introduce el uso de Scaphandre como herramienta de monitoreo metrológico de bajo nivel basada en el estimador de energía RAPL (*Running Average Power Limit*) de Intel y AMD. Scaphandre permite aislar el consumo eléctrico ocasionado de forma exclusiva por los identificadores de proceso (PID) pertenecientes a PostgreSQL, lo que discrimina el costo energético del gestor del resto de procesos del computador [31]. El flujo de estas métricas distribuidas es centralizado en tiempo real por el monitor Prometheus, el cual actúa como una base de datos de series temporales que recolecta e indexa eficientemente las lecturas de potencia de cada instancia de Scaphandre en todos los nodos del clúster de forma síncrona [32].

La evaluación del comportamiento del clúster ante el estrés computacional se realiza mediante las consultas del *benchmark* TPC-H, un estándar de la industria compuesto por veintidós consultas de alta complejidad analítica diseñadas para apoyo a la toma de decisiones (OLAP). La complejidad de tales consultas se pone de manifiesto con operaciones de unión (*JOIN*) sobre varias tablas del esquema TPC-H, subconsultas anidadas correlacionadas, agregaciones, ordenamientos y agrupaciones, además de múltiples filtros. TPC-H provee un generador de datos aleatorios estandarizado (*dbgen*) [33] para la variación controlada del factor de escala del conjunto de datos y la incorporación de la escalabilidad vertical en el estudio.

La investigación se orienta, también, a desentrañar el efecto que poseen las estrategias de diseño físico sobre el tiempo de ejecución y consumo de energía en arquitecturas distribuidas bajo condiciones de escalamiento vertical. Entendidas estas estrategias como la fase culminante del diseño de una base de datos dedicada a estructurar el almacenamiento y optimizar los caminos de acceso [17, 18, 19, 20], el estudio se circunscribe específicamente a evaluar el uso de índices avanzados y la reescritura de consultas SQL. A través de este enfoque empírico, se cuantifica el rendimiento —tiempo de ejecución— y consumo energético frente a los escenarios base (sin diseño físico), y se modela la correlación estadística subyacente entre ambas variables, con el fin de determinar si los criterios tradicionales de optimización temporal convergen o colisionan con las metas de sustentabilidad de la infraestructura. Con este estudio, se desea aportar una perspectiva integral sobre cómo las decisiones de diseño físico no solamente optimizan el tiempo de respuesta, sino que actúan como un factor determinante en la eficiencia energética de los sistemas de bases de datos distribuidas.

El presente trabajo se estructura de la siguiente manera: en el Capítulo I se define el problema de investigación, los antecedentes y los objetivos propuestos, cuyas variables operacionales y criterios de evaluación ya han sido delimitados conceptualmente en las líneas anteriores. El Capítulo II establece el marco teórico y conceptual necesario para el entendimiento del hardware y software involucrado, mientras que el Capítulo III describe minuciosamente la arquitectura de las herramientas tecnológicas empleadas. Posteriormente, el Capítulo IV detalla la metodología, las variables y los escenarios experimentales y el Capítulo V expone los resultados obtenidos junto con su respectivo análisis. Finalmente, se presentan las conclusiones y recomendaciones derivadas de la investigación.

Capítulo I. El problema

En este capítulo se expondrá el planteamiento del problema de investigación, los antecedentes en los que se sustenta esta investigación y los objetivos de la misma, los cuales están centrados en el estudio del rendimiento y consumo energético del sistema gestor de bases de datos PostgreSQL en un entorno distribuido. A tal efecto se emplea el *benchmark* TPC-H sobre diversos volúmenes de datos y para los cuales se analizará el tiempo de ejecución de las consultas planteadas en él y el consumo energético del sistema.

1.1. Planteamiento del problema

El consumo energético de los sistemas humanos y la generación de alternativas más eficientes ha sido un interés para el mundo desde la segunda mitad del siglo XX, cuando aparece la primera gran alerta sobre este particular [34, 35], y en el siglo XXI [36] se formaliza este interés en los procesos informáticos. Los sistemas gestores de bases de datos constituyen la columna vertebral de la infraestructura informática actual, impulsada por el auge de internet y la virtualización de entornos físicos. Esta centralidad ha redirigido el interés hacia la optimización de su eficiencia y consumo energético [5], un fenómeno crítico que impacta tanto a las bases de datos residentes en memoria como a las almacenadas en disco.

Mientras que las bases de datos residentes en memoria (IMDB/MMDB) [12, 37] mitigan el impacto energético al operar directamente en la RAM y eliminar la latencia de los cuellos de botella de lecturas a disco, las bases de datos en disco —donde se circunscribe esta investigación— introducen un consumo crítico ligado a las operaciones de Entrada/Salida (E/S). Es precisamente en este último entorno donde se realizan la mayoría de los estudios disponibles. Dichos estudios se realizan bajo el paradigma de optimizar el tiempo de ejecución de las consultas, pero algunos trabajos consultados objetan la visión sobre el tiempo de ejecución [6, 7, 8].

El tiempo de ejecución de una consulta en un sistema gestor de bases de datos está determinado de forma directa por las decisiones del optimizador relacional y la configuración del diseño físico del sistema. El diseño físico constituye la fase en la que se definen las estructuras de almacenamiento técnico, tales como los índices avanzados y los mecanismos de acceso que permiten al motor localizar los datos sin recurrir a costosas lecturas secuenciales de tablas completas [17, 18, 19, 20]. A partir de este diseño, el optimizador evalúa diferentes planes de ejecución lógicos y selecciona los algoritmos que minimicen el costo temporal estimado. Tradicionalmente, cualquier alteración en estas estructuras físicas o en la sintaxis de la sentencia busca acelerar el procesamiento del motor y se asume que un menor tiempo de respuesta equivale a una operación óptima.

Se han encontrado varios hallazgos como que la disminución del consumo energético en las bases de datos al limitar la frecuencia de CPU por debajo de su límite máximo [6]. Tales investigaciones sugieren que una mejora drástica en tiempo de ejecución de una consulta incrementa el consumo energético [7] y a su vez, que el detrimento en el tiempo de ejecución de una consulta puede

conllevar a una reducción del consumo energético, lo que contraviene el paradigma tradicional de optimización del tiempo de ejecución de las consultas [8].

Para analizar el comportamiento energético en este ámbito, es imperativo descomponer la ejecución de una consulta en sus operaciones constitutivas y su contexto transaccional. El procesamiento de una sentencia implica tanto operaciones lógicas en la CPU (como filtrados, ordenamientos y emparejamientos de datos) como operaciones físicas de E/S para la transferencia de páginas desde el almacenamiento. Cuando estas consultas forman parte de un flujo continuo de modificaciones en el estado de la base de datos, se agrupan en transacciones —unidades de trabajo lógico que ejecutan operaciones elementales de lectura, inserción, eliminación y modificación—. Cada uno de estos subprocesos demanda recursos de hardware de manera diferenciada, lo que fragmenta el perfil de potencia total del sistema.

Bajo esta perspectiva operativa, en [38] se muestra que las operaciones de una consulta pueden estructurarse de forma asimétrica en puntos de eficiencia moderada o altamente óptimos, en función de la densidad de transacciones concurrentes que saturan el sistema. Sin embargo, al mantener una planificación correcta en la asignación de dichas operaciones, la potencia demandada se estabiliza en rangos bajos o moderados, lo que impacta positivamente en el consumo energético global.

A su vez, Niemann, en [9], encuentra que el consumo energético (U) y el tiempo de ejecución guardan una correlación lineal si y solo si las operaciones de la consulta están atadas puramente al uso de la CPU. Por el contrario, cuando intervienen operaciones de E/S masivas, esta correlación se rompe, provocando que ejecuciones temporalmente más rápidas deriven en una mayor demanda de potencia instantánea debido al uso en paralelo de múltiples núcleos de procesamiento. Asimismo, en [10, 11] se demuestra que, para lograr una optimización energética efectiva, el planificador debe considerar la jerarquía de consumo intrínseca de las operaciones elementales de la transacción, donde el gasto energético sigue un orden decreciente bien definido: inserciones $\geq$ modificaciones $>$ eliminaciones $\geq$ lecturas.

Esta escala de consumo se fundamenta en el costo físico y operativo de las acciones internas del motor relacional sobre el hardware. Las inserciones ocupan la cúspide energética debido a que exigen la asignación de nuevas páginas de memoria, la escritura física en el almacenamiento persistente y, críticamente, la actualización y rebalanceo en tiempo real de las estructuras de indexación [17, 18], lo que satura de forma simultánea los ciclos de CPU y los canales de E/S. Las modificaciones (*updates*) reducen levemente este impacto al operar, en ocasiones, sobre registros ya asignados, aunque persisten en la exigencia de escritura y reescritura de punteros en las páginas de datos [19]. Por su parte, las eliminaciones suelen ser energéticamente más económicas, ya que los gestores frecuentemente ejecutan un borrado lógico mediante el marcado de registros o *tombstones*, postergando la reestructuración física del archivo en el disco [20]. Finalmente, las lecturas se ubican en la base de la jerarquía porque prescinden por completo de los mecanismos de bloqueo de concurrencia pesados y de la costosa persistencia de datos, y se limitan a la transferencia de datos que, en entornos optimizados, se resuelve directamente a nivel de *buffer cache* [17, 18, 19, 20].

A pesar del rigor de estos análisis a nivel de operaciones lógicas, históricamente el alcance de tales estudios se vio acotado a las bases de datos centralizadas, tanto relacionales (MySQL, PostgreSQL, SQLite) [6, 7, 8, 9, 10, 38] como no relacionales [11, 39]. Sin embargo, la evolución de las cargas de trabajo impulsó la expansión de esta línea de investigación hacia ambientes distribuidos. En [40], se realizan consultas clásicas de bases de datos SQL en Cassandra (NoSQL) y se enfrentan problemas propios del cambio de paradigma en la arquitectura de los datos. En [41], se presenta un estudio comparativo entre el consumo energético de bases de datos NoSQL con distintos subparadigmas (clave-valor, Redis; documentos, MongoDB; columnas, Cassandra). En tal estudio, se encuentra que la eficiencia depende, en gran medida, de las condiciones de la

carga de trabajo. Asimismo, en [42], se presenta una comparación entre los sistemas NoSQL y las implicaciones de cada uno con el teorema CAP (*es imposible para un sistema distribuido satisfacer simultáneamente consistencia fuerte (C), alta disponibilidad (A) y tolerancia a partición (P)* [43, 44]) frente al consumo energético. En [45], se caracteriza el consumo con notación asintótica para los algoritmos utilizados por los gestores de bases de datos. Estos estudios de eficiencia de las consultas y su consumo energético también han abarcado el desarrollo de herramientas para apoyar en la medición [46, 47].

En los últimos años, se han empezado estudios sobre los sistemas de bases de datos distribuidas [40, 42, 48]. En [49], se usa Cassandra y el tema de la proporcionalidad energética en el aspecto del consumo con cargas variables desde la inactividad hasta cargas elevadas. Se encontró que, en inactividad, el consumo energético es significativo frente al consumo con carga total. También se usa Cassandra en [50], pero se aborda el tema de la predicción del consumo energético y el rendimiento mediante modelos. En [51] se trabaja con PostgreSQL y una herramienta propuesta llamada *EnerQuery* y aborda el asunto de la eficiencia energética para el caso de una base de datos con un solo nodo con múltiples núcleos de CPU. Se encontró que el paralelismo incrementa el consumo energético debido al uso de más núcleos a alta frecuencia sin que el tiempo de ejecución sea significativamente mayor. Este comportamiento fue evaluado en [51] al implementar una versión modificada del optimizador de PostgreSQL en un servidor con procesador Intel Xeon X3430 y 32 GB de RAM; mediante dicha modificación, el gestor estimaba analíticamente el costo de potencia basándose en el perfil de los operadores SQL y la intensidad de las operaciones de CPU y E/S bajo el *benchmark* TPC-H.

Sin embargo, estos estudios aún requieren mucha más atención a los múltiples factores que afectan la eficiencia de las consultas y el consumo energético asociado a cada uno [5]: el volumen de datos y el diseño físico. Sobre el volumen de datos, son pocos los estudios que dedican poco más de una carga a su base de datos (ver tabla 1.1) y el diseño físico de las bases de datos. Cuando se hacen, pueden incluir la reescritura de consultas [8, 12] y el uso de índices [11, 14, 15].

Tabla 1.1: Volúmenes de datos utilizados frecuentemente

Volumen (GB)	< 1	1	5	10	25	50	100
Referencias	[12, 6, 11]	[6, 9]	[6, 9]	[6, 9, 52]	[52]	[52]	[14, 15]

La literatura actual sobre consumo energético en entornos distribuidos se concentra predominantemente en soluciones no relacionales e inherentemente distribuidas, como Cassandra y MongoDB [40, 41, 49]. En contraste, existe una marcada escasez de estudios enfocados en sistemas distribuidos relacionales, especialmente bajo el efecto de estrategias de diseño físico o ante variaciones sistemáticas en el volumen de los datos. Es precisamente esta brecha en el estado del arte la que fundamenta la necesidad de ampliar el conocimiento disponible sobre este particular, consolidando el espacio de estudio de la presente investigación.

1.2. Antecedentes

En esta sección se hará una breve descripción de los estudios realizados previamente y su relación con la presente investigación. Dichos estudios se describirán en dos subsecciones: la concerniente a la eficiencia de las consultas y lo relativo a las herramientas que se han usado en el estado del arte para la medición del consumo energético.

1.2.1. Eficiencia de las consultas

Son numerosos los estudios realizados sobre el comportamiento de las consultas a bases de datos relacionales [6, 7, 8, 9, 10, 12, 15, 51] y no relacionales [11, 14, 15, 21, 39, 40, 41, 42, 48, 49, 50], ambas en el aspecto energético y el uso de recursos del computador. En todos los estudios se destaca el efecto energético de las consultas en términos de las operaciones elementales de una base de datos (lectura, inserciones, eliminaciones y modificaciones) así como operaciones de agregación que solamente usan CPU sin operaciones E/S y sobre esa misma base se han publicado resultados sobre el consumo energético de tales consultas y el efecto que tienen diferentes optimizaciones [8, 11, 12, 13, 14, 15] sobre dicha variable. Bajo esta misma línea de investigación, el enfoque de este estudio está en las bases de datos relacionales donde se vinculan estrechamente el desempeño de las consultas (tiempo de ejecución) y el consumo energético de cada una durante su ejecución en un ambiente distribuido y bajo los efectos de una estrategia de diseño físico.

En este orden de ideas, es fundamental definir la carga de trabajo. En diversos estudios previos, se han empleado conjuntos de datos heterogéneos como registros de Twitter [15], usuarios de Netflix y colección de SMS spam [27]. Otras investigaciones han optado por el uso de pruebas de rendimiento estandarizadas, en adelante, *benchmarks*, como:

- Servicios en la nube de Yahoo! (*Yahoo! Cloud Serving Benchmark*, YCSB): es un *benchmark* que surge de la falta de herramientas para evaluar y comparar efectivamente sistemas NoSQL [53] y es usado en varios trabajos enfocados en trabajos en *la nube* [14, 15, 41, 49, 54, 55].
- *TREC ClueWeb09 category B corpus* [13] es un conjunto de datos fundamental utilizado en la investigación de recuperación de información y tecnologías del lenguaje humano, frecuentemente empleado en las conferencias TREC [56].
- Tienda de DVD de Dell (*Dell DVD Store Benchmark*) es una simulación de código abierto de un sitio de comercio electrónico en línea con implementaciones en Microsoft SQL Server, Oracle, MySQL y PostgreSQL junto con programas controladores y aplicaciones web [57]. Se usó en [58].
- Tienda de mascotas de Java (*JPetStore Benchmark*): es una aplicación web clásica, escrita en Java, que implementa las funcionalidades típicas de una tienda en línea (navegación por el catálogo, agregar productos al carrito, *login* de usuarios, proceso de compra). Utiliza un *backend* de base de datos relacional (MySQL, PostgreSQL, etc.) a través del *framework* MyBatis [59, 60]. Se utilizó en [58].
- Procesamiento de transacciones de aplicaciones de telecomunicaciones (*Telecom Application Transaction Processing Benchmark*, TATP) está diseñado para medir el rendimiento de una combinación relacional DBMS/OS/Hardware en una aplicación de telecomunicaciones típica (procesamiento de transacciones en línea, *On Line Transaction Processing*, OLTP) [61, 62].
- Almacén de datos Trabajos de Aventura (*Adventure Works Data Warehouse*) es una arquitectura de almacén centralizado que consta de tablas de hechos y tablas de dimensiones, y que contiene datos obtenidos de la base de datos OLTP y otras fuentes de datos mediante un proceso tradicional de extracción/transformación/carga [63, 64] presente en [65].
- Suite en la Nube (*CloudSuite Benchmark*): es una suite de *benchmarks* para servicios en la nube propios (creados por y para la nube). Tiene ocho *benchmarks* que representan servicios en línea populares y cargas de trabajo analíticas en centros de datos que se basan en pilas de

software (conjunto de capas de programas que funcionan de manera jerárquica para soportar una aplicación o servicio [66]) de código abierto de última generación y están en contenedores para facilitar su uso [67, 68]. Fue usado por Subramanian en [69].

- Consejo de Rendimiento del Procesamiento de Transacciones (*Transaction Processing performance Council*, TPC). [6, 9, 11, 21, 48, 52, 70, 71, 72].

Los *benchmarks* OLTP (TATP, JPetStore, Dell DVD Store) están diseñados para medir operaciones rápidas y constantes (lecturas simples y altas en escritura) [73, 74]. Para un estudio de eficiencia energética, las cargas OLTP suelen mantener el CPU en estados de uso medio o ráfagas cortas [75], lo que dificulta la identificación de cuellos de botella energéticos complejos. Por su parte, los *benchmark* de enfoque NoSQL y Cloud como YCSB y CloudSuite, a pesar de su robustez para un sistema distribuido, están diseñados para modelos de datos no relacionales y consistencia eventual (modelo donde los nodos convergen a un estado común de forma asíncrona si cesan las actualizaciones [76]) [77, 78]. Este estudio está enfocado hacia PostgreSQL por lo que un *benchmark* NoSQL no permitiría evaluar el costo energético de operaciones críticas como JOIN y también las subconsultas anidadas. En el caso de *Adventure Works* [79] y TREC [80], son conjuntos de datos estáticos, es decir, no permiten escalar la cantidad de datos de forma controlada y normalizada.

TPC es un conjunto de *benchmarks* de apoyo de decisión, entre los cuales TPC-H destaca como uno de los más utilizados en el ámbito académico e industrial [33]. Este *benchmark* se compone de veintidós (22) consultas de alta complejidad que incorporan funciones de agregación, consultas anidadas, operaciones de JOIN masivas. Asimismo, incluye un generador de datos variable y normalizado sobre un esquema de ocho (8) tablas que emula la lógica de negocio de un entorno corporativo real [33]. Debido precisamente a este rigor estructural, a la complejidad algorítmica de sus sentencias y a la capacidad de escalar el volumen de información, se seleccionó TPC-H como el entorno experimental definitivo para evaluar las estrategias de diseño físico en esta investigación.

Los distintos trabajos consultados coinciden en el estudio de las consultas sin diseño físico y con diseño físico (uso de índices) y algunos con cambiar configuraciones del planificador de consultas (*query planner*). La literatura que considera la reescritura de consultas es escasa y reporta que las consultas más eficientes generan un mayor consumo de potencia [21, 22, 23, 16].

1.2.2. Herramientas de medición de consumo

Se han utilizado numerosas herramientas para la medición del consumo energético en operaciones de distinta índole y basados en distintos lenguajes de programación: EcoML (basada en Cumulator, que a su vez se basa en Python [81]), que es una herramienta para registrar la huella de carbono (cantidad de gases de efecto invernadero como consecuencia de consumo energético [82]) del *Machine Learning*, donde se mejoró la estimación del consumo energético [83]; Codecarbon (Python) que estima el consumo de potencia eléctrica (GPU, CPU y RAM) y lo aplica a la intensidad de carbono de la región donde se lleva a cabo el cómputo [84], pyJoules (Python) una herramienta de software para medir la huella energética (cantidad total de energía consumida por un proceso o actividad [85]) de una computadora durante la ejecución de un bloque de código de Python. Monitorea la energía consumida por dispositivos específicos del computador como: socket de CPU Intel, RAM (para servidores de arquitectura Intel), GPU Intel integrados (para arquitecturas de cliente) y GPU Nvidia [24].

Otras herramientas basadas en otros lenguajes de programación son jRAPL, que es un *framework* para perfilar (análisis detallado del consumo de recursos de un proceso durante su ejecución

[86]) programas de Java que se ejecutan en CPU compatibles con RAPL (subsección 3.2.1) funciona con cualquier bloque de código de interés energético para el usuario, el cual se detecta por estar entre declaraciones del método `EnergyCheck.statCheck()` [87]. De la familia Joules (análogas a pyJoules) se tiene a RJoules y CPPJoules, basados en R y C++, respectivamente. RJoules es una interfaz envolvente (de los registros de RAPL) desarrollada sobre Intel RAPL que permite el registro de trazas de energía directamente desde el sistema operativo [25]. CPPJoules utiliza la interfaz `powercap` en Linux para acceder a los datos de consumo energético directamente desde varios dominios de Intel-RAPL. Además, para la medición de energía de la GPU, CPPJoules utiliza la biblioteca NVML9, una API basada en C para supervisar y gestionar varios estados de los dispositivos GPU NVIDIA [26]. DBJoules: es una herramienta análoga especialmente diseñada para el consumo energético de sistemas gestores de bases de datos (DBMS, por sus siglas en inglés) [27].

Como parte de las herramientas orientadas a la evaluación de límites térmicos y eléctricos, se destaca el uso de *software* de estrés como **burnMMX**. Este programa, de interfaz de línea de órdenes (CLI, por sus siglas en inglés: *Command Line Interface*) y perteneciente al paquete **cpuburn**, está diseñado específicamente para someter a la CPU a cargas de trabajo extremas mediante bucles infinitos optimizados en lenguaje ensamblador [88]. A diferencia de otras herramientas del conjunto orientadas puramente a las unidades de punto flotante o aritméticas, el algoritmo de **burnMMX** satura intencionalmente los buses de comunicación y las interfaces entre la memoria caché y la memoria RAM principal en procesadores con soporte MMX. El propósito de este nivel de tensión es forzar el uso sostenido de los controladores de memoria a su máxima capacidad, maximizando así la producción de calor y la demanda de potencia instantánea de la placa base para evaluar la estabilidad del hardware ante picos críticos de consumo [89]. En el presente trabajo se utilizará una herramienta de supervisión de consumo llamada **Scaphandre** que se describirá en la sección 3.2.1, debido a la versatilidad que tiene para trabajar en red y con otras herramientas de supervisión centralizada como **Prometheus** [31].

1.3. Objetivos

1.3.1. Objetivo general

Comparar el rendimiento y el consumo energético en un ambiente distribuido, con y sin diseño físico, utilizando el *benchmark* TPC-H y el sistema de bases de datos PostgreSQL.

1.3.2. Objetivos específicos

1. Determinar el comportamiento del rendimiento¹ y el consumo energético en el sistema de bases de datos PostgreSQL mediante las pruebas de rendimiento TPC-H en un ambiente distribuido sin diseño físico a varios volúmenes de datos.
2. Caracterizar los parámetros de rendimiento y el consumo energético en el sistema gestor de bases de datos PostgreSQL mediante las pruebas de rendimiento TPC-H en un ambiente distribuido con diseño físico a varios volúmenes de datos.
3. Comparar el comportamiento del sistema gestor de bases de datos PostgreSQL en términos de rendimiento y consumo energético bajo las pruebas de rendimiento TPC-H en un ambiente

¹Tiempo de ejecución.

distribuido con y sin diseño físico.

4. Evaluar² el impacto de la estrategia de diseño físico aplicada sobre el sistema gestor de bases de datos PostgreSQL mediante las pruebas de rendimiento TPC-H en un ambiente distribuido.

²Comparación empírica de las variables medidas en el estudio.

Capítulo II. Marco teórico

En el presente capítulo se hablará sobre el *benchmark* utilizado en el estudio así como los aspectos teóricos del consumo energético, el diseño físico de bases de datos y las pruebas estadísticas que se realizan sobre el tipo de datos obtenidos en la investigación.

2.1. *Benchmark* TPC-H

El *Transaction Processing Performance Council* (TPC) es una organización sin fines de lucro fundada en 1988 con el propósito de definir *benchmarks* estandarizados para evaluar el rendimiento de entornos de procesamiento de transacciones y bases de datos [33]. Dentro de su conjunto de especificaciones, el *benchmark* TPC-H constituye el estándar industrial diseñado específicamente para entornos de soporte a la toma de decisiones y procesamiento analítico en línea (OLAP). A diferencia de las pruebas orientadas a entornos transaccionales (OLTP) que ejecutan múltiples operaciones cortas y aisladas, TPC-H ha sido utilizado históricamente por la academia y la industria para evaluar la capacidad del sistema gestor al procesar grandes volúmenes de datos mediante consultas de alta complejidad algorítmica, examinando la eficiencia de los motores frente a un espectro amplio de operadores SQL ejecuciones paralelas y accesos intensivos a disco.

2.1.1. Diseño conceptual de la base de datos

Para la presente investigación, se usará como *benchmark* a TPC-H y en esta sección se describirá brevemente esta herramienta. La base de datos del *benchmark* es una simulación de un negocio real y como tal, se dará una breve descripción del negocio a efectos de introducir la semántica de los elementos de la base de datos. Se indicarán los nombres de las relaciones en español en esta descripción discursiva junto con su nombre en inglés, que se mantendrá en lo sucesivo para efectos de consistencia con la especificación del *benchmark* [33]. El diseño lógico de la base de datos se presenta en el apéndice A.

En una nación (**nation**), la cual está localizada en una región (**region**), hay clientes (**customer**) que realizan pedidos (**orders**) y proveedores (**supplier**) que suministran partes (**part**) para satisfacer los pedidos que están en una línea de artículos (**lineitem**). Cada **region** y **nation** se caracterizan por su nombre (**name**), un comentario (**comment**) y un código único (**key**).

Las **part** tienen su **key**, **name**, **comment**, fabricante (**mfgr**), marca (**brand**), tipo (**type**), tamaño (**size**), precio al detal (**retailprice**) y contenedor (**container**). Los **supplier** tienen su **name**, dirección (**address**), teléfono (**phone**), saldo en su cuenta (**acctbal**), su **comment** y **key**. Se entiende que varios **supplier** pueden suministrar la misma **part**, aunque su disponibilidad (**availqty**) y su costo (**suppdocst**) varían según **supplier** y **part** y pueden tener un **comment**.

Los **customer** comparten las características de los **supplier**, pero agregan un segmento de mercado (**mktsegment**). Las **orders** también tienen su **key** y **comment** y estado (**status**), monto total facturado (**totalprice**), fecha de solicitud (**orderdate**), prioridad (**orderpriority**), empleado (**clerk**), prioridad de envío (**shippriority**); mientras que las **lineitem** se identifican unívocamente por la orden y el número de línea (**linenumber**) y también contienen cantidad (**quantity**), precio bruto (**extendedprice**), descuento (**discount**), impuesto (**tax**), advertencia (**returnflag**), estado de la línea (**linestatus**), fecha acordada (**commitdate**) y de entrega (**receiptdate**), instrucción de envío (**shipinstruct**), modo de envío (**shipmode**) y su **comment**.

Finalmente, se presenta el diagrama entidad-relación simplificado sin atributos para ilustrar el diseño conceptual de la base de datos en la figura 2.1.

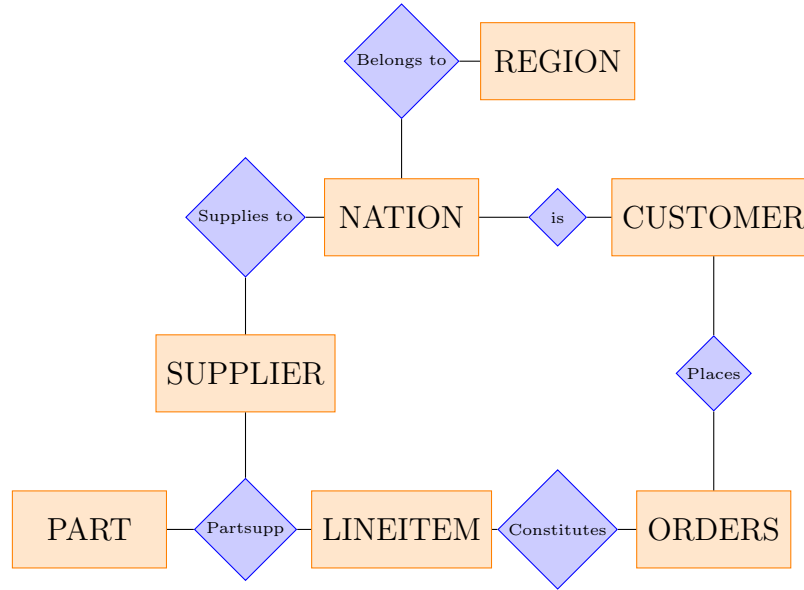


Figura 2.1: Diagrama Entidad Relación simplificado sin atributos

2.1.2. Consultas

El *benchmark* consta de veintidós consultas (Q1..Q22), cuya descripción detallada y su implementación se presenta en el apéndice A.

En la tabla 2.1 donde se comparan las consultas del *benchmark* en términos del número total de operaciones de JOIN de la consulta (JOIN totales), funciones de agregación aplicadas (agregaciones), niveles de anidamiento, tipo de operaciones (Operaciones; S para **SELECT**, C para **CREATE** y D para **DELETE**) y agrupamientos.

En la tabla 2.1 se mencionan varios tipos de JOIN así que, se procederá a definirlos. Primeramente, el Semi JOIN es una operación en álgebra relacional y bases de datos que devuelve las tuplas de una relación izquierda (R) para las cuales existe al menos una tupla coincidente en la relación derecha (S), basándose en una condición de combinación (C). Se diferencia del JOIN, porque no duplica las filas de R si hay múltiples coincidencias en S, ni extrae las columnas de S. Su propósito algorítmico es estrictamente el filtrado por existencia. En cuanto el motor encuentra la primera coincidencia en S para una tupla de R, detiene la búsqueda para esa tupla (evaluación de cortocircuito) y la incluye en el resultado [17]. Formalmente en álgebra relacional, se define como la proyección de los atributos de R sobre el JOIN de R y S como se indica en (1) y se implementa en SQL con **EXISTS** o **IN**

Por su parte, el **LEFT OUTER JOIN** es una operación relacional que preserva toda la información de R, independientemente de si existe una coincidencia en S. El resultado de esta operación incluye todas las tuplas que se generarían en un **JOIN** más aquellas tuplas de R que no cumplen la condición de combinación con ninguna tupla de S. Para estas tuplas sin coincidencia (*dangling tuples*), los atributos de S correspondientes se rellenan con valores nulos (NULL). Algorítmicamente, impide la propiedad conmutativa del optimizador de consultas, ya que el orden lógico de la retención de datos está fijado hacia la relación declarada a la izquierda [18].

$$R \bowtie_C S = \pi_{\text{atributos}(R)}(R \bowtie S) \quad (1)$$

Tabla 2.1: Resumen de las implementaciones de las consultas

Consulta	JOIN totales	Agregación	Niveles de anidamiento	Operaciones	Agrupamientos
Q1	0	8	0	S	2
Q2	4 + 3	1	1	S	0
Q3	2	1	0	S	3
Q4	1 ^b	1	1	S	1
Q5	5	1	0	S	1
Q6	0	1	0	S	0
Q7	5	1	1	S	3
Q8	7	2	1	S	1
Q9	5	1	1	S	2
Q10	3	1	0	S	7
Q11	2 + 2	3	1	S	1
Q12	1	2	0	S	1
Q13 ^a	1 ^c	2	1	S	2
Q14	1	2	0	S	0
Q15	1	2	1	CSD	1
Q16 ^a	1 + 1 ^d	1	1	S	3
Q17	1 + 1 ^b	2	1	S	0
Q18	2 + 1 ^b	2	1	S	5
Q19	1	1	1	S	0
Q20 ^a	1 + 1 ^b + 2	1	2	S	0
Q21	3 + 1 ^b + 1 ^d	1	1	S	1
Q22	1 ^d	5	2	S	1

^a La consulta tiene coincidencia de patrones (*pattern matching*).

^b Es un **SEMI JOIN**.

^c Es un **LEFT OUTER JOIN**.

^d Es un **ANTI JOIN**.

El **ANTI JOIN** es una operación lógica de filtrado que devuelve las tuplas de R para las cuales no existe ninguna tupla coincidente en S bajo una condición específica, C. Es el complemento lógico del **SEMI JOIN**. Algorítmicamente, el motor de la base de datos toma una tupla de R y la evalúa contra S. Si encuentra aunque sea una coincidencia, la tupla de R es inmediatamente descartada. Si recorre las opciones en S y confirma la ausencia absoluta de coincidencias, la tupla de R es enviada al conjunto de resultados. Al igual que el **SEMI JOIN**, no proyecta atributos de la tabla derecha. Formalmente, se expresa como la diferencia de conjuntos entre la relación completa R y

el SEMI JOIN de R y S como se indica en (2) y se implementa en SQL con NOT EXISTS o NOT IN [90].

$$R \triangleright_C S = R - (R \ltimes_C S) \quad (2)$$

2.2. Medición del consumo energético

2.2.1. Potencia eléctrica

Por definición, potencia es el trabajo realizado por un sistema en una unidad de tiempo. Para intervalos de tiempo (dt) infinitesimales se tiene que la fuerza eléctrica ($\vec{F}_e$) hace trabajo para hacer que los portadores de carga se desplacen una cantidad $d\vec{r}$ por lo que se cumple (3):

$$P = \frac{d}{dt} \int \vec{F}_e \cdot d\vec{r} \quad (3)$$

Y se puede demostrar (4), donde ΔV es la diferencia de potencial entre los dos puntos donde ocurre el desplazamiento total de la carga e i la corriente asociada al movimiento de los portadores de carga [91].

$$P = i\Delta V \quad (4)$$

Al trasladar este principio físico a la escala nanométrica de los circuitos integrados modernos, como los microprocesadores que equipan a los servidores, la potencia total demandada por el circuito se manifiesta y divide principalmente en dos componentes operacionales: la potencia estática y la potencia dinámica. La potencia estática representa la energía disipada de forma continua por fugas de corriente en los transistores mientras el circuito se encuentra energizado pero en estado de inactividad (*idle*). Por el contrario, la potencia dinámica constituye la energía consumida directamente por la conmutación activa de los transistores al cambiar de estado lógico para ejecutar instrucciones de CPU.

Como la tasa de conmutación de estos millones de transistores varía milisegundo a milisegundo en función del tipo de instrucciones que procesa el sistema operativo, la potencia eléctrica de un computador no puede ser concebida como una variable estática o un valor constante. Por el contrario, se comporta como una función matemática sumamente volátil que fluctúa en tiempo real según la intensidad de la carga de trabajo impuesta al hardware. En la práctica computacional, esto implica que cualquier intento de cuantificar la demanda eléctrica de un proceso de software no arrojará un resultado fijo, sino una serie temporal de lecturas discretas y altamente variables. Por lo tanto, la determinación de la energía total derivada de esta potencia evita los cálculos algebraicos tradicionales y exige un tratamiento acumulativo en el tiempo, abriendo las puertas al uso de la integración y los métodos numéricos para consolidar dichas fluctuaciones.

2.2.2. Consumo energético

El consumo energético (U) es, por la misma definición de potencia (P), la acumulación de la energía demandada a lo largo de un periodo de ejecución, lo cual se expresa formalmente mediante la ecuación (5), donde t representa el intervalo temporal que toma realizar el traslado de los portadores de carga [91].

$$U = \int_{t_0}^{t_f} P(t)dt \quad (5)$$

En tal sentido, el consumo energético puede calcularse mediante métodos numéricos como la regla del trapecio (donde se aproxima la potencia como segmentos de recta separados a intervalos regulares) [92]. Esta aproximación responde a que las arquitecturas actuales de hardware implementan contadores de rendimiento y sensores de energía internos integrados (como los módulos de telemetría digital de los procesadores modernos). Estos sensores realizan un muestreo discreto de las variables eléctricas a altas frecuencias y digitalizan el comportamiento de la potencia para entregar métricas en intervalos de tiempo fijos y regulares.

2.3. Diseño físico de bases de datos

El diseño físico de una base de datos es la etapa final [19] del proceso de diseño, donde el esquema lógico (tablas, relaciones y restricciones) se traduce en una implementación real [17, 18, 19] dentro de un gestor de base de datos. Es un proceso continuo centrado en el entendimiento de la carga de trabajo, la selección de estrategias y el ajuste posterior a la selección hasta que se alcanza un rendimiento aceptable para el administrador de la base de datos [20]. Dentro de las estrategias que se pueden implementar para el diseño físico están: el uso de índices y la reescritura de consultas. Desde la perspectiva de la eficiencia energética, estas estrategias permiten reducir el número de páginas de disco leídas y los ciclos de CPU invertidos, disminuyendo de forma directa la potencia dinámica del hardware. Dentro de las líneas de acción con mayor impacto que se pueden implementar para el diseño físico están el uso de índices y la reescritura de consultas.

2.3.1. Índices

Son estructuras auxiliares de datos que aceleran la recuperación de la información en la base de datos [17, 18, 19, 20]. Como estructuras, los índices suelen ser *B-Tree* por árbol balanceado (*balanced tree*) con tiempo de búsqueda logarítmico [17] y recomendado para lecturas por rango [20], es decir, consultas donde se usen operadores de desigualdad o **BETWEEN** (en SQL). Y también se suelen usar índices de *Hash* (una implementación particular de tablas de *hash* para gestores de bases de datos), recomendados para búsquedas por igualdad [18], pero no para búsquedas por rango [19].

Hay especializaciones, como los *B+ Trees*, que optimizan a los *B Trees* al unir cada hoja del árbol y formar una lista enlazada lo que hace que las búsquedas por rangos sean aún más rápidas en la práctica [20]. Sin embargo, aún estos índices presentan limitaciones de búsqueda para un caso particular: atributos de texto. Para ello se tienen los índices *Full Text* que funcionan como el índice analítico (o alfabético) de los libros: en estos índices cada palabra es almacenada junto con apuntes a cada lugar donde aparece [17, 20, 93].

Dentro de este tipo de índices, PostgreSQL ofrece una implementación de gran interés para la presente investigación: los índices de trigramas GIN [17, 18, 20]. Este mecanismo, en lugar de guardar palabras completas, descompone las cadenas de texto y almacena solamente subcadenas de tres caracteres (trigramas), lo que acelera notablemente la búsqueda de patrones con el operador **LIKE** al reducir el esfuerzo de procesamiento. Para los fines de este trabajo, se utilizarán las estructuras *B+ Trees* (por ser el estándar por defecto de PostgreSQL) y GIN de trigramas, con el

objetivo de evaluar cómo la elección de cada uno de estos caminos de acceso afecta el rendimiento temporal y el perfil de consumo energético del sistema.

2.3.2. Reescritura de consultas

En primera instancia, el gestor de bases de datos no ejecuta las consultas como están escritas en el lenguaje de alto nivel, sino que primero son optimizadas por el gestor de bases de datos antes de ejecutarlas [17]. Por otro lado, la forma en la que se plantea la consulta le impide al gestor hacer una buena optimización o incluso, decidir si usa un índice o no (por ejemplo, usar solamente el año en un filtro en lugar de un rango: con este último, el gestor sí usa el índice) [20].

El proceso de reescritura de consultas no es necesariamente sencillo y en muchos casos puede llegar a alterar la lógica de la consulta y, por tanto, el resultado de la misma, por lo tanto, se debe avanzar con cautela por esta vía [20, 94, 95]. En el caso de esta investigación, la reescritura de consultas será puntual al usar copias de una tabla que está distribuida por otro atributo.

2.3.3. Expresiones de tabla común (CTE)

Una expresión de tabla común (CTE, por sus siglas en inglés, *Common Table Expression*) se define formalmente como un conjunto de resultados temporales y con nombre, el cual se deriva de una consulta relacional y cuyo ciclo de vida (y alcance de ejecución) está restringido exclusivamente a una única instrucción principal (**SELECT**, **INSERT**, **UPDATE** o **DELETE**).

Fueron incluidas formalmente en el estándar SQL:1999 (también conocido como SQL3) a través de la cláusula **WITH**, las CTE actúan conceptualmente como vistas efímeras (o tablas derivadas *in line*). Su importancia en la arquitectura de consultas radica en dos propiedades fundamentales: la modularidad y reusabilidad algorítmica; y la recursividad (semántica de punto fijo). La modularidad y reusabilidad permiten descomponer consultas analíticas complejas en bloques lógicos secuenciales. A diferencia de las subconsultas anidadas tradicionales, una CTE puede ser referenciada múltiples veces dentro de la misma instrucción principal sin necesidad de reescribir la lógica ni persistir los datos físicamente en el disco. La recursividad, a través de la variante **WITH RECURSIVE**, dotó al lenguaje SQL de la capacidad matemática de procesar grafos y estructuras jerárquicas de profundidad arbitraria. Esto se logra evaluando iterativamente un miembro ancla (caso base) y un miembro recursivo uniéndolos hasta que no se produzcan nuevas tuplas [96].

Materialización

La materialización se define como el proceso de evaluar una expresión relacional (como una vista, una subconsulta o una CTE) y almacenar físicamente su conjunto resultante de tuplas en una estructura de datos temporal o persistente, en lugar de recalculados dichos resultados al vuelo (*on the fly* o *pipelining*) cada vez que son referenciados por una consulta externa. Conceptualmente, esto es la aplicación directa del principio de intercambio entre espacio y tiempo: se sacrifica espacio de almacenamiento (disco o memoria) y frescura de los datos a cambio de una reducción drástica en el costo computacional (CPU e I/O) durante el tiempo de lectura. Se implementa en SQL con **MATERIALIZED** [20].

2.4. Gestión de memoria

La ejecución de consultas complejas sobre un gestor de bases de datos no depende únicamente de la eficiencia algorítmica de su diseño lógico o físico, sino que está profundamente condicionada por la interacción entre el gestor de la base de datos, el sistema operativo y la infraestructura de hardware subyacente. Para comprender cómo escala el rendimiento y el consumo energético ante variaciones drásticas en el volumen de datos, es fundamental analizar los fenómenos físicos y operativos que ocurren durante la transferencia de información entre los diferentes niveles de almacenamiento [97].

En esta sección se abordan los fundamentos de la administración de la memoria principal mediante el espacio de almacenamiento temporal del gestor, el impacto de los patrones de consulta desestructurados sobre la eficiencia del hardware, y las consecuencias del desbordamiento de la memoria de trabajo a través de los mecanismos de transferencia hacia el almacenamiento persistente.

2.4.1. Jerarquía de memoria y administración del *buffer pool*

La arquitectura de un servidor moderno está gobernada por una pirámide jerárquica de almacenamiento. En la cúspide se ubican los componentes más cercanos al procesador (las memorias caché L1, L2 y L3), las cuales ofrecen latencias de acceso extremadamente bajas pero capacidades de almacenamiento muy reducidas. Inmediatamente abajo se encuentra la memoria de acceso aleatorio (RAM) y, finalmente, en la base se sitúa el almacenamiento secundario o persistente (unidades de estado sólido o discos mecánicos), caracterizado por poseer capacidades masivas a costa de tiempos de respuesta exponencialmente mayores.

En este entorno, los DBMS implementan un componente crítico de software denominado *buffer pool* o administrador de búferes. El *buffer pool* es una región de la memoria RAM reservada explícitamente por el gestor de la base de datos para almacenar temporalmente las páginas de datos y de índices que se leen desde el almacenamiento persistente. El objetivo primordial de este mecanismo es minimizar las operaciones físicas de E/S, aprovechando el principio de localidad espacial y temporal de las consultas para resolver la mayor cantidad de peticiones directamente en los componentes circuitales rápidos [20].

Cuando el gestor de evaluación de consultas requiere procesar un conjunto de registros, el administrador de búferes verifica si las páginas solicitadas ya residen en la memoria principal, un fenómeno conocido como *buffer hit*. En ese caso, el acceso a los datos se realiza de forma casi instantánea a la velocidad nativa de la RAM, o incluso de las memorias caché de la CPU si el volumen de datos es lo suficientemente compacto como para ser retenido en los registros del procesador.

Por el contrario, si las páginas no se encuentran en memoria se produce un *buffer miss*, lo que obliga al gestor a suspender momentáneamente la operación en la CPU para emitir una solicitud de lectura física al almacenamiento secundario [98]. Debido a que el costo temporal de leer desde el disco es de varios órdenes de magnitud superior al de la memoria RAM, la tasa de aciertos del *Buffer Pool* se convierte en el factor determinante del rendimiento general de la base de datos [20].

Cuando el tamaño de las tablas de un esquema de base de datos escala y supera la capacidad física asignada al *buffer pool*, el espacio de memoria se satura. Ante un *buffer miss*, en este escenario, el gestor no puede cargar la nueva página sin antes desalojar una preexistente. Para ello, los gestores implementan políticas de reemplazo de páginas como el algoritmo *Least Recently Used* (LRU) [20]. Este algoritmo identifica y expulsa la página que lleva más tiempo sin ser consultada; sin embargo,

si la carga de trabajo exige una lectura secuencial masiva que supera el tamaño del búfer, LRU se vuelve ineficiente, provocando un ciclo infinito de desalojos y cargas físicas. Este fenómeno marca el inicio de la degradación drástica del rendimiento, trasladando el cuello de botella desde la lógica del software hacia las restricciones físicas del hardware de almacenamiento.

2.4.2. Tormenta de accesos aleatorios

Las técnicas de diseño físico descritas en las subsecciones anteriores tienen como propósito fundamental mitigar el impacto adverso de los patrones de acceso ineficientes al almacenamiento secundario. De acuerdo con [99], el motor de ejecución de un sistema gestor de bases de datos opera bajo una arquitectura donde el subsistema de Entrada/Salida (E/S) representa el principal cuello de botella, dado que las operaciones para recuperar páginas de datos desde el almacenamiento hacia el *buffer pool* de la memoria RAM son órdenes de magnitud más lentas que los ciclos de procesamiento de la CPU [99].

La tormenta de accesos aleatorios es una condición de saturación crítica en el subsistema de almacenamiento. Este fenómeno se caracteriza por la ejecución masiva y concurrente de operaciones sobre bloques de datos que no guardan contigüidad física en el disco. Esta dispersión de solicitudes hacia bloques de datos no contiguos desencadena la patología en el hardware que satura críticamente las colas de E/S. Al no implementarse estructuras de acceso físico que agrupen u ordenen la ubicación de los datos en el disco, la controladora de almacenamiento colapsa bajo el peso de miles de peticiones de lectura pequeñas y arbitrarias que compiten entre sí por los recursos de transferencia.

La consecuencia directa de este escenario no es solo una degradación exponencial del rendimiento temporal del DBMS, sino también un incremento drástico en la demanda de potencia eléctrica y el consumo energético del hardware de almacenamiento, el cual debe operar a su máxima capacidad de transferencia para resolver un mapa caótico de direccionamiento físico [100, 99]. Así, la tormenta de accesos aleatorios se consolida como el principal enemigo de la eficiencia energética en el software de bases de datos lo que justifica la necesidad de implementar estrategias de optimización física que transformen el acceso aleatorio concurrente en lecturas ordenadas y predecibles.

2.4.3. Mecanismos de paginación y fallos de página

La eficiencia de las estructuras de diseño físico, particularmente de los índices densos, depende críticamente de la velocidad del sustrato de hardware donde residen durante la ejecución de las consultas. Los sistemas operativos modernos gestionan la memoria a través de un esquema de memoria virtual, el cual abstrae el almacenamiento físico en bloques de tamaño uniforme denominados páginas [97]. El gestor de la base de datos confía en que las páginas que contienen los nodos del índice se encuentren alojadas en la RAM para garantizar tiempos de respuesta en el orden de los microsegundos [101].

Sin embargo, cuando el volumen de los datos o el tamaño acumulado de las estructuras auxiliares indexadas excede la capacidad de la memoria asignada al buffer del gestor, la unidad de gestión de memoria (MMU por sus siglas en inglés) se ve imposibilitada para resolver las direcciones lógicas de forma nativa [98]. Al intentar acceder a una sección del índice que no se encuentra cargada físicamente en la memoria, el sistema operativo interrumpe el hilo de procesamiento y genera una excepción de hardware denominada fallo de página (*page fault*), específicamente un fallo de página mayor (*major page fault*) [97]. A diferencia de un fallo menor (*minor page fault*) —donde la página ya reside en la RAM y solo requiere una reasignación de punteros a nivel de software—, el *major*

page fault obliga al sistema operativo a suspender la ejecución del proceso de la consulta en la CPU para realizar una operación de lectura física en el almacenamiento secundario, y trasladar la página faltante desde el disco hacia la RAM.

La ocurrencia de un *major page fault* altera drásticamente el perfil de consumo energético a través de tres mecanismos de bajo nivel. En primer lugar, se genera un escenario de sincronización y conmutación a un estado inactivo en la CPU, donde el procesador debe suspender la ejecución del hilo de la consulta SQL y ceder los ciclos de reloj al kernel del sistema operativo para que este programe la operación de E/S. Dado que las pruebas se realizan en un entorno de servidor dedicado exclusivo para el DBMS, al bloquearse la consulta la cola de procesos listos de usuario queda vacía; en consecuencia, la CPU se ve obligada a conmutar hacia el proceso inactivo del sistema (*idle*), entrando en un estado de baja potencia por inactividad donde consume energía basal sin avanzar en el cálculo de la consulta [102]. En segundo lugar, ocurre la activación del almacenamiento secundario, donde el controlador de almacenamiento debe localizar y transferir la página faltante desde el soporte físico hacia la RAM [101]. Esta activación tanto del bus de datos como de los circuitos integrados de la memoria secundaria dispara la demanda de potencia eléctrica del componente de almacenamiento, un impacto energético que ha sido documentado al evaluar cargas analíticas pesadas [103]. En tercer lugar, se produce la destrucción de la localidad de datos dentro del subsistema de memoria. Cuando una consulta realiza búsquedas sobre un índice cuyo tamaño desborda el espacio asignado en la RAM, se rompe el principio de localidad espacial, provocando que los accesos posteriores no puedan reutilizar los datos ya cargados [97]. Para dar espacio a las nuevas páginas requeridas, el sistema operativo se ve obligado a desalojar de forma continua otras páginas del *buffer pool*. Esto somete al subsistema de memoria a un ciclo constante de reemplazo y lectura física indexada que incrementa el tiempo de ejecución y sostiene una alta demanda de potencia en los buses de comunicación [98].

Desde la perspectiva de la eficiencia energética de la infraestructura, el fallo de página representa el punto de transición donde el gasto energético deja de estar determinado por la potencia dinámica del procesador (procesamiento lógico) y pasa a estar dominado por la latencia y el consumo de los canales de E/S [102, 103]. Este mecanismo mitiga los beneficios de cualquier optimización algorítmica previa, introduciendo una penalización térmica y eléctrica constante mientras persista el desborde de la memoria principal [21].

2.5. Pruebas estadísticas

2.5.1. Prueba de Shapiro-Wilk

La prueba de Shapiro-Wilk [104] está basada en el cálculo de un estadístico, W , sobre una muestra aleatorizada de $n \leq 50$ observaciones ordenadas ascendentemente, independientes e idénticamente distribuidas de una variable continua [105]. La hipótesis nula (H_0) es que la muestra proviene de una población que sigue una distribución normal, la hipótesis alternativa H_a es que la muestra no proviene de una población con distribución normal. El estadístico de prueba es (6) donde $x_{(i)}$ es el i -ésimo estadístico de orden, $\bar{x}$ es la media muestral, a_i son coeficientes calculados a partir de las medias, varianzas y covarianzas de los estadísticos de orden de una distribución normal estándar¹ y $k \approx \frac{n}{2}$ [104].

¹Valores disponibles en: https://dm.udc.es/profesores/ricardo/Archivos/tablas_estadisticas.pdf#page=22

$$W = \frac{\left(\sum_{i=1}^k a_i (x_{(n-i+1)} - x_{(i)}) \right)^2}{\sum_{i=1}^n (x_{(i)} - \bar{x})^2} \quad (6)$$

En esta prueba, cada valor $x_{(i)}$ está apareado con su simétrico $x_{(n-i+1)}$ en la lista ordenada de valores, es decir, el primero está apareado con el último, el segundo con el penúltimo, tercero y antepenúltimo, etc. El estadístico de prueba está acotado ($0 < W \leq 1$) y se rechaza la hipótesis nula cuando $W < W_{(\alpha, n)}$ ². En cuanto al p valor es la probabilidad de observar un estadístico W tan extremo (pequeño) como el obtenido asumiendo que la hipótesis nula es cierta, es decir, se rechaza H_0 si y solo si, $p < \alpha$ [105].

A manera de ejemplo, considere el conjunto de datos $X = \{20, 25, 18, 20, 20\}$, se ordenan los datos $X = \{18, 20, 20, 20, 25\}$ y se toma la media muestral de los datos $\bar{x} = 20,6$ y se relacionan primero con último y segundo con penúltimo (se ignora el central).

$$W = \frac{((0,6646 \times (25 - 18) + 0,2413 \times (20 - 20))^2}{(18 - 20,6)^2 + 3(20 - 20,6)^2 + (25 - 20,6)^2} = \frac{21,6}{27,2} = 0,796 > 0,762$$

por lo tanto, con 95 % de confianza, no se puede rechazar la hipótesis de que X provenga de una población con distribución normal.

2.5.2. Pruebas no paramétricas: Spearman y Kendall

Al igual que con la correlación de Pearson, las pruebas de Spearman y Kendall estudian la probabilidad de correlación entre dos variables aleatorias bajo un cierto modelo. Spearman evalúa la descripción de la relación entre dos variables mediante una función monótona (como una recta). La prueba de Spearman no trabaja con los valores originales de las medidas, X_i y Y_i , sino con su jerarquía o posición relativa, es decir, se les asigna un *rango* (*rgo*) de forma creciente y en el caso de empates, se les asigna el promedio de los rangos. Si no hay empates significativos, el estadístico de prueba, ρ , de Spearman, para n observaciones, es (7) donde $d_i = rgo(X_i) - rgo(Y_i)$ para la observación i -ésima [106].

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)} \quad (7)$$

Para que dicha prueba sea válida, se requiere que exista un orden en los datos observados y que dichas observaciones estén apareadas, esto es, cada X_i tiene asociada una observación Y_i , e independientes bajo una relación monótona. H_0 es que no existe correlación entre las variables ($\rho = 0$) que sea significativa, H_a que sí existe una correlación significativa. El estadístico está acotado $|\rho| \leq 1$ y el p-valor es la probabilidad de obtener un coeficiente de correlación tan grande como el observado si la verdadera correlación en la población fuera cero. No se rechaza la hipótesis nula si y solo si $p \geq \alpha$ [107].

Por ejemplo, consdiere las medidas de dos variables X , Y donde Y es dependiente de X ,

$$\{(1, 20), (2, 18), (3, 25), (4, 20), (5, 20)\}$$

²Valores disponibles en: <https://dm.udc.es/profesores/ricardo/Archivos/tablas.estadisticas.pdf#page=20>

Se desprenden los conjuntos de datos $X = \{1, 2, 3, 4, 5\}$, $Y = \{20, 18, 25, 20, 20\}$. X está ordenado y Y , no. Se ordena $Y = \{18, 20, 20, 20, 25\}$, como las posiciones segunda a cuarta están empatadas, se promedian las posiciones (2, 3, 4). Las posiciones primera y quinta no tienen problemas, así que su rango es el cardinal correspondiente. Se puede escribir, luego, que:

(X, Y)	$rgoX_i$	$rgoY_i$	d_i	d_i^2
(1, 20)	1	$\frac{2+3+4}{3} = 3$	-2	4
(2, 18)	2	1	1	1
(3, 25)	3	5	-2	4
(4, 20)	4	$\frac{2+3+4}{3} = 3$	1	1
(5, 20)	5	$\frac{2+3+4}{3} = 3$	2	4

Luego, $\rho = 1 - \frac{6(14)}{5(5^2-1)} = 1 - \frac{12(7)}{5(24)} = 1 - \frac{7}{10} = 0,3$ Lo que indica una débil correlación positiva (se dice entonces que si X crece, no siempre Y crece, mas no decrece).

A diferencia de Spearman, Kendall no solo propuso una medida de asociación, sino que construyó todo el andamiaje probabilístico para su validación: la teoría de Kendall se aleja de la *distancia entre rangos* de Spearman y se basa en la probabilidad de ordenamiento [108]. Para Kendall, existen dos tipos de pares: los concordantes (C, aquellos donde si la abscisa y la ordenada del par mantienen la misma tendencia en la serie) y los discordantes (D, aquellos donde la tendencia se invierte para uno de ellos). El estadístico τ se define como la diferencia entre la probabilidad de que un par sea concordante y la de que sea discordante ver ecuación (8).

$$\tau = \frac{C - D}{\binom{n}{2}} = 2 \frac{C - D}{n(n-1)} \quad (8)$$

Con los mismos conjuntos de datos que en el ejemplo de Spearman, el orden que importa es el de X por convención y para decidir si un par es concordante (C) o discordante (D) se escribe:

Nombre	(X, Y)	A	B	C	D
A	(1, 20)	—	—	—	—
B	(2, 18)	D	—	—	—
C	(3, 25)	C	C	—	—
D	(4, 20)	—	C	D	—
E	(5, 20)	—	C	D	—

Por lo tanto, $C = 4$ y $D = 3$ y $\tau = \frac{2(4-3)}{5(4)} = 0,1$. Aquí, el coeficiente indica que la correlación es casi inexistente.

Los requerimientos de Kendall son los mismos que para Spearman y en el caso del p valor es la probabilidad de que $C - D$ tome un valor tan extremo o más extremo que el observado asumiendo que no hay asociación. Se rechaza H_0 si y solo si $p \leq \alpha$ [109].

Capítulo III. Marco tecnológico

En este capítulo se desarrollará todo lo concerniente a las aplicaciones tecnológicas involucradas con el trabajo: las relacionadas con la creación del ambiente distribuido (Citus), las relacionadas a la medición de consumo energético (Scaphandre y Prometheus) y la manipulación de los datos. La instalación de cada una de ellas se presenta a detalle en el apéndice B.

3.1. Ambiente distribuido

La implementación del entorno de pruebas se basa en la integración de tres componentes clave: PostgreSQL, como motor relacional de base de datos; Citus, como la extensión encargada de transformar dicho motor en un sistema distribuido mediante *sharding* (mecanismo descrito en la subsección 3.1.2) y procesamiento en paralelo; y DBGen, como la herramienta del *benchmark* TPC-H destinada a la generación escalable y controlada de la carga de datos. Esta combinación permite evaluar el comportamiento de consultas complejas sobre un clúster relacional, lo que facilita la medición del rendimiento y el consumo energético bajo condiciones de carga variables. La instalación de estas herramientas se detalla en el apéndice B. A continuación, se describen las especificaciones de cada componente.

3.1.1. PSQL

PostgreSQL es un sistema de gestión de bases de datos relacionales de código abierto y nivel empresarial. PSQL (PostgreSQL CLI) es la interfaz de línea de comandos para interactuar con PostgreSQL. Permite ejecutar consultas SQL complejas, gestionar privilegios y administrar la estructura de datos (esquemas) con un alto grado de cumplimiento de estándares y fiabilidad [28].

3.1.2. Citus

Citus es una extensión de código abierto para PostgreSQL que transforma este motor de base de datos en un sistema de base de datos distribuido. A diferencia de otras soluciones que requieren migrar a un motor nuevo, Citus extiende Postgres permitiendo la fragmentación horizontal de las tablas (*sharding*) a través de un clúster de múltiples nodos. Bajo este enfoque de arquitectura «nada compartido», una tabla lógica se divide en múltiples subtablas físicas (*shards*) distribuidas algorítmicamente en los nodos trabajadores mediante una clave de distribución basada en *hashing* [110]. El nodo coordinador de Citus se encarga de recibir las consultas, las fragmenta y las distribuye a múltiples nodos trabajadores que procesan los datos en paralelo. Asimismo, permite superar las limitaciones de cómputo y almacenamiento de un solo servidor solo con añadir más nodos al clúster. Es especialmente potente para cargas de trabajo analíticas (OLAP). Al ejecutar una consulta SQL,

Citus la descompone para que cada nodo trabaje sobre su fragmento de datos simultáneamente, lo que reduce drásticamente los tiempos de respuesta. [29, 30]

En el ambiente de Citus existen dos tipos de tablas: *reference tables* (replicadas en todos los nodos) y las *distributed table* (distribuidas por los nodos mediante una clave de repartición). Cuando dos tablas comparten la misma clave de repartición (a lo que se referirá como co-locación) todas los JOIN se pueden realizarse dentro de cada nodo y cuando no, se deben enviar los datos a través de la red (lo que se denominará particionamiento dinámico) para culminar la ejecución de los JOIN y enviar los resultados parciales al coordinador. El particionamiento dinámico también incluye el efecto de escribir los datos localmente en memoria o disco (en función del tamaño de estos) lo que tiene un impacto ineludible en el rendimiento de las consultas.

Se incluirán las tablas `lineitem.by_part` y `orders.by_customers` dentro del diseño de la base de datos que son réplicas de las tablas originales del modelo, `lineitem` y `orders` y cuya función es aplicar la co-locación como estrategia de diseño en el sistema distribuido.

3.1.3. DBGen

Es la herramienta de generación de datos que forma parte del *benchmark* TPC-H. Se utiliza para crear bases de datos sintéticas de gran volumen que simulan un entorno de soporte de decisiones (almacenes de datos). Permite escalar el conjunto de datos desde megabytes hasta terabytes de manera controlada [111].

3.2. Consumo

La medición de la energía en cualquier experimento no es directa y en el caso de energía eléctrica, menos. En la siguiente sección se describe cómo se puede aprovechar el mismo diseño de las computadoras para medir el consumo energético a través de sus circuitos y de capas de abstracción como Scaphandre y Prometheus.

3.2.1. RAPL y Scaphandre

En un circuito, la corriente y la diferencia de potencial se encuentran bien definidas a cada componente del circuito, por lo cual, la medida de la potencia entregada por cada componente es fácilmente medible por la medición de la diferencia de potencial y la corriente, tal como se detalló en la sección 2.2. No obstante, en un computador (y en cualquier dispositivo electrónico suficientemente complejo) este enfoque es impráctico (justamente por la necesidad de conectarse para medir físicamente las variables eléctricas implicadas con la potencia) y para resolver esta situación existen microcircuitos especializados en el conteo de energía y registrar sus medidas. Los procesadores INTEL poseen una interfaz de hardware *Running Average Power Limit* (RAPL) cuya función es monitorizar los contadores de energía del hardware y registrar el consumo [112]. La documentación de jRAPL provee información más precisa al respecto:

RAPL es un conjunto de interfaces de bajo nivel con la capacidad de monitorizar, controlar y recibir notificaciones de datos de energía y consumo de energía de diferentes niveles de hardware. Diseñado originalmente por Intel para permitir la gestión de energía a nivel de chip, RAPL es ampliamente compatible con las arquitecturas Intel actuales, incluyendo las CPU Xeon a nivel de servidor y los populares i5 e i7. Las arquitecturas compatibles con RAPL monitorizan la información de consumo de energía

y la almacenan en Registros Específicos de la Máquina (MSR). El sistema operativo puede acceder a estos MSR, como el módulo de kernel msr en Linux. RAPL es un diseño atractivo, especialmente porque permite reportar el consumo de energía /energía de forma detallada, por ejemplo, monitorizando el núcleo de la CPU, la CPU fuera del núcleo (caché L3, GPU en chip e interconexiones) y la DRAM por separado. jRAPL puede considerarse como un contenedor de software para acceder a los MSR. [87]

El consumo registrado por RAPL no es la medida de la potencia asociada a un procesador. En realidad, RAPL controla el registro del consumo total de energía de un computador y son variables como el tiempo de CPU que mantiene el *kernel* del sistema de operación, lo que permite que con unos cálculos adicionales se obtenga la potencia de todo el computador y de los procesos que en él corren. La interfaz que realiza estos cálculos es Scaphandre [113, 31].

Scaphandre es una interfaz que lee los registros de RAPL y entrega al usuario la potencia consumida, entre otras métricas, tanto del computador (*host*), un proceso (un programa corriendo y utilizando recursos) y un *socket* (el puerto donde se expone toda la información de Scaphandre) [113, 31].

3.2.2. Prometheus

Scaphandre puede correr de forma independiente en varios ordenadores y para monitorizar el consumo energético en estos, la herramienta Prometheus permite monitorear y recolectar las métricas de Scaphandre como un archivo JSON (*JavaScript Object Notation*). Prometheus es un conjunto de herramientas de código abierto para la supervisión y alerta de sistemas, desarrollado originalmente en SoundCloud (un servicio de transmisión, *streaming*, de la empresa alemana SoundCloud Global Limited & Co. KG.). Prometheus recopila y almacena sus métricas como datos de series temporales, es decir, la información de las métricas se almacena con la marca de tiempo en la que fueron registrados, junto con pares clave-valor opcionales denominados etiquetas [32]. La decisión del uso de esta herramienta obedece a la facilidad que ofrece para recolectar las métricas de forma simultánea.

3.3. Herramientas suplementarias

El procesamiento, limpieza y análisis de los datos recolectados se llevó a cabo mediante un conjunto de herramientas especializadas que garantizan la integridad de los resultados. Para la manipulación de archivos masivos y la automatización de tareas dentro del sistema operativo, se emplearon Sed y Coreutils, en conjunto con Jq para la extracción de métricas en formato JSON, descargadas desde Prometheus con Wget. La integración y ordenamiento inicial de los datos no procesados se realizó con Python, mientras que el análisis estadístico riguroso y la generación de representaciones gráficas de alta calidad se ejecutaron en R mediante el entorno R Studio.

3.3.1. Coreutils

GNU Core Utilities, es el paquete básico de herramientas de software de los sistemas operativos Unix y Linux. Contiene las utilidades esenciales para la manipulación de archivos, procesos y textos: **ls**, **cp**, **grep**, **cat**, entre muchos otros. Estos programas permiten la automatización de tareas mediante *scripts* de CLI [114].

3.3.2. Sed

Sed (*Stream Editor*) es un editor de flujo de texto que se usa para realizar transformaciones básicas en archivos. A diferencia de un editor de texto convencional, sed procesa los datos línea por línea de forma no interactiva. Su uso es para realizar limpieza masiva de datos (buscar y reemplazar) en archivos planos de gran tamaño sin necesidad de cargarlos en memoria [115].

3.3.3. Jq

Jq se define como el «sed para datos JSON». Es un procesador de texto ligero y flexible diseñado específicamente para filtrar, transformar y consultar datos en formato JSON. En arquitecturas modernas, es la herramienta estándar para extraer información de API o archivos de configuración complejos directamente desde la CLI [116].

3.3.4. Python

Es un lenguaje de programación de alto nivel, interpretado e imperativo. Es extremadamente versátil para la automatización, el *scraping* de datos y la integración de sistemas. Su uso fue para la automatización de la conversión no estadística de datos: extraer, agrupar y ordenar datos no procesados estadísticamente [117].

3.3.5. R y R Studio

R es un lenguaje especializado en la computación estadística y gráficos. A diferencia de Python, R fue diseñado por y para estadísticos, lo que le otorga una ventaja competitiva en el análisis de datos riguroso, la bioinformática y la visualización de datos de alta calidad. [118].

R Studio es el entorno de desarrollo integrado (IDE) estándar para el lenguaje R. Proporciona una interfaz gráfica amigable que incluye una consola, un editor de sintaxis que apoya la ejecución directa de código y herramientas para el trazado, el historial, la depuración y la gestión del espacio de trabajo [119].

3.3.6. Wget

Wget es una utilidad de línea de comandos para descargar archivos desde la web a través de protocolos como HTTP, HTTPS y FTP. Es capaz de realizar descargas recursivas y recuperar conexiones interrumpidas, lo que la hace ideal para la ingesta de conjuntos de datos remotos en entornos de servidor donde no hay una interfaz gráfica [120].

Capítulo IV. Estudio experimental

En el presente capítulo, se presentan los detalles de los experimentos de la investigación con especial atención a la arquitectura utilizada y la metodología de los experimentos. Con respecto a la arquitectura se presenta la descripción de los ordenadores y programas utilizados así como sus respectivas versiones. En la metodología experimental se presentan los procesos de población (carga) de la base de datos, recolección de medidas y tratamiento de los datos.

4.1. Arquitectura

Se utilizaron tres (3) computadoras con las siguientes características:

- Un nodo coordinador: Lenovo ThinkPad X250 con memoria RAM 8 GB y procesador Intel Core i5-5300U $\times 4$ @ 2,90 GHz con sistema de operación Ubuntu 24.04.3 LTS «noble». Memoria caché L1 (64 kB), L2 (512 kB) y L3 (4 MB).
- Dos nodos trabajadores: Lenovo IdeaPad 320-15IKB (Modelo 80XL) con memoria RAM 8 GB y procesador Intel Core i7-7500U CPU $\times 4$ @ 2.70GHz con sistema de operación Ubuntu 24.04.3 LTS «noble». Memoria caché L1 (64 kB), L2 (512 kB) y L3 (4 MB).

Los componentes de software utilizados dentro de la arquitectura fueron: Prometheus 2.42.0+ds como la pieza de supervisión. PSQL 16.10 y Citus como el motor del gestor de bases de datos y la capa de abstracción del ambiente distribuido. Scaphandre 1.0.0 como el componente de recolección de potencia de los procesos de PSQL y DBGen (de TPC-H) 3.0.0 como el generador de datos para poblar la base de datos.

4.2. Metodología experimental

La metodología de la investigación consiste en la automatización de las consultas y lectura de la potencia en cada nodo mediante Shell (CLI) para luego realizar el tratamiento de datos pertinente.

4.2.1. Carga de datos

Para evaluar el rendimiento en distintos escenarios de hardware, se seleccionaron volúmenes de datos que representan transiciones críticas en la jerarquía de memoria: 1 GB y 2 GB: Operación mayoritaria en caché y memoria volátil de alta velocidad. 5 GB: Escenario que reside íntegramente en la memoria RAM asignada. 10 GB y 20 GB: Volúmenes que exceden la RAM disponible, lo que fuerza operaciones de paginación y acceso recurrente a almacenamiento persistente (E/S de

disco). 50 GB: Escenario de alta carga diseñado para estresar el subsistema de disco y evaluar la degradación del rendimiento bajo saturación.

Para poblar las bases de datos, primero, se crearon las tablas dentro de las bases de datos (con el archivo `dss.ddl` de TPC-H, modificado para incluir las restricciones de clave primaria en cada tabla) luego, se generaron las cargas (`$CARGA`) de datos aleatorios con `dbgen` de TPC-H para cada volumen de datos seleccionados y se cargaron vía CLI. El procedimiento de generación es como se sigue:

1. Cargar las tablas a la base de datos
2. Replicar: `nation`, `region` y `supplier`.
3. Distribuir el resto de las tablas por su clave primaria.

4.2.2. Diseño físico

La evaluación experimental del sistema distribuido se fundamenta en la aplicación selectiva de tres técnicas de optimización sobre el conjunto de consultas seleccionadas del benchmark TPC-H. Esta estrategia se diseñó con el propósito de aislar y medir el impacto individual y combinado de las estructuras de acceso físico, la reescritura algorítmica y la topología de almacenamiento distribuido. Las intervenciones se clasifican de la siguiente manera: indexación asistida por inteligencia artificial, reescritura lógica mediante CTE y optimización de arquitectura por co-locación de tablas.

Indexación asistida por inteligencia artificial

Se diseñó una estrategia de indexación a medida para todas las sentencias analizadas, con excepción de la consulta Q1 (debido a su naturaleza analítica global que requiere una lectura completo de la tabla). Las estructuras de indexación óptimas fueron preseleccionadas mediante un modelo de lenguaje masivo (LLM), utilizando los planes de ejecución iniciales como contexto de entrada. Los índices resultantes se consolidaron en el script automatizado `indexes.sql`. En el flujo metodológico de las pruebas, la activación de estas estructuras auxiliares se encuentra parametrizada en el módulo de control del entorno de pruebas (Figura 4.3), permitiendo alternar de forma determinista entre los escenarios con y sin diseño físico.

Reescritura lógica mediante CTE

Para las consultas Q1, Q13 y Q16, se implementó una reescritura de código SQL basada en estructuras de tipo CTE. El objetivo de esta modificación es fragmentar la complejidad de las consultas anidadas originales, lo que facilita al optimizador del gestor la generación de subplanes de ejecución más eficientes y mitigando la sobrecarga en el uso de memoria RAM.

Optimización de arquitectura por co-locación de tablas

Las consultas Q18 y Q19 fueron seleccionadas exclusivamente para evaluar el impacto del acoplamiento físico en entornos distribuidos. Para estas sentencias, se modificó el esquema de particionamiento primario, forzando la co-locación de las tablas involucradas en las operaciones de JOIN. Esta técnica busca eliminar el cuello de botella que representa la redistribución dinámica de datos a través de la red, lo que permite que los nodos resuelvan las combinaciones localmente.

4.2.3. Medidas

Primeramente, desde todos los nodos se inicia **Scaphandre** en modo **prometheus** para que exponga sus métricas a **Prometheus**. Se verifica el estado de todos los nodos mediante una consulta **up** en la interfaz web. Finalmente, desde el coordinador se monitorizan y recolectan las métricas que Prometheus reúne como se sigue:

1. Consultar a Prometheus por las métricas mediante **wget**.
2. Mantener las métricas en un archivo intermedio para su posterior procesamiento.
3. Esperar dos segundos y repetir hasta recibir una señal terminación (**kill**).

Con el sistema recolectando métricas de forma exitosa, se procede a ejecutar las consultas, cada una con treinta repeticiones de acuerdo a la siguiente secuencia:

1. Seleccionar un factor de escala (1, 2, 5, 10, 20 o 50).
2. Generar los datos aleatorios con dicho factor. PostgreSQL puede necesitar una modificación postgeneración debido a un final de línea por defecto que tiene **dbgen**.
3. Cargar los datos a las tablas.
4. Si el escenario de estudio es con diseño físico, cargar los índices a la base datos.
5. Ejecutar cada consulta treinta veces, asegurando que PostgreSQL registre el tiempo de ejecución de forma explícita con el interruptor **timing on**.
6. Eliminar los índices de la base de datos si fueron creados.
7. Eliminar todos los datos y repetir con un factor de escala diferente.

4.2.4. Consumo energético

Para calcular el consumo energético de cada nodo en la base de datos, dado que el gestor solamente reporta el tiempo global de una consulta, mas no el tiempo que cada nodo realizó trabajo, se siguió el siguiente esquema: se midió la potencia entregada por cada nodo a espacio de dos segundos durante el tiempo de ejecución de la consulta y si la consulta reúne solamente una medida de potencia, se multiplica dicha potencia por el tiempo de ejecución global de la consulta como se indicó en la subsección 2.2.2, con $t_0 = 0$, t_f = tiempo reportado y $P(t)$ constante.

Si se reúne más de una medida de potencia, se integra numéricamente la potencia con el método del trapecio de la librería **numpy** de Python. El método del trapecio [92] consiste en que una función continua $f(t)$ puede ser muestreada en la sucesión de puntos $f(t_0), \dots, f(t_n)$. Los segmentos de $f(t)$ que unen a los puntos de la sucesión se aproximan con rectas y esto define una sucesión de trapecios, cuyas áreas acumuladas corresponden a una aproximación de la integral. Así, el i -ésimo trapecio está entre t_i y t_{i+1} con sus imágenes respectivas, $f(t_i)$ y $f(t_{i+1})$. El área de este trapecio es $\frac{1}{2}(f(t_i) + f(t_{i+1}))(t_{i+1} - t_i)$. Cuando todos los t_i están regularmente espaciados, es decir,

($\forall i | 0 \leq i \leq n : t_{i+1} - t_i = h$), el área del i-ésimo trapecio es $\frac{h}{2}(f(t_i) + f(t_{i+1}))$ y, finalmente, se puede decir que:

$$\int_{t_0}^{t_n} f(t)dt \approx \frac{h}{2} \sum_{i=0}^{n-1} (f(t_i) + f(t_{i+1})) = \frac{h}{2} \left(f(t_0) + 2 \sum_{i=1}^{n-1} f(t_i) + f(t_n) \right)$$

4.2.5. Tratamiento de los datos

De cada consulta se recolectó el tiempo de ejecución promedio así como la energía consumida cada dos segundos (obtenida por integración de la potencia con el tiempo mediante la regla del trapecio). Ambas métricas se analizaron con gráficos de barras para comparar el efecto de la estrategia de diseño sobre todas las consultas del *benchmark* para cada volumen de datos utilizado. Asimismo, se usaron gráficos de dispersión (apéndices E y F) y pruebas estadísticas para analizar la normalidad de los datos (Shapiro-Wilk) y la correlación entre tiempo de ejecución y consumo energético (Spearman y Kendall). Dichos análisis se realizaron con R y R-Studio.

Capítulo V. Resultados y discusión

En este capítulo se presentan los resultados de la investigación agrupados por secciones y en cada una se presentan los resultados con su respectivo análisis y una breve conclusión de ellos.

5.1. Tamaño de la base de datos

En esta sección se presenta el tamaño de la base de datos para cada carga de datos, donde se comparan los escenarios con y sin diseño físico. Dicha estrategia ya fue detallada en la subsección 4.2.2, mientras que los resultados de los tamaños se resumen en las tablas 5.1 y 5.2, respectivamente. En todas las regresiones presentadas, los números entre paréntesis representan el error estándar reportado por el método summary de R-Studio sobre la última cifra significativa de cada coeficiente.

Tabla 5.1: Espacio ocupado por las estructuras de la base de datos sin estrategia de diseño¹

Carga (GB) Tabla	0	1	2	5	10	20	50
Metadatos	10 188 kB	12	14	20	30	49	108
customer	1 536 kB	33	65	158	324	634	1 575
lineitem	256 kB	1 016	2 024	5 050	10 087	20 GB	49 GB
lineitem.by_part	256 kB	1 069	2 129	5 307	10 GB	21 GB	52 GB
nation ²	8 kB	24 kB					
orders	256 kB	244	484	1 190	2 374	4 736	12 GB
orders.by_customer	256 kB	244	482	1 190	2 371	4 733	12 GB
part	256 kB	38	74	188	369	732	1822
partsupp	256 kB	156	314	781	1 550	3 090	7710
region ²	8 kB	24 kB					
supplier ²	8 kB	2 056 kB	4 040 kB	10 072 kB	20	39	98
Espacio neto	1 864 kB	2 806	5 584	14 GB	27 GB	54 GB	135 GB

¹ Unidades en megabytes a menos que se indique lo contrario.

² Tablas de referencia (replicadas).

El valor de carga 0 GB indica el *overhead* inicial del esquema y los metadatos del sistema. Al realizar la regresión lineal del espacio neto (Y) vs. la carga (X) a partir de los datos de la tabla 5.1 la ecuación obtenida es (1). En esta se observa que el tamaño real de la base de datos es aproximadamente 2.7 veces el valor de los datos cargados, incremento atribuible a los índices de clave primaria de las tablas.

$$Y = 2,697(4)X + 0,12(9) \quad (1)$$

Tabla 5.2: Espacio ocupado por las estructuras de la base de datos con estrategia de diseño¹

Carga (GB) Tabla	0	1	2	5	10	20	50
Metadatos	10	14	17	28	44	76	175
customer	1 536 kB	261	110	267	539	1 062	2 642
lineitem	3 840 kB	3 759	7496	18 GB	37 GB	73 GB	183 GB
lineitem.by_part	1 024 kB	1 913	3814	9 517	19 GB	37 GB	93 GB
nation ²	32 kB	72 kB					
orders	1 792 kB	517	1028	2 545	5 081	10 145	25 GB
orders.by_customer	1 024 kB	447	859	2 021	3 962	7 804	19 GB
part	1 792 kB	104	203	505	1 001	1 991	4 964
partsupp	1 280 kB	261	522	1 299	2 584	5 153	13 GB
region ²	24 kB	56 kB					
supplier ²	32 kB	3 464 kB	6 792 kB	17	33	66	164
Espacio neto	12	7 069	14 GB	34 GB	68 GB	136 GB	339 GB

¹ Unidades en megabytes a menos que se indique lo contrario.

² Tablas de referencia (replicadas).

Análogamente, al aplicar la regresión lineal sobre los datos de la tabla 5.2, se obtiene la ecuación (2). Aquí se aprecia que el tamaño real de la base de datos es alrededor de 6.8 veces el valor de los datos cargados, reflejando el peso de las estructuras de optimización adicionales.

$$Y = 6,777(4)X + 0,21(9) \quad (2)$$

Para contrastar ambos escenarios, se realizó una regresión lineal del espacio neto con diseño (Y) vs. sin diseño (X) en la ecuación (3). Este análisis verifica que el impacto de la estrategia de diseño sobre el espacio en disco es de 2,5 veces, debido a la inclusión de índices secundarios.

$$Y = 2,513(5)X - 0,1(3) \quad (3)$$

Finalmente, se observa que aunque las estrategias de diseño físico optimizan el rendimiento de las consultas y, potencialmente, el consumo energético por operación, generan un impacto directo en el espacio ocupado. Este incremento conlleva un costo energético indirecto, al aumentar el volumen de estructuras que el sistema debe leer del almacenamiento persistente y cargar en la memoria principal.

5.2. Tiempos de ejecución

En esta sección se divide en subsecciones donde, primeramente, se encuentran las tablas con el resumen de tiempos de ejecución promedio con sus respectivas desviaciones estándar para las ejecuciones sin diseño y con diseño. Luego se muestran las gráficas de barras donde se comparan las ejecuciones sin diseño y con diseño para los diferentes volúmenes de datos, dos a dos, en orden y por último se muestra un mapa de calor con el resumen del impacto que tuvo la estrategia de diseño aplicada a cada consulta del *benchmark*.

5.2.1. Resumen de tiempos

En la tabla 5.3 se muestran los tiempos de ejecución promedio de las consultas sin (SD) y con (CD) estrategia de diseño físico, respectivamente. En dicha tabla, la primera observación que puede destacarse es que el coeficiente de variación puede superar el umbral de veinte por ciento y parece que este efecto disminuye con el incremento de la cantidad de datos a consultar. Tal efecto se asocia al diseño experimental donde no basta con medir sólo un tiempo por una cantidad de repeticiones sino que, además, hay que tomar lecturas de potencia de todos los nodos del clúster. Dicha medida de potencia está mediada por *Scaphandre* y *Prometheus* y estas herramientas no fueron diseñadas para medir a intervalos menores de un segundo (tiempo de ejecución de muchas consultas). En consecuencia, para alcanzar el mínimo de medidas válidas (lectura de tiempo con potencia registrada para cada nodo del sistema distribuido), se necesitaron muchas más repeticiones que mencionadas en el capítulo anterior. Esto se debió a que no todas las repeticiones válidas mantenían valores reproducibles (cuyo coeficiente de variación a lo sumo de diez por ciento¹) de tiempo. Queda como propuesta para estudios futuros estudiar otras herramientas de software y de hardware para realizar medidas cuando las ejecuciones duren fracciones de segundo.

5.2.2. Gráficos de barras: comparación por volumen de datos

En las figuras 5.1 a 5.3, se presentan los tiempos promedio de cada consulta mediante gráficos. Los tiempos se expresan en escala logarítmica para facilitar la apreciación de las diferencias entre los órdenes de magnitud. Esto implica que si n es la medida en la gráfica, el valor medido está en el intervalo $[1, 10) \times 10^{[n]}$. Bajo esta lógica, los valores de tiempo inferiores a un segundo se grafican como resultados negativos en el eje correspondiente.

En lo sucesivo, se adoptará una nomenclatura específica para referirse a las consultas analizadas. Cada consulta se identificará mediante un índice n (donde $1 \leq n \leq 22$), para englobar su comportamiento tanto con estrategia de diseño como sin ella (base). Para separar los escenarios experimentales, la notación Qn se referirá exclusivamente a la consulta ejecutada sobre el escenario base (sin diseño), mientras que $Qn(\text{estrategia})$ indicará la aplicación de una optimización específica. Como se detalla en la subsección 4.2.2, estas estrategias corresponden a reescrituras (CTE), creación de índices (idx) y co-localización de tablas ($\star$). Por ejemplo, hacer mención a la «consulta 3» implicará el estudio de la consulta número tres en líneas generales; en cambio, el uso de $Q3$ representará la consulta en su estado base y $Q3(\text{idx})$ especificará su ejecución tras la carga de índices. En todos los gráficos, las barras anaranjadas corresponden a las ejecuciones sin diseño y las azules, con diseño. Asimismo, las consultas se clasificaron en: consultas rápidas (menos de un segundo), intermedias (hasta cinco segundos) y pesadas (más de cinco segundos). El resumen de la clasificación para los gráficos se presenta en la tabla 5.4.

Se espera que la estrategia de diseño no incremente el orden de magnitud del tiempo de ejecución de la consulta, es decir, que mantenga el orden (una consulta que se tardaba cuarenta segundos, que se tarde veinte) o lo disminuya y que con el incremento del volumen de datos, se mantenga la tendencia de mejora en términos relativos al tiempo sin estrategia de diseño.

¹Este umbral permite distinguir las variaciones propias del comportamiento del motor de base de datos del ruido estocástico introducido por el sistema operativo o el hardware.

Tabla 5.3: Tiempos de ejecución en segundos en la base de datos sin diseño (SD) y con diseño (CD)¹

Carga (GB) Consulta	1 (SD)	1 (CD)	2 (SD)	2 (CD)	5 (SD)	5 (CD)	10 (SD)	10 (CD)	20 (SD)	20 (CD)	50 (SD)	50 (CD)
Q1	2,38(1)	1,46(9)	4,4(1)	2,61(8)	10,9(2)	6,24(6)	23(4)	19(3)	57(8)	49(3)	1,4(2) ³	128(8)
Q2	0,265(3)	0,28(6)	0,37(1)	0,24(2)	0,8(3)	0,8(6)	1,25(3)	1,4(3)	2,4(3)	0,9(3)	46(9)	2(2)
Q3	2,75(3)	3,04(5)	3,7(3)	2,9(2)	5,7(1)	5,3(8)	23,9(8)	9,7(5)	73(7)	4(1) ²	4,1(4) ³	1,6(1) ³
Q4	0,724(3)	0,3(1)	1,15(7)	0,31(3)	1,4(2)	2,4(4)	22(2)	12(3)	6(2) ²	24(3)	1,4(4) ³	79(7)
Q5	3,33(2)	3,3(5)	4,6(1)	3,6(2)	9,3(4)	6,5(5)	28(1)	1(2) ²	66(5)	18(2)	1,6(1) ³	1,3(1) ³
Q6	0,557(8)	0,27(6)	0,70(4)	0,25(1)	1,6(1)	1,2(1)	14(2)	0,6(1)	32,7(5)	1,01(2)	86(3)	2,24(3)
Q7	21(1)	21(1)	19,6(6)	19,0(5)	24,3(5)	26(1)	50(2)	30(1)	82(5)	48(3)	2,2(1) ³	111(8)
Q8	41(1)	41(2)	45(1)	44(1)	70(2)	76(3)	137(6)	1,3(1) ³	3,3(2) ³	3,5(1) ³	2,20(7) ⁴	1,26(3) ⁴
Q9	64(2)	64(2)	86(2)	86(3)	156(3)	174(6)	3,6(2) ³	4,4(2) ³	1,14(4) ⁴	1,14(4) ⁴	5,5(1) ³	5(2) ⁴
Q10	3,265(3)	3,6(9)	4,5(3)	3,9(2)	7,6(4)	8(1)	29(3)	12,6(8)	63(4)	25(2)	2,2(2) ³	1,2(1) ³
Q11	0,35(2)	0,37(7)	0,52(9)	0,33(4)	1,1(5)	2,1(5)	1,6(4)	1,1(8)	2,8(3)	1,0(2)	25,4(5)	2,84(2)
Q12	0,615(3)	0,29(6)	0,84(9)	0,33(2)	2,0(1)	1,3(1)	18(3)	0,90(5)	39,7(2)	8(2)	107(4)	52(5)
Q13	0,444(3)	0,6(2)	0,4(2)	1,0(4)	1,6(1)	1,60(9)	5(3)	6,6(8)	7,0(1)	14,2(3)	26(5)	26(4)
Q14	3,3(2)	2,8(7)	3,3(2)	2,7(2)	5,5(2)	4,1(8)	21(3)	5,9(3)	42,7(9)	11(1)	114(6)	26(2)
Q15	1,8(2)	1,14(4)	3,0(2)	1,87(8)	6,8(8)	5(1)	29,0(6)	9,4(4)	73,0(6)	18,5(7)	2,2(2) ³	49(2)
Q16	1,16(9)	1,08(6)	2,0(2)	1,9(1)	4,6(3)	4,2(8)	9,8(3)	9,5(2)	19,7(4)	21,2(9)	6(2) ²	6(2) ²
Q17	6,645(3)	0,2(1)	29,3(1)	0,17(2)	1(1) ³	0,2(1)	21(3)	0,4(3)	8(1) ²	0,4(3)	3,5(5) ³	0,5(4)
Q18	0,685(3)	0,57(4)	0,74(4)	0,63(3)	1(1) ³	0,8(1)	2,4(4)	1,16(6)	6(2)	2,10(2)	18(6)	4,38(9)
Q19	3,14(3)	0,2(1)	4,1(2)	0,21(4)	1,0(2)	0,5(4)	25(4)	0,3(2)	52(4)	0,4(2)	1,5(1) ³	0,6(5)
Q20	9,73(8)	0,35(8)	42,3(4)	0,35(6)	7,8(3)	1,4(8)	22(4)	0,9(6)	4(2) ²	1,05(1)	1,4(3) ³	6,34(9)
Q21	0,833(3)	0,74(5)	1,21(4)	1,11(6)	266,4(7)	2,6(2)	21,6(8)	17,2(5)	51(6)	44(3)	143(4)	120(6)
Q22	0,45(1)	0,5(1)	0,56(9)	0,44(4)	2,8(2)	1,6(9)	2(1)	2(3)	4(1)	2,43(1)	16(5)	6,4(2)

¹ Los números entre paréntesis son la desviación estándar sobre la última cifra significativa.² medida $\times 10^1$ ³ medida $\times 10^2$ ⁴ medida $\times 10^3$

Tabla 5.4: Clasificación de consultas según su tiempo de ejecución

Carga (GB)	Rápidas ($t < 1$)	Intermedias ($1 \leq t \leq 5$)	Pesadas ($t > 5$)
1	2, 4, 6, 11, 12, 13, Q17(idx), 18, Q19(*), Q20(idx), 21 y 22	1, 3, 5, 10, 14, 15, 16 y 19	7, 8, 9, Q17 y Q20
2	2, Q4(idx), 11, 12, Q13, Q17(idx), 18, Q19(*), Q20(idx) y 22	1, 3, 5, 14, 15, 16 y 21	7, 8, 9, Q17 y Q20
5	2, Q17(idx), Q18(*) y 19	Q1(CTE), 3, 4, Q5(idx), 6, 11, 12, 13, 14, 15, 16, Q20(idx), Q21(idx) y 22	Q1, Q5, 7, 8, 9, 10, Q17, Q18, Q20, Q21
10	Q6(idx), Q17(idx), Q19(*) y Q20(idx)	2, 11, Q13, Q18 y 2	1, 2, 3, 4, 5, Q6, 7, 8, 9, 10, Q12, Q13(CTE), 14, 15, 16, Q17, Q19, Q20, 21
20	Q2(idx), Q17(idx) y Q19(*)	Q2, Q6(idx), 11, Q18(*), Q20(idx) y 22	1, 3, 4, 5, Q6, 7, 8, 9, 10, 12, 13, 14, 15, 16, Q18, Q19, Q20 y 21
50	Q17(idx) y Q19(*)	Q2(idx), Q6(idx), Q11(idx), Q18(*), Q20(idx) y Q22(idx)	1, Q2, 3, 4, 5, Q6, 7, 8, 9, 10, Q11, 12, 13, 14, 15, 16, Q17, Q18, Q19, Q20, 21 y Q22

En la figura 5.1, se observa un predominio de consultas rápidas e intermedias (concordante con los datos de la tabla 5.4). Este comportamiento es esperado porque estos son los volúmenes de datos más pequeños, aún cuando estas están diseñadas para explotar diversos aspectos de los gestores de bases de datos. Resultan de especial interés las consultas pesadas, las cuales coinciden para este par de volúmenes de datos.

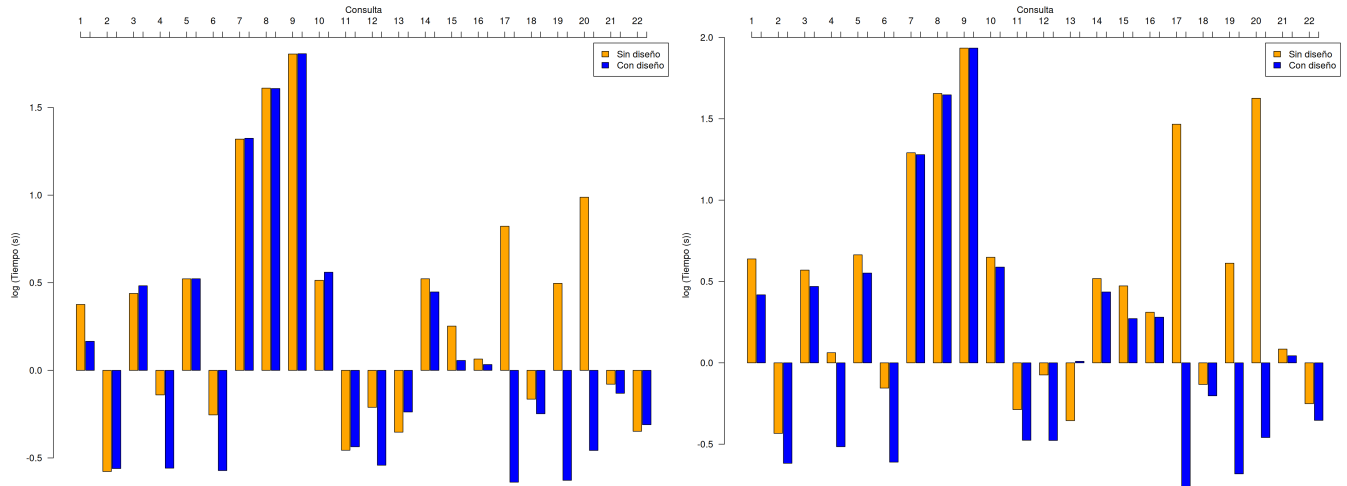


Figura 5.1: Tiempo por consulta 1 GB (izquierda) y 2 GB (derecha)

Para los casos específicos de Q7 hasta Q9, estas consultas se clasifican como pesadas por diseño, al involucrar operaciones de `JOIN` entre cinco (Q7 y Q9) o siete (Q8) tablas (ver tabla 2.1). En estas consultas se incluyen, además, las tablas de mayor volumen y se realizan agregaciones,

agrupamientos y ordenamientos, además de la aplicación de diversos filtros. Adicionalmente, estas consultas emplean particionamiento dinámico, por lo que presentan un retardo inherente a la latencia de la red.

Por otro lado, en Q17 y Q20, los cuellos de botella se localizan en las consultas anidadas correlacionadas que realizan agregaciones internas. A diferencia de los casos anteriores, aquí las tablas están co-locadas, lo que evita el particionamiento dinámico y el tráfico adicional de red. La estrategia de diseño fue el uso de índices, de los cuales —según el plan de ejecución entregado por el gestor— se usaron:

- Q17:
 - q17i2 ON lineitem_by_part (l_partkey, l_quantity, l_extendedprice)
 - q19i1 ON part (p_brand, p_container, p_size, p_partkey)
 - q20i1 ON lineitem_by_part (l_partkey, l_suppkey, l_shipdate, l_quantity)
- Q20:
 - q02i4 ON supplier (s_nationkey, s_suppkey)
 - q20i1 ON lineitem_by_part (l_partkey, l_suppkey, l_shipdate, l_quantity)
 - q20i2 ON partsupp (ps_partkey, ps_suppkey, ps_availqty)
- Para Q7, Q8 y Q9, debido a la naturaleza del particionamiento dinámico, el plan de ejecución proporcionado por el gestor no detalla las tareas locales realizadas de forma atómica en cada nodo; por lo tanto, no es posible verificar explícitamente el uso de los índices en los nodos remotos. No obstante, la reducción significativa en los tiempos de ejecución permite inferir que las estructuras de optimización fueron empleadas efectivamente durante el procesamiento local de los datos.

La nomenclatura de los índices corresponde a la consulta (q2, q17, q19 y q20) y para cada una, un índice propuesto (i1, i2, etc.), así, por ejemplo, q17i2 es el segundo índice propuesto para la consulta Q17. El impacto de estas estructuras es significativo para este volumen de datos, lo que permite que consultas originalmente pesadas se clasifiquen ahora como consultas rápidas.

En la figura 5.2, se presentan diagramas análogos a los de la figura 5.1 para las dos cargas intermedias (cinco y diez gigabytes). En estos se observa una transición —totalmente esperada— donde la cantidad de consultas rápidas disminuye en favor de las intermedias y las pesadas (ver tabla 5.4). Resulta relevante analizar con detenimiento aquellas consultas que, pese al incremento de carga, se mantienen dentro de la clase de consultas rápidas.

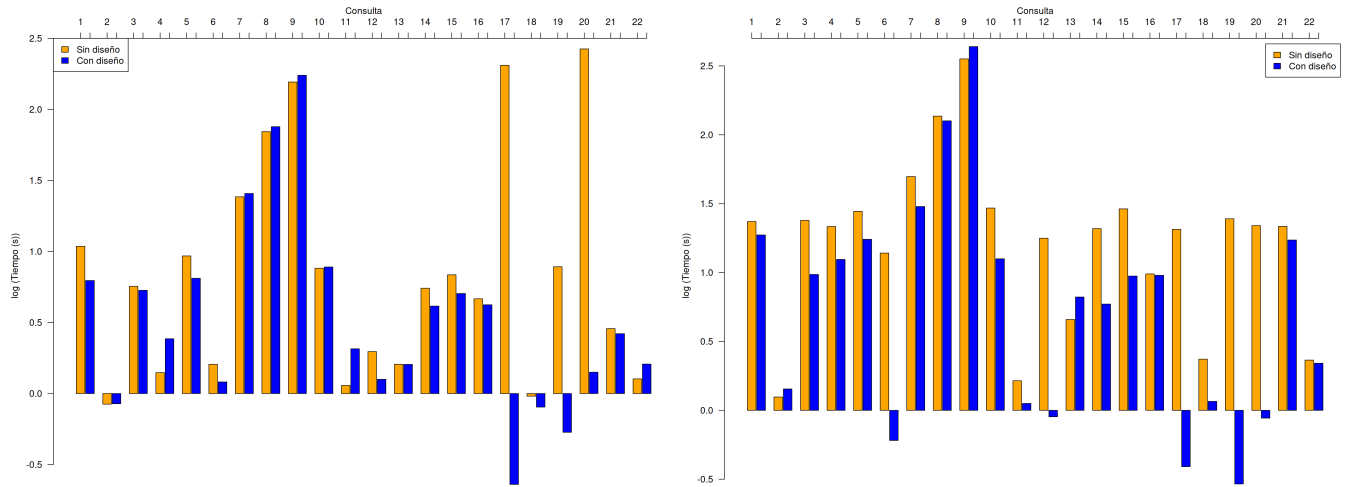


Figura 5.2: Tiempo por consulta 5 GB (izquierda) y 10 GB (derecha)

La consulta 2 es una consulta que tiene un filtro complejo de evaluar —debido al uso de un comodín al comienzo de un patrón de coincidencia—, una consulta anidada correlacionada y varios ordenamientos. Sin embargo, opera sobre tablas replicadas y dos tablas distribuidas co-locadas por diseño; por lo tanto, el procesamiento se realiza íntegramente en los nodos trabajadores sin transferencia de datos por la red. Q19, por su parte, tiene muchos filtros y, a pesar de que las tablas involucradas no están co-locadas, es altamente probable que cada nodo aplique primero todos los filtros de selección para enviar una cantidad de datos significativamente menor. Esto permite que, incluso con cinco gigabytes, la consulta se mantenga en la clase de las consultas rápidas. Respecto a las estrategias de diseño aplicadas, para Q2 y Q19 se crearon índices que, de acuerdo al plan de ejecución, el gestor utilizó de la siguiente forma:

■ Q2:

- q02i5 ON partsupp (ps_partkey, ps_supplycost, ps_suppkey)
- q05i4 ON supplier (s_suppkey, s_nationkey)
- q05i5 ON nation (n_nationkey, n_regionkey, n_name)
- q09i3 ON partsupp (ps_partkey, ps_suppkey, ps_supplycost)

■ Q19:

- q19i1 ON part (p_brand, p_container, p_size, p_partkey)
- q19i2 ON lineitem_by_part (l_partkey, l_quantity, l_shipmode, l_shipinstruct)

Adicionalmente, la consulta Q19(*) co-locas las tablas (al cambiar `lineitem` por `lineitem_by_part`) y este efecto, como se analizará más adelante, resulta determinante para el rendimiento al combinarse el uso de índices. Finalmente, en el escenario de diez gigabytes, aparece Q6(idx) dentro de la clase de consultas rápidas. Su ausencia en el volumen de cinco gigabytes se atribuye a la variabilidad intrínseca del sistema de operación durante el muestreo. Es probable que una mayor carga de procesos concurrentes en los nodos retrasara la ejecución en el primer escenario, mientras que en el segundo la consulta operó con menor interferencia.

La sexta consulta solamente trabaja sobre una única tabla, aunque se trata de la de mayor volumen: `lineitem`. Por lo tanto, el desempeño medido resulta consistente con su estructura, al igual que la mejora observada tras la aplicación de índices como estrategia de diseño. De acuerdo con el plan de ejecución proporcionado por el gestor, se utilizó el índice cobertor diseñado específicamente para la consulta: `q06i1 ON lineitem (l_shipdate, l_discount, l_quantity) INCLUDE (l_extendedprice)`.

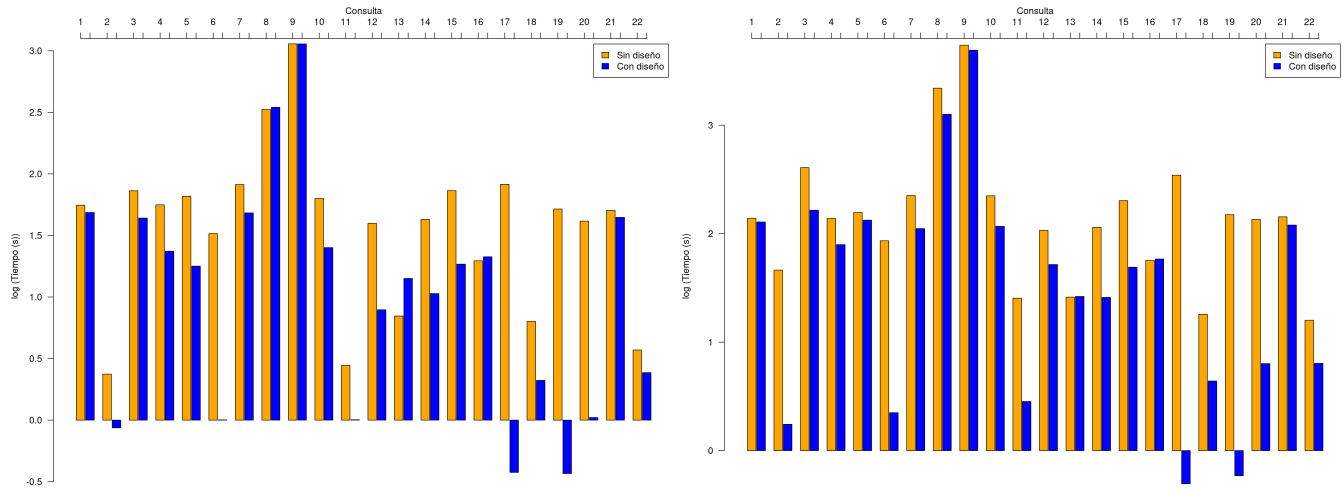


Figura 5.3: Tiempo por consulta 20 GB (izquierda) y 50 GB (derecha)

En la figura 5.3, se observa que la mayoría de las consultas transicionaron a la categoría de pesadas, aunque persistieron algunas intermedias y unas pocas, rápidas (específicamente en las que existe co-localización total los datos). En este punto, se analiza el comportamiento de las que se mantienen como intermedias. En el caso de la consulta 11, la ejecución incluye una consulta anidada correlacionada que compromete significativamente su rendimiento. La consulta 18, se aplicó una reescritura para evitar el particionamiento dinámico. Sin embargo, la presencia de una consulta anidada correlacionada obliga al gestor a realizar dos lecturas sobre la tabla de mayor volumen: `lineitem`. Finalmente, la consulta 22 emplea consultas anidadas y la función `SUBSTRING`, lo cual resulta costoso de evaluar sobre grandes volúmenes de datos sin el apoyo de índices para el filtrado. De acuerdo a los planes de ejecución de estas consultas a excepción de la 22, usaron los siguientes índices:

- Q11:
 - q02i4 ya estudiado.
 - q11i1 ON partsupp (ps_suppkey, ps_partkey, ps_supplycost, ps_availqty)
- Q18:
 - q18i1 ON lineitem (l_orderkey, l_quantity)
 - q18i3 ON customer (c_custkey, c_name)

La mejora en el desempeño de la Q22(idx) se atribuye a que, tras la creación del índice sobre el atributo `c_phone`, el gestor actualizó automáticamente las estadísticas de dicha columna a través de sus procesos internos [121]. Por otro lado, resulta llamativo el comportamiento observado en

el escenario de cincuenta gigabytes para las estrategias de diseño físico; en particular, destacan aquellas consultas donde el impacto de la optimización no es determinante: 1, 5, 9, 13, 16 y 21.

Para la consulta 1, a pesar de que lee únicamente a `lineitem`, el filtro aplicado es poco selectivo, lo que obliga a leer casi la totalidad de la tabla y realizar múltiples cálculos. En consecuencia, incluso con la reescritura propuesta, la mejora no es significativa debido al alto volumen de datos procesados. Por su parte, la consulta 5 comparte con las consultas 7, 8 y 9 la complejidad derivada de la cantidad de operaciones de `JOIN` y la ausencia de co-localización que penaliza el rendimiento. Aunque el plan de ejecución de `Citus` no permite verificar explícitamente el uso de los índices creados debido a la falta de soporte para consultas con particionamiento dinámico, se infiere que la disminución en los tiempos de ejecución es consecuencia directa de su aplicación [121].

En el caso de la consulta 13, el cuello de botella reside en la consulta anidada correlacionada que realiza una unión con una tabla co-locada bajo una condición `NOT LIKE`. Tal condición también posee un comodín al inicio del patrón de coincidencia y ambas situaciones invalidan cualquier índice propuesto. Como estrategia de diseño, se empleó una expresión de tabla común (CTE) no materializada, aunque esta no contribuyó a mejorar el rendimiento para cargas menores a cincuenta gigabytes.

Finalmente, para la consulta 16, el rendimiento se ve afecto por las cláusulas `textttNOT IN` y `NOT LIKE`. Se propuso una reescritura con CTE no materializada y el uso de índices de texto (`q16i3 ON supplier USING GIN (s_comment gin_trgm_ops)`). Si bien esta aproximación reduce la carga computacional, se requiere un estudio más profundo para optimizar el desempeño. Por último, la consulta 21 presenta dos consultas anidadas que actúan como `SEMI JOIN` y `ANTI JOIN` sobre la tabla más grande: `lineitem`.

Tal y como se pudo constatar en los resultados, las dos estrategias aplicadas tuvieron al menos un impacto positivo sobre el rendimiento de las consultas estudiadas, al menos, en términos cualitativos. Queda por estudiar el impacto en términos cuantitativos en la subsección siguiente para confirmar el efecto que puede tener una buena estrategia de diseño y posteriormente comparar el efecto de esta optimización contra el consumo energético de la consulta. La consulta que no tuvo un impacto positivo fue la decimotercera consulta debido al filtro `NOT LIKE` con comodín al inicio.

5.2.3. Resumen del impacto de la estrategia de diseño

En la figura 5.4, se muestra el porcentaje de mejora en el rendimiento de las consultas por la estrategia de diseño aplicada. Se observan tendencias no monótonas que se explicarán en las siguientes líneas. Sin embargo, antes de hacer los análisis por consulta, conviene separar los volúmenes de datos en el mapa de calor como zonas donde se puede trabajar en memoria caché (1 GB), en memoria RAM (2 GB a 5 GB) y zonas de desborde de la RAM (10 GB a 50 GB), donde predomina el acceso al almacenamiento persistente.

Para la consulta 1, se observa que la estrategia de diseño logra una mejora en el desempeño de la consulta en todos los casos. La mejora es fuertemente dependiente de la región de memoria en donde se ubica el volumen de datos utilizados y alcanza su máximo cuando la RAM está a su límite (cinco gigabytes). Dichas mejoras describen una tendencia que se alinea con el comportamiento descrito en la subsección 2.4.1. Por un lado, con pocos datos (uno y dos gigabytes), la mejora dada por la estrategia de diseño se ve opacada por la velocidad de procesamiento de la memoria caché. Por otro lado, cuando la RAM se desborda y los datos necesitan leerse del disco (diez gigabytes en adelante), la estrategia pierde efectividad por el volumen de datos manejado.

A diferencia de la tendencia observada en el caso anterior, la consulta 2 presenta un comportamiento fluctuante no monótono que evidencia el impacto de las decisiones del optimizador de

consultas ante el escalado del volumen de datos. En los extremos de baja carga (uno y cinco gigabytes), el impacto de la estrategia es neutro, y hay un repunte aislado a los dos gigabytes. Sin embargo, al llegar a diez gigabytes, se produce una regresión en el desempeño ($-14,47\%$) que se revierte al escalar a cargas masivas de veinte (63,36 %) y cincuenta (96,21 %) gigabytes.

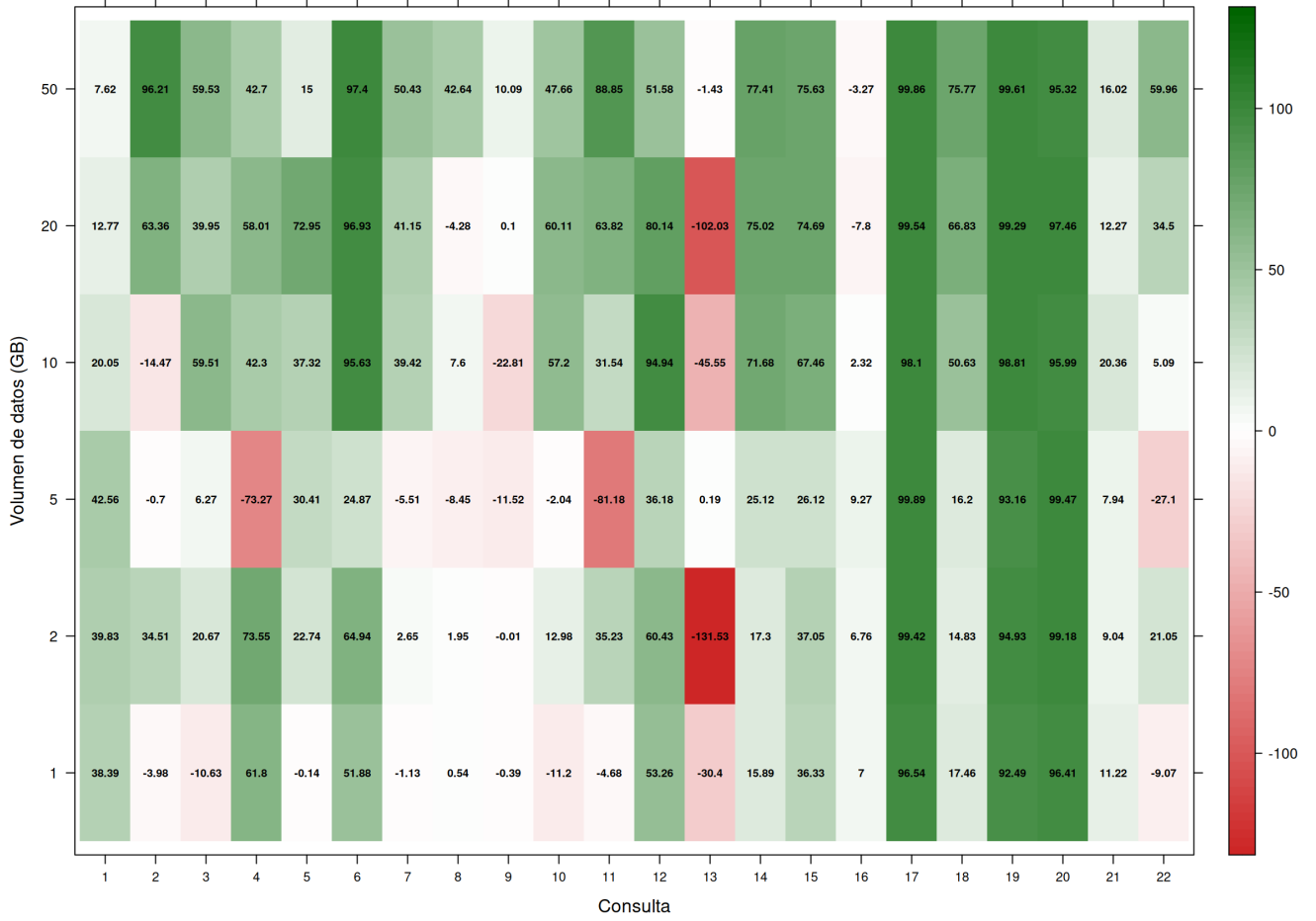


Figura 5.4: Mapa de calor de mejoras en rendimiento de las consultas

El impacto neutro se debe a que el gestor, con base en sus estadísticas, decidió ignorar los índices propuestos y continuar con el plan en estado base. La mejora aislada a los dos gigabytes se explica porque los datos empiezan a desbordar la memoria caché y ese intercambio permite ver que la estrategia tiene éxito. Con diez gigabytes, el gestor cambió el plan de ejecución de lecturas secuenciales a ciclos anidados (*nested loops*). Las lecturas secuenciales se optimizan en el manejo interno tanto del gestor como del sistema operativo de forma nativa [100]. Por su parte, los ciclos anidados rompen la secuencialidad y fuerzan los accesos aleatorios al disco, con los consecuentes *buffer misses* (subsección 2.4.1) y el respectivo incremento de la latencia. Sin embargo, cuando el volumen de datos se incrementa hasta los límites del almacenamiento (veinte y cincuenta gigabytes), incluso los accesos aleatorios se vuelven más eficientes, a pesar de los *buffer misses* debido al descarte de páginas que vienen de los ciclos anidados.

Para las consultas 3, 5, 7, 8, 9, 10 y 14, las cuales realizan particionamiento dinámico, se observa que, con un gigabyte, la estrategia de diseño no aporta mejoras e incluso llega a empeorar

el desempeño (casos de la 3 y la 10). Esto ocurre porque el uso de índices introduce un *overhead* que no supera la eficiencia de métodos más simples, como la lectura secuencial, cuando los datos residen íntegramente en la memoria caché. Con dos gigabytes, la estrategia comienza a ser efectiva para consultas ligeras (3, 5, 10 y 14). Sin embargo, en consultas más complejas como 7, 8 y 9, el beneficio sigue siendo opacado por el costo del particionamiento.

Al alcanzar los cinco gigabytes, con la RAM cerca de su límite, sólo 5 y 14 mantienen un crecimiento en su mejora, mientras que las demás sufren una degradación por la denominada tormenta de accesos aleatorios. Con diez gigabytes, a pesar de la necesidad de acceder al disco, las mejoras aumentan para casi todas las consultas, con excepción de la 9, cuyo volumen de lectura obligatorio y accesos aleatorios penalizan el tiempo de respuesta.

Con veinte gigabytes, todas presentan mejoras salvo la 3 y la 8, probablemente debido a que el tamaño de los índices excede la memoria disponible. Finalmente, con cincuenta gigabytes y la capacidad física de los nodos al límite, los índices logran apoyar al sistema estresado. No obstante, en la 5 y la 9, la mejora es marginal debido a su baja selectividad y al excesivo tráfico de red generado. Cabe destacar que la 14 fue la única consulta que exhibió una mejora monótona a través de todos los volúmenes de datos.

Un segundo bloque de consultas, integrado por los casos 4, 6, 11, 12 y 15, evidencia una vulnerabilidad crítica compartida a la escala de los cinco gigabytes. En este punto de inflexión, lo que coincide con la saturación del espacio de trabajo en la memoria RAM, todas ellas sufren una regresión en su desempeño. Este fenómeno colectivo responde a la introducción de la tormenta de accesos aleatorios; a esta escala intermedia de datos, el costo de recuperar páginas dispersas mediante las nuevas estructuras de indexación resulta incapaz de superar la velocidad y eficiencia de las lecturas secuenciales que el gestor realizaba originalmente [100].

A pesar de este bache común, la evolución de estas consultas se bifurca al escalar el volumen de datos. Por un lado, las consultas 6, 11 y 15 muestran un comportamiento homólogo posterior: tras el impacto adverso a los cinco gigabytes, inician un crecimiento sostenido a partir de los diez gigabytes. Una vez que la RAM se desborda por completo y el disco pasa a dominar los tiempos, la selectividad de la estrategia de diseño supera con creces las penalizaciones por acceso aleatorio del plan base. Por otro lado, la consulta 12 presenta una anomalía tardía a los veinte gigabytes, atribuible a que el tamaño crítico de sus índices específicos excede la capacidad de la memoria disponible para dicha operación, lo que replica el mecanismo de desborde analizado previamente en el bloque de consultas que realizan particionamiento dinámico.

Las consultas 13, 16 y 22 representan los escenarios donde decisiones de diseño físico no alcanzaron la optimización esperada. En la 13, la introducción de una CTE no materializada provocó que el optimizador del gestor seleccionara rutas de ejecución subóptimas; de hecho, los rendimientos más bajos se registraron justamente al manifestarse las limitaciones físicas del hardware. En cuanto a consulta 16, centrar el diseño en el filtro `LIKE` con los comodines iniciales demostró no atacar el cuello de botella principal, el cual residía realmente en las tablas `partsupp` y `part` debido a su volumen significativamente mayor en comparación con `supplier`.

Finalmente, respecto a la consulta 22, la estrategia exhibió un impacto intermitente. A la escala de un gigabyte, el rendimiento disminuyó debido a la superioridad de las lecturas secuenciales en caché frente a los accesos aleatorios del índice. A los diez gigabytes, este fenómeno de penalización se repitió: a pesar de que la tabla `customer` aún residía íntegramente en memoria, el costo computacional de los accesos aleatorios forzados por la consulta al índice superó el beneficio de la estructura física, lo que degradó el tiempo de respuesta frente a la lectura secuencial base.

Las consultas 18 y 21 comparten tendencias similares, evidenciadas por mejoras inferiores al veinte por ciento en el escenario de un gigabyte, causadas por la velocidad de la memoria caché

frente a hacer búsqueda indexada. A los dos gigabytes, su porcentaje de mejora disminuye por el incremento de accesos aleatorios. Sin embargo, a los cinco gigabytes —con la memoria al límite de su capacidad— el efecto del diseño se hace notar, a pesar de que el gestor seleccionó una ruta de ejecución poco favorable. Posteriormente, a los diez gigabytes, la mejora se recupera, aunque de forma moderada debido al beneficio intrínseco de las lecturas secuenciales para este perfil de consultas.

A los veinte gigabytes, los comportamientos divergen: la 18 mejora mientras que la 21 no. La 18 progresa porque sus índices aún caben en RAM y su acceso aventaja a la lectura secuencial en disco. En contraste, la 21 se degrada porque, con ese volumen de datos y con la obligación de leer tres veces la tabla `lineitem`, el rendimiento colapsa por la densidad de accesos aleatorios al disco. No obstante, aunque ambas consultas elevan su mejora a los cincuenta gigabytes (con el hardware saturado), el beneficio en la consulta 18 supera en cinco veces al de la 21.

Finalmente, las consultas 17, 19 y 20 constituyen los escenarios donde la estrategia de diseño físico alcanzó su mayor impacto positivo, registrando optimizaciones cercanas al cien por ciento en todos los volúmenes estudiados. Este comportamiento excepcional y uniforme se fundamenta en la naturaleza específica de los mecanismos implementados. En los casos de la 17 y la 20, la incorporación de índices cobertores permitió al gestor resolver las demandas mediante lecturas exclusivas de índice, lo que elimina por completo la necesidad de acceder a las páginas de la tabla base. Por su parte, la consulta 19 capitalizó la co-localización física de las tablas `part` y `lineitem` (`lineitem_by_part`), lo que suprimió el costo de transferencia y reordenamiento de datos a través de la red durante las operaciones de unión (`JOIN`).

De este modo, el beneficio de estas estructuras es tan determinante que vuelve al plan de ejecución inmune a las fluctuaciones de la memoria caché o el disco. Por lo tanto, más allá de las causas que gobiernan cada región del mapa de calor, es altamente probable que las sutiles diferencias en los porcentajes calculados para este grupo correspondan a la variabilidad propia del entorno experimental y no sean estadísticamente significativas.

En síntesis, el comportamiento de las consultas refleja que el escalado en el volumen de datos dicta una progresión ascendente en las demandas temporales del sistema. Ante este escenario, las estructuras de diseño físico implementadas logran mitigar este impacto de forma estadísticamente significativa en la mayor parte del espectro experimental, demostrando la viabilidad de la propuesta con la salvedad de las anomalías estructurales identificadas en los casos de las consultas 13, 16 y 22.

No obstante, este beneficio en la velocidad de respuesta constituye solo una dimensión del impacto arquitectónico; como señala la literatura científica especializada, la optimización del tiempo de ejecución no se traduce necesariamente en una reducción del consumo energético [7, 8, 9], fenómeno que se analiza en la sección siguiente.

5.3. Consumo energético neto

Análogamente al análisis del tiempo de ejecución, se presenta en la tabla 5.5 el consumo energético estimado para cada consulta con su media y desviación estándar. Asimismo, en las figuras 5.5 a 5.7, se muestran los gráficos logarítmicos de consumo neto para las consultas en los diferentes volúmenes de datos.

5.3.1. Resumen de consumo neto

El consumo neto se calculó como la suma del consumo energético de cada nodo, mientras que la desviación estándar correspondiente se determinó mediante aplicación de la teoría de propagación de errores para cantidades aditivas [122] (ver ecuación (4)).

$$U_{neto} = \sum_{i=1}^n U_i \Rightarrow \sigma_{U_{neto}} = \sqrt{\sum_{i=1}^n \sigma_{U_i}^2} \quad (4)$$

Bajo un enfoque estrictamente teórico, la hipótesis que guía este análisis plantea que la estrategia de diseño físico disminuirá el consumo energético de las consultas y que, con el incremento del volumen de datos, se mantendrá la tendencia de mejora en términos relativos al escenario base sin optimizaciones.

Sin embargo, al contrastar esta expectativa con los valores empíricos de la tabla 5.5, la primera observación destacable es que el consumo energético presenta un coeficiente de variación predominantemente superior o igual al treinta por ciento. Dicho umbral se considera como el rango de alta volatilidad [123, 124]. En efecto, esta condición se verifica en 134 de los 264 escenarios estudiados. Este fenómeno no responde a una falta de consistencia en las estrategias de diseño, sino a las limitaciones del muestreo para consultas con tiempos de ejecución menores a la unidad. Al ejecutarse las consultas en los escenarios con menor volumen de datos (uno y dos gigabytes) en menos de un segundo, la baja resolución temporal de las herramientas introdujo un ruido severo —particularmente en el nodo coordinador debido a su rol de planificación—, el cual se propagó de forma incremental hacia la desviación del consumo neto final según la ecuación (4).

Al analizar la progresión de los datos en escenarios de mayor escala, se observa un incremento abrupto en los valores a partir del umbral de los diez gigabytes en consultas como la 4 y la 13. Es crucial destacar que este comportamiento no representa la presencia de valores atípicos, sino que refleja fielmente el quiebre operacional provocado por la transición hacia la lectura de disco. Al saturarse la memoria RAM disponible, el esfuerzo del hardware para solventar las operaciones de E/S en disco dispara la demanda energética en consonancia directa con el escalado del tiempo de ejecución.

A su vez, el comportamiento de la consulta 16 en el escenario de veinte gigabytes merece una mención analítica particular, dado que registra un consumo masivo (18 971 J) con una desviación estándar extremadamente baja (± 7 J). La consistencia de esta varianza demuestra que no se trata de una anomalía estadística, sino de una decisión totalmente determinista del optimizador de consultas. Este fenómeno, conocido como un atrapamiento por costo del optimizador [94], ocurre por la combinación de una estrategia de indexación subóptima (índices sobre la tabla *supplier* en lugar de *part* o *partsupp*) y la estructura interna de Q16, la cual emplea predicados *LIKE* con comodines iniciales.

A la escala de veinte gigabytes, los estimadores de costo del motor determinan erróneamente que el uso del índice disponible es ventajoso fuerzan un plan de ejecución degradado (como un bucle anidado o una lectura completa de índice) que se repite de forma idéntica en cada iteración. Paradójicamente, al escalar a cincuenta gigas, el volumen de datos supera el umbral de tolerancia del optimizador, lo que le obliga a abandonar dicho índice y conmutar hacia una lectura secuencial paralelo o un acoplamiento por hash (*hash join*). Esta reestructuración del plan a gran escala resulta energéticamente más eficiente que el bucle catastrófico del escenario de veinte gigabytes, validando la consistencia interna del motor relacional.

Por esta razón, formular aseveraciones deterministas sobre el impacto absoluto de la estrategia en el ahorro energético global a partir de estos valores promedios podría inducir a conclusiones sesgadas por el entorno experimental. En consecuencia, la evaluación de este comportamiento se aborda mediante un análisis de correlación no paramétrico (ver la sección 5.4), el cual permite aislar la tendencia macro frente al ruido. Asimismo, se establece como trabajo futuro y necesidad metodológica explorar herramientas de alta frecuencia capaces de realizar mediciones energéticas precisas en ejecuciones sub-segundo.

5.3.2. Gráficos de barras: consumo contra volumen de datos

En la figura 5.5, se presentan dos diagramas de barras totalmente análogos a los diagramas de tiempo estudiados en la sección anterior. En tal sentido, se pueden distinguir de tres categorías de consultas: las que consumen poco (fracciones de joules), las de consumo moderado (menos de diez joules) y las de consumo alto (más de diez joules). El resultado de esta clasificación se muestra en la tabla 5.6.

Debido al volumen de datos, es esperado que la mayoría de las consultas registren consumos correspondientes a fracciones de joules, un comportamiento consistente con tiempos de ejecución inferiores a un segundo. Por lo tanto, el foco del análisis se centra en las consultas de consumo moderado y alto.

Tabla 5.5: Consumo energético neto en joules sin diseño (SD) y con diseño (CD) físico¹

Carga (GB) Consulta	1 (SD)	1 (CD)	2 (SD)	2 (CD)	5 (SD)	5 (CD)	10 (SD)	10 (CD)	20 (SD)	20 (CD)	50 (SD)	50 (CD)
Q1	0,2(5)	0,1(3)	1(1) ²	3(5)	6(2) ²	2(1) ²	1,9(3) ³	8(2) ²	4,5(4) ³	2,0(3) ³	1,18(5) ⁴	5,5(3) ³
Q2	0,027(2)	0,016(5)	0,04(1)	0,020(4)	0,05(2)	0,02(1)	0,02(2)	0,07(3)	1(1)	0,05(2)	5(1) ²	27(4)
Q3	1(1)	1,2(8)	0,6(2)	1,1(9)	4(4)	3(2)	5(1) ²	2(2) ²	1,2(2) ³	1,0(2) ³	3,1(2) ³	3,1(3) ³
Q4	0,055(4)	0,019(5)	0,09(1)	0,037(8)	0,10(3)	0,1(1)	22(4)	4(2)	65(7)	20(6)	163(9)	1,2(1) ³
Q5	2(2)	1(1)	2(3)	1(1)	2(2) ²	4(2)	90(2) ²	3(3) ²	1,8(1) ³	1,8(3) ³	2,3(2) ³	1,9(2) ³
Q6	0,08(2)	0,018(4)	0,08(3)	0,020(5)	0,1(2)	0,03(2)	12(5)	0,03(2)	30(3)	0,02(1)	93(6)	0,03(6)
Q7	2(2)	2(1)	4(4)	2(1)	1(2) ²	7(7)	37(9)	2(2) ²	89(8)	6(3) ²	2,3(3) ³	2,6(4) ³
Q8	10(7)	9(5)	2(1) ²	3(2) ²	6(3) ²	7(4) ²	1,9(4) ³	1,8(4) ³	4,0(4) ³	3,8(4) ³	9,7(6) ³	9,8(5) ³
Q9	3(2) ²	3(1) ²	7(4) ²	8(4) ²	2,4(4) ³	2,5(4) ³	4,8(4) ³	4,4(4) ³	9,8(5) ³	9,6(4) ³	2,24(9) ⁴	1,9(2) ⁴
Q10	1,2(8)	1,5(9)	1(1)	1,1(6)	4(2)	3(1)	3(1) ²	9(2)	80(9)	3(1) ²	2,7(2) ³	1,0(3) ³
Q11	0,029(5)	0,015(3)	0,029(7)	0,019(3)	0,05(3)	0,09(4)	0,1(3)	0,03(4)	0(1)	0,1(1)	3(1) ²	0,08(1)
Q12	0,085(4)	0,022(5)	0,10(2)	0,08(1)	0,2(5)	0,02(3)	20(5)	0,02(2)	43(3)	6(6)	125(6)	9(1) ²
Q13	0,037(4)	0,1(1)	0,04(2)	0,3(8)	0,06(2)	0,05(2)	3(9)	1(3) ⁵	3(2) ²	4(3) ²	1,6(3) ³	1,3(6) ³
Q14	1,0(8)	1,2(7)	1,1(7)	9(8)	3(3)	1,1(7)	22(4)	4(2)	47(4)	9(4)	1,2(1) ³	2(1) ²
Q15	0,07(1)	0,026(4)	0,4(9)	0(1)	9(8)	2(1)	5(2) ²	8(2)	1,1(1) ³	4(2) ²	3,0(3) ³	9(3) ²
Q16	0,029(5)	0,026(4)	0,04(2)	0,04(2)	1,9(2)	2,53(7)	8(2)	7(1)	3(1) ²	18 971(7)	1,3(3) ³	7(3) ²
Q17	2(1) ²	0,013(7)	2,9(2) ³	2,9(2) ³	2,30(4) ⁴	0,012(6)	24(6)	0,012(5)	89(6)	0,03(2)	4,8(8) ³	0,04(1)
Q18	0,052(8)	0,05(1)	0,056(8)	0,05(2)	0,05(2)	0,01(1)	0,06(4)	0,03(3)	5(4)	0,04(6)	3(1) ²	1(5)
Q19	0,8(6)	0,014(5)	2(2)	0,014(5)	0,05(2)	0,03(1)	39(7)	0,03(2)	76(7)	0,03(3)	2,2(1) ³	0,045(8)
Q20	5(2)e1	0,03(3)	4,6(2) ³	0,021(6)	10(6)	0,02(2)	24(7)	0,03(3)	6(1) ²	0,03(5)	1,7(1) ³	4(7)
Q21	0,093(9)	0,10(2)	0,09(2)	0,10(2)	3,27(3) ⁴	1(3)	4(1) ²	3(1) ²	87(8)	68(6)	2,6(1) ³	174(7)
Q22	0,024(2)	0,022(3)	0,028(6)	0,030(6)	1(2)	0,1(1)	0,2(4)	1(2)	2(3)	1(3)	17(8)	18(9)

¹ Los números entre paréntesis son la desviación estándar sobre la última cifra significativa.

² medida $\times 10^1$

³ medida $\times 10^2$

⁴ medida $\times 10^3$

⁵ medida $\times 10^4$

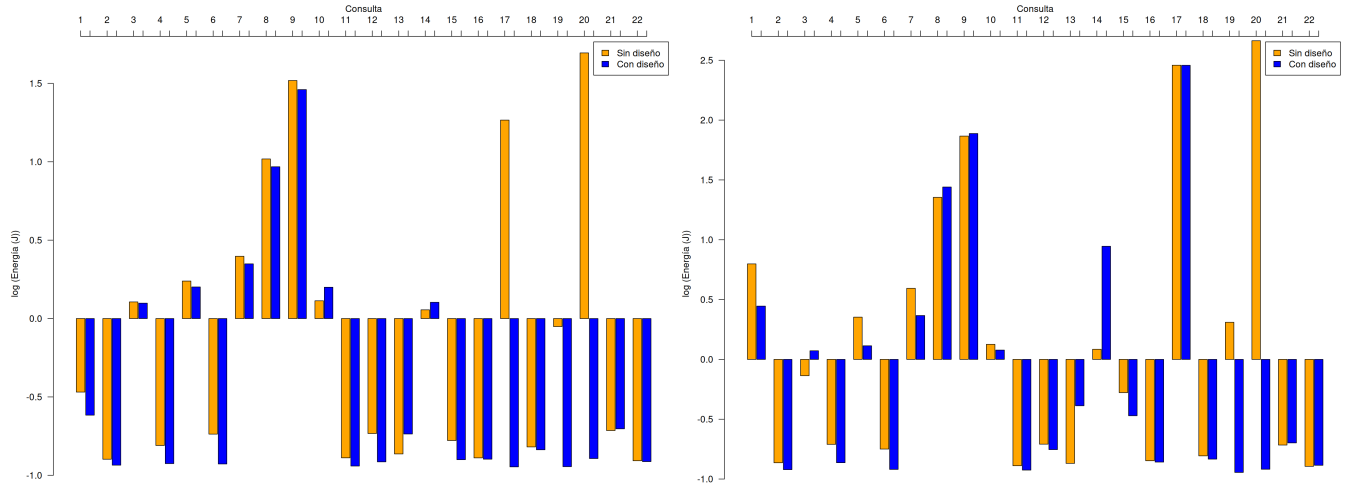


Figura 5.5: Consumo neto por consulta 1 GB (izquierda) y 2 GB (derecha)

El gasto energético de la consulta 3 está condicionado por el costo del particionamiento dinámico al realizar la unión de tablas no co-locadas. La estrategia de diseño físico implementada para esta operación consistió en la creación de índices, pero debido al particionamiento dinámico, el plan de ejecución global no reporta las tareas hechas por los nodos trabajadores. A pesar de esta limitación de observabilidad en el nodo coordinador, la disminución registrada en el tiempo de ejecución permite inferir que los optimizadores locales de los nodos trabajadores utilizaron efectivamente las estructuras indexadas [125, 126].

Este uso se evidencia de forma analítica al contrastar las escalas de uno y dos gigabytes (ver figura 5.5). Mientras que a un gigabyte se registra una ligera atenuación energética, el escenario de dos gigabytes con estrategia de diseño exhibe un incremento en el consumo respecto al caso base. Este comportamiento constituye el indicador físico de la activación del índice: al duplicarse el volumen, el patrón de acceso aleatorio inherente a la búsqueda indexada desborda las líneas de la memoria caché, provocando un incremento severo en los fallos de búfer (*buffer misses*) en comparación con la predictibilidad y eficiencia del pre-procesamiento lineal de una lectura secuencial. Por lo tanto, el aumento energético no deviene de la inactividad de la estrategia, sino del costo de hardware asociado al acceso no contiguo a los datos en esta escala volumétrica. Por el contrario, en las escalas de cinco y diez gigabytes (ver figura 5.6) se alcanza el punto óptimo de la estrategia, donde se registra una disminución significativa del consumo. En este rango, el volumen de datos ya justifica la búsqueda indexada, puesto que la reducción drástica en el total de operaciones de E/S compensa el costo de los accesos aleatorios a disco frente a una lectura secuencial masiva.

Finalmente, al escalar a los escenarios de veinte y cincuenta gigabytes (ver figura 5.7), la consulta deja de experimentar variaciones significativas tras la implementación de la estrategia de diseño físico. Este estancamiento responde al fenómeno de desbordamiento en disco durante la fase de particionamiento dinámico: al saturarse la memoria intermedia de los trabajadores, el sistema se ve obligado a realizar operaciones masivas de escritura y lectura de archivos temporales en disco, un cuello de botella de E/S de alta demanda energética que termina por neutralizar cualquier beneficio teórico aportado por la indexación local.

Por su parte, la consulta 4 opera sobre la segunda tabla de mayor volumen, **orders**, e incluye una consulta anidada correlacionada con **lineitem**. Al tratarse de tablas co-locadas, el intercambio de datos a través de la red es nulo, lo que se traduce en un perfil de consumo altamente eficiente

que, en las escalas de uno y dos gigabytes, se mantiene por debajo de los diez joules.

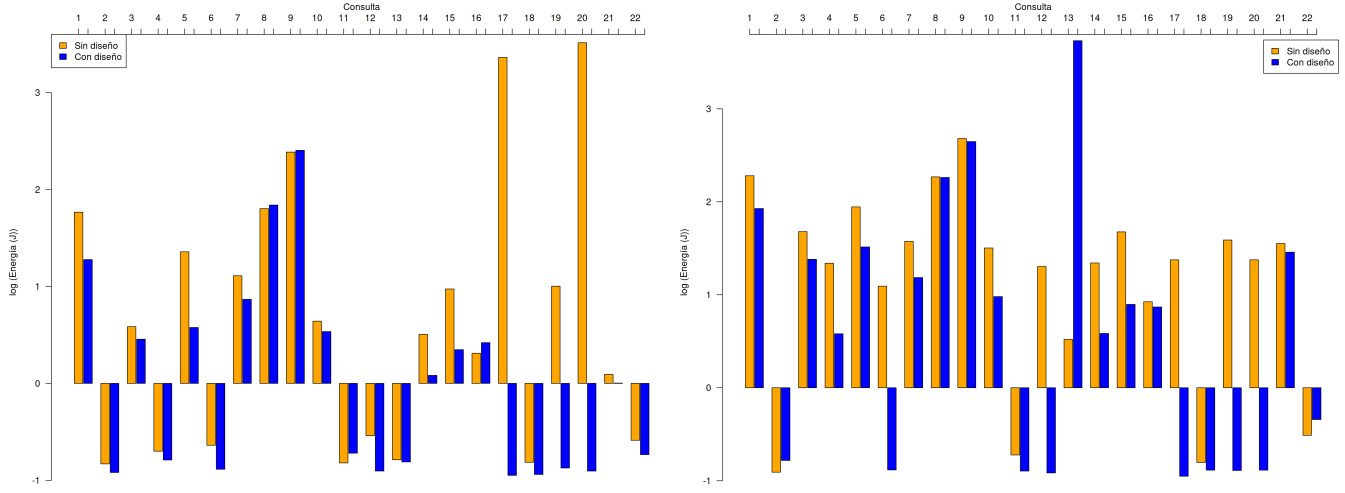


Figura 5.6: Consumo neto por consulta 5 GB (izquierda) y 10 GB (derecha)

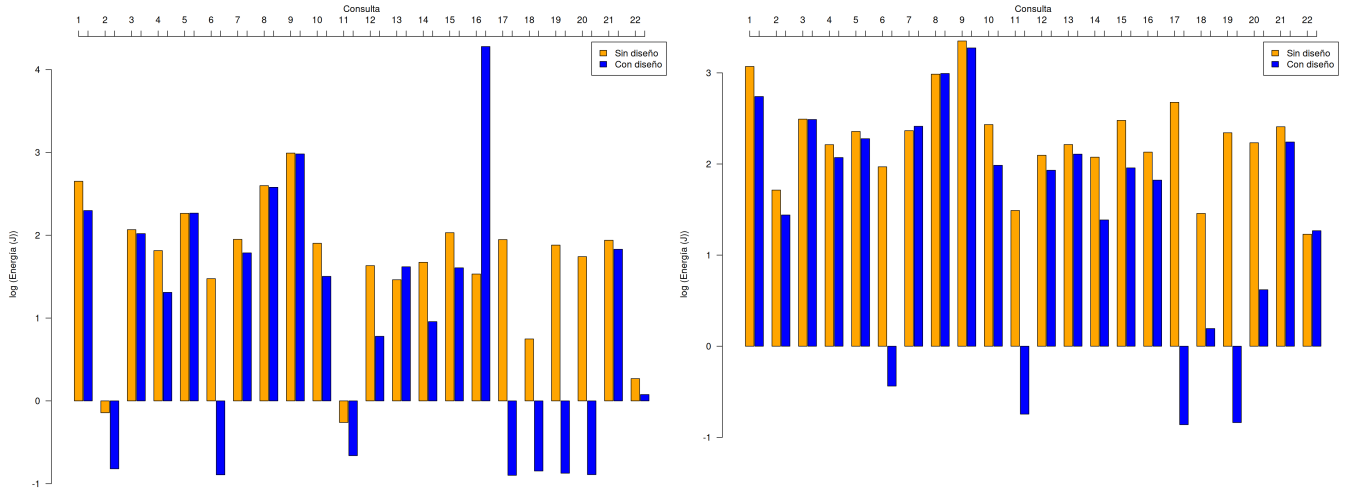


Figura 5.7: Consumo neto por consulta 20 GB (izquierda) y 50 GB (derecha)

A nivel general, en las gráficas se observa una tendencia global de optimización energética gracias a la incorporación del diseño físico (ver figuras 5.5 y 5.6). Según los planes de ejecución analizados, el gestor sustituye las lecturas secuenciales por accesos indexados directos mediante las estructuras `q21i1 ON lineitem (l_orderkey, l_suppkey, l_receiptdate, l_commitdate)` y `q04i1 ON orders (o_orderdate, o_orderkey, o_orderpriority)`. Sin embargo, al alcanzar el escenario crítico de cincuenta gigabytes, la disminución del consumo energético deja de ser estadísticamente significativa (ver figura 5.7). Este comportamiento responde a que, a dicha escala volumétrica, el tamaño de los índices compuestos supera la capacidad de la memoria RAM asignada a las cachés del sistema. Esto fuerza al hardware a ejecutar operaciones de lectura aleatoria directamente sobre el almacenamiento secundario, un cuello de botella de E/S que incrementa el gasto de potencia y termina por equiparar el costo energético del acceso indexado con el del escaneo secuencial del escenario base.

Finalmente, la consulta 10 también requiere la ejecución de particionamiento dinámico. No

obstante, los tiempos de ejecución y los registros de consumo sugieren que se beneficia de la optimización. En este escenario, el impacto positivo de la estrategia de diseño es evidente en casi todo el espectro volumétrico, a pesar de la limitación de observabilidad en el plan de ejecución global.

La única excepción a esta tendencia de ahorro se registra en la escala de un gigabyte, donde la estrategia de diseño físico provocó un ligero incremento en el consumo energético respecto al caso base. Este fenómeno responde al costo fijo de inicialización y procesamiento de las estructuras indexadas (*overhead*): a un volumen tan reducido, los datos residen por completo en las cachés de alta velocidad del procesador, haciendo que una lectura secuencial lineal sea energéticamente más eficiente que el patrón de acceso no contiguo del índice. Sin embargo, a partir de los dos gigabytes, la indexación local comienza a mitigar el impacto del particionamiento dinámico. Al filtrar los registros en el origen, el índice reduce drásticamente el volumen de tuplas que deben ser serializadas y transmitidas a través de la red durante la fase de particionamiento dinámico, lo que despliega un ahorro energético sostenido hasta los cincuenta gigabytes (ver figuras 5.5 a 5.7).

Por su parte, la consulta 14 exhibe un comportamiento estrechamente análogo. En las escalas de uno y dos gigabytes (figura 5.5), la implementación de la estrategia de diseño físico resulta más costosa energéticamente que el escenario base, lo que confirma nuevamente que la penalización por el acceso indexado supera los beneficios de la optimización cuando el volumen de datos es mínimo. No obstante, a partir de los cinco gigabytes y hasta el escenario crítico de cincuenta gigabytes (figuras 5.6 y 5.7), la tendencia se revierte por completo: el consumo energético disminuye de forma sostenida, demostrando que la estrategia logra amortizar eficazmente el costo del particionamiento dinámico a medida que la carga de trabajo escala.

Las consultas restantes dentro del conjunto de pruebas no se discuten de manera individualizada en esta sección, debido a que sus factores críticos de rendimiento y comportamiento estructural fueron exhaustivamente analizados en función del tiempo de ejecución en el apartado anterior. Las consultas detalladas en este segmento de consumo neto fueron seleccionadas específicamente por presentar particularidades u omisiones en sus planes globales de observabilidad; no obstante, el resto de la carga de trabajo replica de manera homóloga las tendencias temporales previamente identificadas, trasladadas ahora al espectro del gasto de potencia absoluto.

Con el propósito de ofrecer una visión de conjunto que condense el comportamiento del consumo del clúster, se introduce la tabla 5.6. En esta herramienta de síntesis, las consultas evaluadas se categorizan formalmente en tres niveles discretos en función de su demanda energética neta: bajo consumo, consumo intermedio y alto consumo. Esta taxonomía permite identificar de un vistazo qué perfiles de carga representan el costo operativo crítico del sistema y cuáles se mantienen en umbrales de alta eficiencia, lo que sirve como puente hacia el análisis de correlación estadística.

Tabla 5.6: Clasificación de consultas según su consumo neto

Carga (GB)	Poco consumo ($U < 1$)	Consumo intermedias ($1 \leq U \leq 10$)	Consumo alto ($U > 10$)
1	1, 2, 4, 6, 11, 12, 13, 15, 16, Q17(idx), 18, 19, Q20(idx), 21 y 22	Q3(idx), 5, 7, Q8(idx), 10 y 14	Q8, 9, Q17 y Q20
2	2, Q3, 4, 6, 11, 12, 13, 15, 16, 18, Q19(*), Q20(idx), 21 y 22	1, Q3(idx), 5, 7, 10, 14 y Q19	8, 9, 17 y Q20
5	2, 4, 6, 11, 12, 13, Q17(idx), 18, Q19(*), Q20(idx), 22	3, Q5(idx), Q7(idx), 10, 14, 15, 16, Q19 y 21	1, Q5, Q7, 8, 9, Q17, Q19 y Q20
10	2, Q6(idx), 11, Q12(idx), Q17(idx), 18, Q19(*), Q20(idx), Q22	Q4(idx), Q10(idx), Q13, Q14(idx), Q15(idx), 16, Q19(*) y Q22(idx)	1, 3, Q4, 5, Q6, 7, 8, 9, Q10, Q12, Q13(CTE), Q14, Q15, Q17, Q19, Q20 y 21
20	2, Q6(idx), 11, Q17(idx), Q18(*), Q19(*) y Q20(idx)	Q12(idx), Q14(idx), Q18 y 22	1, 3, 4, 5, Q6, 7, 8, 9, 10, Q12, 13, Q14, 15, 16, Q17, Q19, Q20 y 21
50	Q6(idx), Q11(idx), Q17(idx) y Q19(*)	Q18(*) y Q20(idx)	1, 2, 3, 4, 5, Q6, 7, 8, 9, 10, Q11, 12, 13, 14, 15, 16, Q17, Q18, Q19, Q20, 21 y 22

5.3.3. Consumo del nodo coordinador

En cuanto al comportamiento del consumo energético del coordinador, en la tabla 5.7, se presenta la clasificación de consultas bajo los mismos criterios aplicados para el consumo neto. Por otra parte, debido a que el comportamiento de los nodos trabajadores resulta análogo al desempeño del sistema, los gráficos se presentan en el apéndice D.

Tabla 5.7: Clasificación de consultas según el consumo del coordinador

Carga (GB)	Poco consumo ($U < 1$)	Consumo intermedias ($1 \leq U \leq 10$)	Consumo alto ($U > 10$)
1	1 a 7 y 10 a 22	8 y 9	
2	1 a 7 y 10 a 22	8 y 9	
5	1 a 6, Q7, 10 a 14, Q15, 17 a 18, Q19(*) y 20 a 22	Q7(idx), 8, Q15(idx), 16 y Q19	9
10	1 a 2, Q3, 4, Q5, 6, 11 a 12, Q13, Q14, 17 a 18, Q19(*) y 20 a 22	Q3(idx), Q5*, 7, 10, Q14(idx), 15 a 16 y Q19	8, 9 y Q13(CTE)
20	1 a 2, 4, 6, 11 a 12, Q13, 17 a 18, Q19(*) y 20 a 22	3, 5, 7, Q13(CTE), 14 y Q19	8 a 10 y 15 a 16
50	1 a 2, 4, 6, 11 a 13, 17 a 18, Q19(*) y 20 a 22	Q3(idx), Q5(idx), Q7(idx), Q8(idx), Q9(idx), Q10(idx), 14, Q15(idx) y Q16(CTE)	Q3, Q5, Q7, Q8, Q9, Q10, Q15, Q16 y Q19

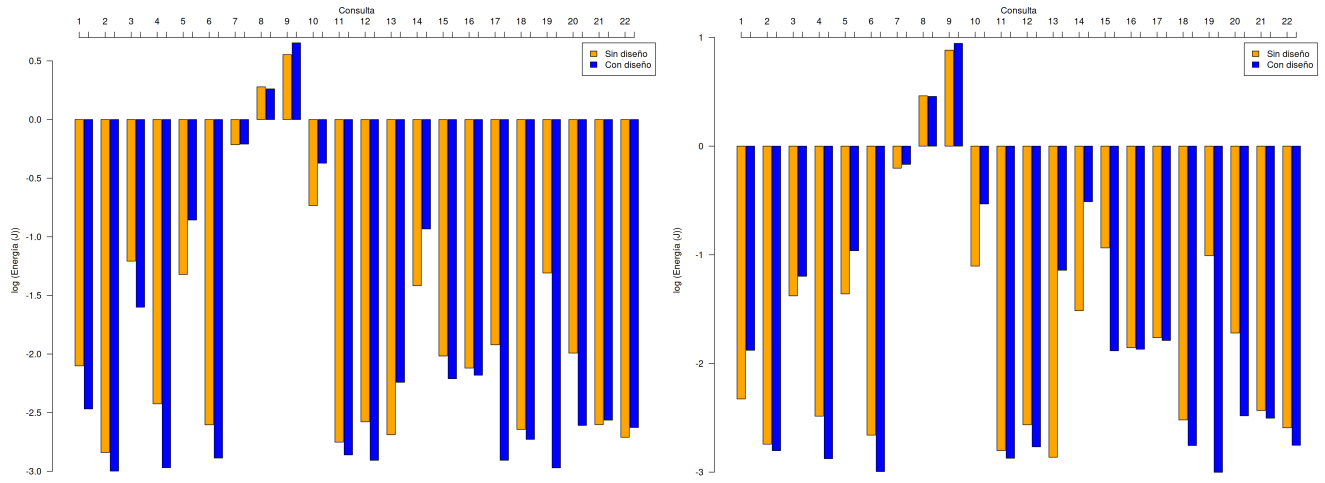


Figura 5.8: Consumo del nodo coordinador por consulta 1 GB (izquierda) y 2 GB (derecha)

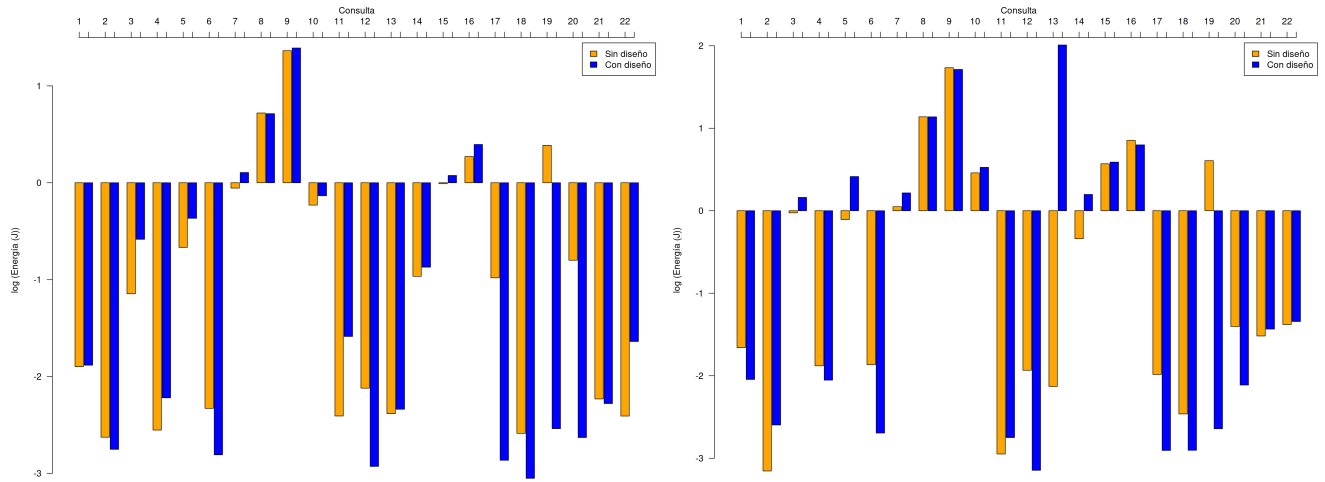


Figura 5.9: Consumo del nodo coordinador por consulta 5 GB (izquierda) y 10 GB (derecha)

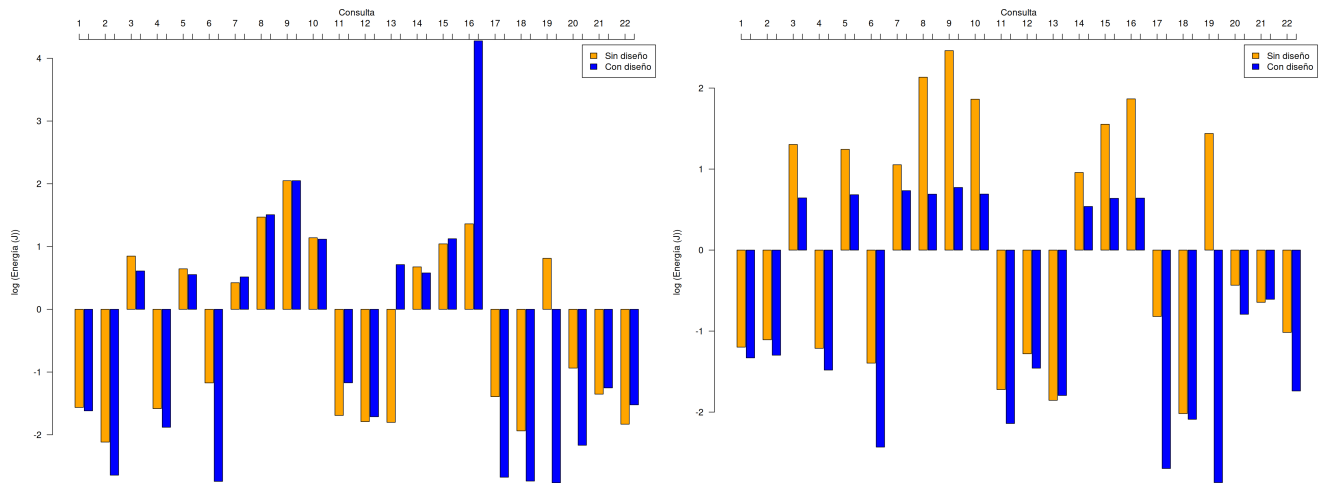


Figura 5.10: Consumo del nodo coordinador por consulta 20 GB (izquierda) y 50 GB (derecha)

En las figuras 5.8, 5.9 y 5.10, se muestran los diagramas homólogos para el consumo del nodo coordinador. Es fundamental señalar la ausencia de una tendencia monótona en el comportamiento de algunas consultas al incrementar el volumen de datos. Esta fluctuación es atribuible a la naturaleza del diseño experimental, el cual se ejecutó bajo un entorno de caché caliente con un intervalo de muestreo de dos segundos (muy cercano al límite operativo nominal de un segundo de la herramienta) [127, 128].

En ese sentido, dado que múltiples consultas se procesan en fracciones de segundo, fue necesario realizar decenas de miles de repeticiones para capturar registros válidos. Aún así, los datos obtenidos no siempre mantenían el mismo orden de magnitud que las mediciones previas. Incluso, en escenarios de alta velocidad, la ejecución finalizaba antes de que los sensores logaran registrar una variación en la potencia. Este comportamiento evidenció una limitación de observabilidad no prevista al inicio de la investigación, lo que fundamenta la necesidad de extender este estudio en el futuro mediante el uso de herramientas de software de mayor frecuencia o instrumentación física externa.

Por otra parte, los resultados empíricos permiten constatar que las estrategias de diseño físico lograron disminuir el consumo energético en la mayoría de las cargas evaluadas, lo que sugiere la existencia de una estrecha relación entre el tiempo de ejecución y la energía consumida. Las excepciones donde no se apreció una reducción del consumo energético se manifiestan en dos vertientes: las penalizaciones a baja escala (uno y dos gigabytes) en consultas con particionamiento dinámico como la 3, 10 y 14; y por otro lado, los casos globales de las consultas 8 y 9. Estos últimos casos corresponden a las consultas más pesadas del *benchmark*, las cuales se vieron penalizadas por la obligación de escribir datos temporalmente de forma local para resolver el particionamiento dinámico.

5.4. Revisión estadística

En esta sección se encuentra todo el tratamiento estadístico de los resultados de tiempo y consumo obtenidos.

5.4.1. Normalidad (Shapiro-Wilk)

Para analizar estadísticamente la relación entre las variables, primero se descartó la normalidad de los datos con la prueba de Shapiro-Wilk. Los resultados de esta prueba se observan en la tabla 5.8. En dicha tabla se presentan los p -valores para las variables medidas en este estudio: tiempo de ejecución (tiempo) y consumo energético (cons.) de los nodos coordinador (coord.), trabajadores (trab.) y neto. Dichos p -valores se obtuvieron con `shapiro.test`, función nativa de R.

Tabla 5.8: Prueba de Shapiro-Wilk (p -valores)

Carga (GB)	Tiempo	Cons. Coord.	Cons. Trab. 1	Cons. Trab. 2	Cons. Neto
1 (SD)	4×10^{-39}	2×10^{-42}	3×10^{-41}	9×10^{-42}	2×10^{-40}
1 (CD)	2×10^{-40}	5×10^{-43}	3×10^{-44}	1×10^{-44}	2×10^{-43}
2 (SD)	6×10^{-37}	1×10^{-43}	4×10^{-42}	3×10^{-42}	5×10^{-42}
2 (CD)	3×10^{-41}	8×10^{-44}	1×10^{-43}	4×10^{-44}	2×10^{-43}
5 (SD)	1×10^{-38}	5×10^{-43}	9×10^{-43}	7×10^{-43}	9×10^{-43}
5 (CD)	6×10^{-42}	3×10^{-43}	1×10^{-43}	1×10^{-43}	2×10^{-43}
10 (SD)	6×10^{-41}	2×10^{-42}	8×10^{-37}	4×10^{-37}	8×10^{-38}
10 (CD)	6×10^{-43}	4×10^{-49}	2×10^{-49}	2×10^{-49}	2×10^{-49}
20 (SD)	2×10^{-42}	5×10^{-41}	2×10^{-36}	3×10^{-36}	3×10^{-37}
20 (CD)	3×10^{-43}	6×10^{-46}	6×10^{-38}	5×10^{-40}	2×10^{-45}
50 (SD)	2×10^{-42}	2×10^{-38}	9×10^{-36}	1×10^{-35}	2×10^{-36}
50 (CD)	6×10^{-44}	1×10^{-32}	4×10^{-38}	2×10^{-37}	7×10^{-38}

Dado que los p -valores $< 0,05$ se cumple que ningún dato sigue una distribución normal y con esto se descarta correlacionar las variables con el coeficiente de correlación de Pearson y en su lugar, se usa los de Kendall o Spearman.

5.4.2. Correlaciones no paramétricas: Kendall y Spearman

En las tablas 5.9 y 5.10 se muestran los coeficientes τ y ρ con sus p -valores para el consumo estimado por la integración numérica de los datos de Scaphandre y la ejecución de las consultas. Dichos resultados se obtuvieron con `cor.test`, función nativa de R.

Los resultados de las tablas 5.9 y 5.10 indican que, en efecto, el consumo energético guarda una correlación significativa con el tiempo de ejecución para cualquier nivel de significancia razonablemente escogido. Por el signo de los coeficientes, la correlación es directa; es decir, a mayor tiempo, más consumo. Para la visualización de estas tendencias, en los apéndices E y F se presentan las gráficas de dispersión de los datos recopilados, en escala logarítmica en ambos ejes para mejorar la visualización de los puntos.

 Tabla 5.9: Coeficientes de Kendall (τ)

Carga (GB)	$\tau_M(p)$	$\tau_1(p)$	$\tau_2(p)$	$\tau_N(p)$
1 (SD)	0,6222(2×10^{-126})	0,6603(4×10^{-142})	0,6720(4×10^{-147})	0,7117(1×10^{-164})
1 (CD)	0,7425(4×10^{-179})	0,6922(6×10^{-156})	0,6916(1×10^{-155})	0,7439(8×10^{-180})
2 (SD)	0,6244(3×10^{-127})	0,6612(2×10^{-142})	0,6423(1×10^{-134})	0,6918(1×10^{-155})
2 (CD)	0,6424(1×10^{-134})	0,4953(9×10^{-81})	0,4737(5×10^{-74})	0,5367(2×10^{-94})
5 (SD)	0,5783(2×10^{-109})	0,7144(6×10^{-166})	0,7244(1×10^{-170})	0,7812(5×10^{-198})
5 (CD)	0,6204(2×10^{-125})	0,6495(2×10^{-137})	0,6149(3×10^{-123})	0,6880(7×10^{-154})
10 (SD)	0,6020(2×10^{-118})	0,7239(2×10^{-170})	0,6047(2×10^{-119})	0,7090(2×10^{-163})
10 (CD)	0,5214(2×10^{-89})	0,6706(2×10^{-146})	0,6955(2×10^{-157})	0,7037(4×10^{-161})
20 (SD)	0,5181(3×10^{-88})	0,6536(3×10^{-139})	0,7344(3×10^{-175})	0,7296(5×10^{-173})
20 (CD)	0,5148(4×10^{-87})	0,6910(2×10^{-155})	0,6612(2×10^{-142})	0,6707(2×10^{-146})
50 (SD)	0,5886(3×10^{-113})	0,6811(5×10^{-151})	0,6782(9×10^{-159})	0,7197(2×10^{-168})
50 (CD)	0,5700(2×10^{-106})	0,7082(5×10^{-163})	0,7276(5×10^{-172})	0,7659(2×10^{-190})

Tabla 5.10: Coeficientes de Spearman (ρ)

Carga (GB)	$\rho_M(p)$	$\rho_1(p)$	$\rho_2(p)$	$\rho_N(p)$
1 (SD)	0,8030(0)	0,8551(0)	0,8723(0)	0,9003(0)
1 (CD)	0,9081(0)	0,8721(0)	0,8627(0)	0,9077(0)
2 (SD)	0,8068(0)	0,8554(0)	0,8417(0)	0,8807(0)
2 (CD)	0,8005(0)	0,5915(0)	0,5758(2×10^{-59})	0,6272(0)
5 (SD)	0,7813(0)	0,8865(0)	0,8925(0)	0,9323(0)
5 (CD)	0,8219(1×10^{-162})	0,8419(3×10^{-178})	0,8113(2×10^{-155})	0,8756(8×10^{-210})
10 (SD)	0,7657(3×10^{-128})	0,8921(4×10^{-229})	0,7857(0)	0,8813(0)
10 (CD)	0,7411(6×10^{-116})	0,8631(2×10^{-197})	0,8874(2×10^{-223})	0,8910(8×10^{-228})
20 (SD)	0,6980(2×10^{-97})	0,8395(2×10^{-176})	0,9030(1×10^{-243})	0,9024(1×10^{-242})
20 (CD)	0,7260(4×10^{-109})	0,8806(1×10^{-215})	0,8612(1×10^{-195})	0,8618(4×10^{-196})
50 (SD)	0,7579(0)	0,8539(0)	0,8503(0)	0,8805(0)
50 (CD)	0,7726(6×10^{-132})	0,8887(5×10^{-225})	0,9033(5×10^{-244})	0,9267(6×10^{-282})

De la revisión estadística, es evidente que la relación entre tiempo y consumo energético es de proporcionalidad directa; por lo tanto, reducir el tiempo de ejecución de una consulta con diseño físico sí reduce el consumo energético de la misma. Este resultado contrasta directamente con los postulados clásicos de Niemann *et al.* [9], quienes afirman que tal correlación se rompe invariablemente en presencia de operaciones de E/S, lo que deriva incluso en consumos mayores para ejecuciones más veloces. En el escenario distribuido relacional evaluado en este trabajo, la evidencia empírica indica lo contrario: las optimizaciones físicas y lógicas que redujeron el tiempo de procesamiento también mitigaron la demanda energética global.

Esta discrepancia con la literatura tradicional se explica a través del impacto de las estrategias de diseño físico implementadas, particularmente la co-localización de tablas. Al suprimir dinámicamente las transferencias de datos internodo a través de la red, se eliminaron los tiempos de espera pasivos del procesador (estados de espera de E/S). Al transformarse en un entorno donde el procesamiento local se ejecuta sin interrupciones, el consumo de potencia eléctrica se volvió directamente proporcional al tiempo de ocupación activa del hardware. De este modo, la estadística de Kendall y Spearman aporta evidencia robusta de que, bajo un diseño físico optimizado, la eficiencia temporal y la eficiencia energética en entornos distribuidos sí caminan en la misma dirección.

Por otra parte, la presencia de una correlación estadísticamente significativa bajo los coeficientes de Kendall y Spearman introduce un matiz importante sobre los hallazgos de Lang *et al.* [38]. Mientras que en tal estudio se argumenta que las operaciones de consulta se estructuran de forma asimétrica no lineal (donde el rendimiento transaccional y la potencia eléctrica se desacoplan), los datos de este trabajo revelan que la aplicación rigurosa de estrategias de diseño físico actúa como un agente homogeneizador. Al estabilizar los planes de ejecución y reducir las operaciones aleatorias, la asimetría postulada en [38] disminuye, lo que obliga a las variables de tiempo y energía a comportarse de manera directamente proporcional y predecible.

Al profundizar en el análisis de sensibilidad según la escala de los datos, se descubrió que los coeficientes de Spearman (tabla 5.10) y Kendall (tabla 5.9) no describen una tendencia monótona. En lugar de registrar un comportamiento lineal o un crecimiento constante en la fuerza de la correlación a medida que aumenta el volumen de información, los coeficientes experimentan fluctuaciones y puntos de inflexión críticos.

Desde una perspectiva estadística, la pérdida de monotonía en la evolución de los coeficientes de Spearman demuestra que la relación entre el tiempo de ejecución y el consumo energético

sufre una alteración estructural profunda dependiendo de la carga. Este fenómeno valida de forma directa las conclusiones de Gomes *et al.* [41], donde se postula que la eficiencia energética es una variable estrictamente dependiente de las condiciones de contorno de la carga de trabajo, y no una constante del motor de bases de datos.

El quiebre de la monotonía matemática halla su explicación física en los cambios de comportamiento del optimizador del gestor y en los límites del hardware. En escenarios de baja carga, el sistema opera bajo una dinámica de ráfagas rápidas controladas por la CPU; sin embargo, al escalar el volumen de datos a rangos donde las estructuras de diseño físico exceden la memoria RAM disponible, el sistema incurre en fallos de página (*page faults*) y accesos masivos a disco. Esta transición de un régimen síncrono a uno dominado por la latencia de E/S destruye la relación monótona de las variables, lo que aporta evidencia estadística robusta de que el volumen de los datos actúa como un factor de quiebre energético en arquitecturas distribuidas.

Conclusiones

A partir de las consultas evaluadas mediante el *benchmark* TPC-H —las cuales demostraron ser idóneas para explotar y medir las capacidades del sistema distribuido frente a operaciones de entrada/salida (E/S) y latencia de red— se evidencia que la ejecución de consultas SQL sin una estrategia de diseño planificada incrementa el uso de recursos computacionales y, en consecuencia, el consumo energético. Por el contrario, la aplicación de estrategias de diseño, guiada estrictamente por los planes de ejecución, logra optimizar tanto el tiempo de ejecución como la eficiencia energética. Este ahorro se produce gracias a la disminución del tiempo en que los procesos de las consultas permanecen activos y a la reducción drástica de las operaciones de E/S, las cuales representan una alta demanda de energía.

Se demostró mediante un análisis estadístico no paramétrico (coeficientes de Spearman y Kendall) que, en contraposición a las teorías tradicionales para sistemas centralizados [9], el tiempo de ejecución y el consumo energético guardan una correlación positiva y significativa en entornos distribuidos relacionales bajo la aplicación de estas estrategias de diseño físico. La evidencia empírica recolectada confirma que, al suprimir la latencia de red y optimizar los flujos lógicos, el comportamiento del hardware se estabiliza, logrando que la reducción del tiempo de procesamiento se traduzca de manera directa y estadísticamente robusta en un ahorro proporcional del consumo energético global del clúster.

Sin embargo, la implementación de estas optimizaciones no es universal y presenta sus propias contrapartidas. Una estrategia de diseño ingenua (que está basada en un análisis superficial de la consulta sin considerar las implicaciones algorítmicas del código SQL), apresurada (sin haber estudiado el plan de ejecución a detalle) o impuesta de manera forzada sobre bases de datos pequeñas puede degradar el desempeño, ya que en volúmenes menores de información, los algoritmos base del gestor suelen ser más eficientes por sí solos. Además, aunque las estrategias de diseño físico mejoran el rendimiento y reducen el consumo energético a nivel de procesamiento, incrementan el uso de espacio en disco. Esto introduce un costo energético latente derivado de la necesidad de leer un mayor volumen de datos almacenados en los soportes físicos. Esto quiere decir que si las estructuras auxiliares creadas son más grandes que la memoria RAM disponible y la consulta obliga a realizar lecturas completas o aleatorios sobre ellas, el sistema incurre en una penalización constante de energía eléctrica. Específicamente, si las estructuras auxiliares creadas superan la capacidad de la memoria RAM disponible, el sistema incurre en fallos de página (*page faults*) —fenómeno en el cual el gestor se ve obligado a suspender la ejecución para buscar los datos faltantes en el almacenamiento secundario— lo que se traduce en una penalización térmica y de potencia eléctrica continua sobre el hardware, destruyendo la monotonía de la eficiencia energética.

Finalmente, en el contexto específico de las arquitecturas distribuidas, la co-localización —técnica que consiste en almacenar los registros relacionados de distintas tablas en un mismo nodo físico para evitar transferencias de datos en la red— de tablas emerge como factor definitivo y crítico para sostener el rendimiento y mitigar la latencia de red [129, 130]. La relevancia de esta técnica radica en que garantiza que cada nodo del clúster trabaje de forma totalmente independiente [129, 130] de los otros lo que justifica las reducciones de varios órdenes de magnitud de los tiempos de ejecución.

A partir de este hallazgo empírico, queda propuesto como trabajo futuro evaluar y comparar el rendimiento de un sistema distribuido que carezca de co-localización frente a las capacidades de un sistema de bases de datos centralizado bajo escalabilidad vertical. La ruta metodológica para simular la co-localización requiere el rediseño y las reescritura de las consultas del *benchmark*. Para ello, se plantea la creación de réplicas de las tablas modificando la clave de repartición primaria y

combinándolas mediante operaciones de `JOIN`, de modo que se maximice la necesidad de redistribución de datos entre los nodos. Otra alternativa viable consiste en inyectar atributos sintéticos en el esquema para segmentar artificialmente los datos para forzar un escenario de particionamiento dinámico extremo que permita medir el punto de quiebre del sistema.

Consecuentemente, el éxito en la optimización o simulación de la co-localización conlleva a que el rendimiento de las consultas disminuya a fracciones de segundo. De ahí, surge la necesidad del estudio sobre las mediciones del consumo energético para consultas de ese orden de magnitud, superando la problemática subyacente es que las herramientas utilizadas (Scaphandre y Prometheus) las cuales no fueron diseñadas para la alta frecuencia requerida por entornos distribuidos con cargas ultrarrápidas. Para resolver esta restricción de observabilidad que afectó la reproducibilidad en los escenarios de menor volumen, las futuras investigaciones deberán orientarse al estudio de otras herramientas de software y de hardware optimizadas para realizar mediciones de energía cuando las ejecuciones duren fracciones de segundo, así como al desarrollo de mecanismos que soporten la medición de consumo en ambientes distribuidos dentro de las herramientas disponibles.

Referencias

- [1] H. Rong, H. Zhang, S. Xiao, C. Li, and C. Hu, “Optimizing energy consumption for data centers,” *Renewable and Sustainable Energy Reviews*, vol. 58, pp. 674–691, May 2016. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1364032115016664>
- [2] S. Maroulis, N. Zacheilas, and V. Kalogeraki, “ExpREsS: EneRgy Efficient Scheduling of Mixed Stream and Batch Processing Workloads,” in *2017 IEEE International Conference on Autonomic Computing (ICAC)*. Columbus, OH, USA: IEEE, Jul. 2017, pp. 27–32. [Online]. Available: <http://ieeexplore.ieee.org/document/8005324/>
- [3] A. P. Florence, V. Shanthi, and C. B. S. Simon, “Energy Conservation Using Dynamic Voltage Frequency Scaling for Computational Cloud,” *The Scientific World Journal*, vol. 2016, no. 1, p. 9328070, 2016. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1155/2016/9328070>
- [4] J. Krzywda, A. Ali-Eldin, T. E. Carlson, P.-O. Östberg, and E. Elmroth, “Power-performance tradeoffs in data center servers: DVFS, CPU pinning, horizontal, and vertical scaling,” *Future Generation Computer Systems*, vol. 81, pp. 114–128, Apr. 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X17304910>
- [5] C. Bunse, H. Höpfner, S. Klingert, E. Mansour, and S. Roychoudhury, “Energy Aware Database Management,” in *Energy-Efficient Data Centers*, S. Klingert, X. Hesselbach-Serra, M. P. Ortega, and G. Giuliani, Eds. Berlin, Heidelberg: Springer, 2014, pp. 40–53.
- [6] Z. Xu, Y.-C. Tu, and X. Wang, “Exploring power-performance tradeoffs in database systems,” in *2010 IEEE 26th International Conference on Data Engineering (ICDE 2010)*, Mar. 2010, pp. 485–496. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/5447840>
- [7] W. Lang and J. Patel, “Towards Eco-friendly Database Management Systems,” Sep. 2009. [Online]. Available: <http://arxiv.org/abs/0909.1767>
- [8] W. Lang, R. Kandhan, and J. M. Patel, “Rethinking Query Processing for Energy Efficiency: Slowing Down to Win the Race,” *IEEE Data Eng. Bull.*, vol. 34, no. 1, pp. 12–23, 2011.
- [9] R. Niemann, N. Korfiatis, R. Zicari, and R. Göbel, “Does query performance optimization lead to energy efficiency? A comparative analysis of energy efficiency of database operations under different workload scenarios,” Mar. 2013. [Online]. Available: <http://arxiv.org/abs/1303.4869>

- [10] ———, “Evaluating the Energy Efficiency of OLTP Operations,” in *Availability, Reliability, and Security in Information Systems and HCI*. Berlin, Heidelberg: Springer, 2013, pp. 28–43.
- [11] D. Duarte and O. Belo, “Evaluating Query Energy Consumption in Document Stores,” in *Emerging Technologies for Developing Countries*, F. Belqasmi, H. Harroud, M. Agueh, R. Dssouli, and F. Kamoun, Eds. Cham: Springer International Publishing, 2018, pp. 79–88.
- [12] J. Pisharath, A. Choudhary, and M. Kandemir, “Reducing energy consumption of queries in memory-resident database systems,” in *Proceedings of the 2004 international conference on Compilers, architecture, and synthesis for embedded systems*, ser. CASES ’04. New York, NY, USA: Association for Computing Machinery, Sep. 2004, pp. 35–45. [Online]. Available: <https://doi.org/10.1145/1023833.1023840>
- [13] C. Macdonald, N. Tonello, and I. Ounis, “Efficient & Effective Selective Query Rewriting with Efficiency Predictions,” in *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR ’17. New York, NY, USA: Association for Computing Machinery, Aug. 2017, pp. 495–504. [Online]. Available: <https://doi.org/10.1145/3077136.3080827>
- [14] D. Mahajan and Z. Zong, “Energy efficiency analysis of query optimizations on MongoDB and Cassandra,” in *2017 Eighth International Green and Sustainable Computing Conference (IGSC)*, Oct. 2017, pp. 1–6. [Online]. Available: <https://ieeexplore.ieee.org/document/8323581>
- [15] D. Mahajan, C. Blakeney, and Z. Zong, “Improving the energy efficiency of relational and NoSQL databases via query optimizations,” *Sustainable Computing: Informatics and Systems*, vol. 22, pp. 120–133, Jun. 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2210537918301112>
- [16] Binglei Guo, Jiong Yu, Bin Liao, and Dexian Yang, “SQL energy consumption modeling and optimization research,” *Journal of Computational Science*, vol. 42, no. 10, pp. 202–207, 2015.
- [17] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2019.
- [18] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.
- [19] T. Connolly and C. Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*, 6th ed. Pearson, 2014.
- [20] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed. McGraw-Hill, 2003.
- [21] B. Guo, J. Yu, D. Yang, H. Leng, and B. Liao, “Energy-Efficient Database Systems: A Systematic Survey,” *ACM Computing Surveys*, vol. 55, no. 6, pp. 1–53, Jul. 2023. [Online]. Available: <https://dl.acm.org/doi/10.1145/3538225>

- [22] C. Bunse, H. Höpfner, E. Mansour, and S. Roychoudhury, “Exploring the Energy Consumption of Data Sorting Algorithms in Embedded and Mobile Environments,” in *2009 Tenth International Conference on Mobile Data Management: Systems, Services and Middleware*. Taipei, Taiwan: IEEE, 2009, pp. 600–607. [Online]. Available: <http://ieeexplore.ieee.org/document/5089010/>
- [23] H. Hopfner and C. Bunse, “Towards an Energy Aware DBMS – Energy Consumptions of Sorting and Join Algorithms,” *Deutsche Nationalbibliothek*, 2009.
- [24] M. c. Belgaid, R. Rouvoy, and L. Seinturier, “Pyjoules: Python library that measures python code snippets,” Nov. 2019. [Online]. Available: <https://github.com/powerapi-ng/pyJoules>
- [25] R. Chattaraj and S. Chimalakonda, “RJoules: An Energy Measurement Tool for R,” in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Sep. 2023, pp. 2026–2029. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10298336>
- [26] S. S. A. P. R. Chattaraj, and S. Chimalakonda, “CPPJoules: An Energy Measurement Tool for C++,” Dec. 2024. [Online]. Available: <http://arxiv.org/abs/2412.13555>
- [27] H. S. Lella, K. Manasa, R. Chattaraj, and S. Chimalakonda, “DBJoules: An Energy Measurement Tool for Database Management Systems,” Nov. 2023. [Online]. Available: <http://arxiv.org/abs/2311.08961>
- [28] PostgreSQL Global Development Group, *PostgreSQL 16.0 Documentation*, 2023. [Online]. Available: <https://www.postgresql.org/docs/16/index.html>
- [29] J. Cunha *et al.*, “Citrus: Distributing postgresql for analytics and real-time workloads,” in *Proceedings of the SIGMOD International Conference on Management of Data*. ACM, 2021.
- [30] Citus Data, a Microsoft Company, *Citus Documentation: Distributed PostgreSQL as an Extension*, 2024. [Online]. Available: <https://docs.citusdata.com/>
- [31] Hubblo, “hubblo-org/scaphandre,” Sep. 2025. [Online]. Available: <https://github.com/hubblo-org/scaphandre>
- [32] SoundCloud, “Overview | Prometheus,” 2016. [Online]. Available: <https://prometheus.io/docs/introduction/overview/>
- [33] Transaction Processing Performance Council (TPC), “TPC Benchmark TM H Standard Specification Revision,” apr 2022. [Online]. Available: https://www.tpc.org/TPC_Documents_Current_Versions/pdf/TPC-H.v3.0.1.pdf
- [34] T. L. D.-O.-. Rep. Ashley, “H.R.8444 - 95th Congress (1977-1978): National Energy Act,” Oct. 1977. [Online]. Available: <https://www.congress.gov/bill/95th-congress/house-bill/8444>
- [35] M. J. Horowitz, “Purchased energy and policy impacts in the US manufacturing sector,” *Energy Efficiency*, vol. 7, no. 1, pp. 65–77, Feb. 2014. [Online]. Available: <http://link.springer.com/10.1007/s12053-013-9200-3>

- [36] S. Murugesan, “Going Green with IT: Your Responsibility Toward Environmental Sustainability — Executive Summary | Cutter Consortium,” 2007. [Online]. Available: <https://www.cutter.com/article/going-green-it-your-responsibility-toward-environmental-sustainability-380516>
- [37] S. Noll, H. Funke, and J. Teubner, “Energy Efficiency in Main-Memory Databases,” *Datenbank-Spektrum*, vol. 17, no. 3, pp. 223–232, Nov. 2017. [Online]. Available: <https://doi.org/10.1007/s13222-017-0262-9>
- [38] W. Lang, S. Harizopoulos, J. M. Patel, M. A. Shah, and D. Tsirogiannis, “Towards Energy-Efficient Database Cluster Design,” Aug. 2012. [Online]. Available: <http://arxiv.org/abs/1208.1933>
- [39] M. Shah, A. Kothari, and S. Patel, “A Comprehensive Survey on Energy Consumption Analysis for NoSQL,” *Scalable Computing: Practice and Experience*, vol. 23, no. 1, pp. 35–50, Apr. 2022. [Online]. Available: [/scpe.org/index.php/scpe/article/view/1971](https://scpe.org/index.php/scpe/article/view/1971)
- [40] A. Karras, C. Karras, A. Pervanas, S. Sioutas, and C. Zaroliagis, “SQL Query Optimization in Distributed NoSQL Databases for Cloud-Based Applications,” in *Algorithmic Aspects of Cloud Computing*, L. Foschini and S. Kontogiannis, Eds. Cham: Springer International Publishing, 2023, vol. 13799, pp. 21–41. [Online]. Available: https://link.springer.com/10.1007/978-3-031-33437-5_2
- [41] C. Gomes, E. Tavares, and M. N. d. O. Junior, “Energy Consumption Evaluation of NoSQL DBMSs,” in *Workshop em Desempenho de Sistemas Computacionais e de Comunicação (WPerformance)*. SBC, Jul. 2016, pp. 2828–2838. [Online]. Available: <https://sol.sbc.org.br/index.php/wperformance/article/view/9729>
- [42] C. Gomes, M. N. De O. Junior, B. Nogueira, P. Maciel, and E. Tavares, “NoSQL-based storage systems: influence of consistency on performance, availability and energy consumption,” *The Journal of Supercomputing*, vol. 79, no. 18, pp. 21 424–21 448, Dec. 2023. [Online]. Available: <https://link.springer.com/10.1007/s11227-023-05488-6>
- [43] E. A. Brewer, “Towards robust distributed systems,” in *Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC '00)*, Portland, Oregon, USA, July 2000, p. 7.
- [44] S. Gilbert and N. Lynch, “Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services,” *Acm Sigact News*, vol. 33, no. 2, pp. 51–59, 2002.
- [45] J. Song, X. Zhao, C. Guo, Y. Gu, and G. Yu, “Towards an Energy Complexity Model for Distributed Data Processing Algorithms,” *IEEE Transactions on Big Data*, vol. 9, no. 6, pp. 1510–1524, Dec. 2023. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10146456>
- [46] H. S. Lella, R. Chattaraj, S. Chimalakonda, and M. Kurra, “Towards Comprehending Energy Consumption of Database Management Systems - A Tool and Empirical Study,” in *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE '24. New York, NY, USA: Association for Computing Machinery, Jun. 2024, pp. 272–281. [Online]. Available: <https://dl.acm.org/doi/10.1145/3661167.3661174>

- [47] V. Prokhorenko and M. A. Babar, “Offloaded Data Processing Energy Efficiency Evaluation,” *Informatica*, vol. 35, no. 3, pp. 649–669, jul 2024. [Online]. Available: <https://informatica.vu.lt/journal/INFORMATICA/article/1343>
- [48] B. Guo, J. Wu, Y. Pu, J. Zhang, and J. Yu, “Energy consumption estimation and profiling for queries in distributed database systems based on a bottom-up comprehensive energy model,” *Future Generation Computer Systems*, vol. 159, pp. 379–394, Oct. 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X24001973>
- [49] B. Subramaniam and W.-c. Feng, “On the Energy Proportionality of Distributed NoSQL Data Stores,” in *High Performance Computing Systems. Performance Modeling, Benchmarking, and Simulation*, S. A. Jarvis, S. A. Wright, and S. D. Hammond, Eds. Cham: Springer International Publishing, 2015, pp. 264–274.
- [50] R. Niemann, “Towards the Prediction of the Performance and Energy Efficiency of Distributed Data Management Systems,” in *Companion Publication for ACM/SPEC on International Conference on Performance Engineering*. Delft The Netherlands: ACM, Mar. 2016, pp. 23–28. [Online]. Available: <https://dl.acm.org/doi/10.1145/2859889.2859891>
- [51] A. Roukh, L. Bellatreche, N. Tziritas, and C. Ordonez, “Energy-Aware Query Processing on a Parallel Database Cluster Node,” in *Algorithms and Architectures for Parallel Processing*. Cham: Springer International Publishing, 2016, vol. 10048, pp. 260–269. [Online]. Available: http://link.springer.com/10.1007/978-3-319-49583-5_20
- [52] P.-A.-T. Tran, L. d’Orazio, T.-C. Phan, and L. Gruenwald, “Energy and Performance Evaluation of Serverless and Serverful Models on Spark for Database Join Operations,” in *International Conference on Database and Expert Systems Applications (DEXA)*, Bangkok, Thailand, Aug. 2025. [Online]. Available: <https://hal.science/hal-05136844>
- [53] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, “Benchmarking cloud serving systems with YCSB,” in *Proceedings of the 1st ACM symposium on Cloud computing*. Indianapolis Indiana USA: ACM, Jun. 2010, pp. 143–154. [Online]. Available: <https://dl.acm.org/doi/10.1145/1807128.1807152>
- [54] A. García-Recuero, “On the energy efficiency of client-centric data consistency management under random read/write access to Big Data with Apache HBase,” Nov. 2016. [Online]. Available: <http://arxiv.org/abs/1509.02640>
- [55] E. Casalicchio, L. Lundberg, and S. Shirinbab, “Energy-aware auto-scaling algorithms for Cassandra virtual data centers,” *Cluster Computing*, vol. 20, no. 3, pp. 2065–2082, Sep. 2017. [Online]. Available: <https://doi.org/10.1007/s10586-017-0912-6>
- [56] The Lemur Project, “The ClueWeb09 dataset,” <https://lemurproject.org/clueweb09/>, 2009.
- [57] Dell Technologies, “Index of /dvdstore,” Dec. 2010. [Online]. Available: <https://linux.dell.com/dvdstore/>
- [58] B. Bani, F. Khomh, and Y.-G. Guéhéneuc, “A Study of the Energy Consumption of Databases and Cloud Patterns,” in *Service-Oriented Computing*. Cham: Springer International Publishing, 2016, vol. 9936, pp. 606–614. [Online]. Available: https://link.springer.com/10.1007/978-3-319-46295-0_40

- [59] R. Jung and M. Adolf, “The JPetStore Suite: A concise Experiment Setup for Research,” in *Symposium on Software Performance*, 2018.
- [60] MyBatis, “mybatis/jpetstore-6,” Sep. 2025. [Online]. Available: <https://github.com/mybatis/jpetstore-6>
- [61] A. Wolski, “TATP Benchmark Description,” *IBM Software Group Information Management*, Mar. 2009.
- [62] T. Kissinger, D. Habich, and W. Lehner, “Adaptive Energy-Control for In-Memory Database Systems,” in *Proceedings of the 2018 International Conference on Management of Data*, ser. SIGMOD ’18. New York, NY, USA: Association for Computing Machinery, May 2018, pp. 351–364. [Online]. Available: <https://doi.org/10.1145/3183713.3183756>
- [63] M. Mitri, “Active Learning via a Sample Database: The Case of Microsoft’s Adventure Works,” *Journal of Information Systems Education*, vol. 26, no. 3, pp. 177–185, 2015.
- [64] MashaMSFT, “AdventureWorks Sample Databases - SQL Server,” Sep. 2025. [Online]. Available: <https://learn.microsoft.com/en-us/sql/samples/adventureworks-install-configure?view=sql-server-ver17>
- [65] J. Saraiva, M. Guimarães, and O. Belo, “AN ECONOMIC ENERGY APPROACH FOR QUERIES ON DATA CENTERS,” in *3rd International Conference on Energy and Environment: bringing together Engineering and Economics*, Jun. 2017.
- [66] M. Ferdman, A. Adileh, O. Kocberber, S. Volos, D. Jevdjic, C. Kaynak, A. A. Popescu, B. Pozović, and B. Falsafi, “Clearing the clouds: A study of emerging scale-out workloads,” in *Proceedings of the 17th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2012, pp. 37–48.
- [67] EPFL PARSA, “CloudSuite | A Benchmark Suite for Cloud Services,” 2025. [Online]. Available: <https://www.cloudsuite.ch/>
- [68] PARSA EPFL, “parsa-epfl/cloudsuite,” Aug. 2025. [Online]. Available: <https://github.com/parsa-epfl/cloudsuite>
- [69] B. Subramaniam and W.-c. Feng, “On the Energy Proportionality of Scale-Out Workloads,” Jan. 2015. [Online]. Available: <http://arxiv.org/abs/1501.02729>
- [70] P. O’Neil, B. O’Neil, and X. Chen, “Star Schema Based on TPC-H,” *UMass*, Jun. 2009.
- [71] M. Poess and C. Floyd, “New TPC benchmarks for decision support and web commerce,” *SIGMOD Rec.*, vol. 29, no. 4, pp. 64–71, dec 2000. [Online]. Available: <https://dl.acm.org/doi/10.1145/369275.369291>
- [72] M. Poess and R. O. Nambiar, “Energy cost, the key challenge of today’s data centers: a power consumption analysis of TPC-C results,” *Proc. VLDB Endow.*, vol. 1, no. 2, pp. 1229–1240, aug 2008. [Online]. Available: <https://doi.org/10.14778/1454159.1454162>
- [73] J. Gray and A. Reuter, *Transaction processing: concepts and techniques*. Elsevier, 1992.

- [74] D. E. Difallah, A. Pavlo, C. Curino, and P. Cudre-Mauroux, “Oltp-bench: An extensible testbed for benchmarking relational databases,” *Proceedings of the VLDB Endowment*, vol. 7, no. 4, pp. 277–288, 2013.
- [75] D. A. Wood, “Dbmss on a modern processor: Where does time go?” in *VLDB’99, Proceedings of 25th International Conference on Very Large Data Bases, September 7-10, 1999, Edinburgh, Scotland, UK*, 1999, pp. 266–277.
- [76] W. Vogels, “Eventually consistent,” *Communications of the ACM*, vol. 52, no. 1, pp. 40–44, 2009.
- [77] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, “Benchmarking cloud serving systems with ycsb,” in *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC)*. ACM, 2010, pp. 143–154.
- [78] M. Ferdman *et al.*, “Cloudsuite: A benchmark suite for cloud services,” 2022.
- [79] Microsoft Corporation, *AdventureWorks sample databases*, 2023. [Online]. Available: <https://learn.microsoft.com/en-us/sql/samples/adventureworks-install-configure>
- [80] E. M. Voorhees, “The Cranfield paradigm: TREC, recall, and precision,” in *Information Retrieval: Uncertainty and Logics*, F. Crestani, M. Lalmas, and C. J. Rijsbergen, Eds. Boston: Kluwer Academic Publishers, 2001, pp. 113–126.
- [81] T. Trébaol, “CUMULATOR — a tool to quantify and report the carbon footprint of machine learning computations and communication in academia and healthcare,” Jun. 2020. [Online]. Available: <https://infoscience.epfl.ch/server/api/core/bitstreams/8598c120-d9fb-4956-ae04-ba67dbe910d9/content>
- [82] S. Baldwin, “Carbon footprint of electricity generation,” *London: Parliamentary Office of Science and Technology*, vol. 268, 2006.
- [83] S. Igescu, G. Monea, and V. Pecorella, “EcoML: A tool to track and predict the carbon footprint of machine learning tasks,” 2023.
- [84] Mlco2, “mlco2/codecarbon,” Sep. 2025. [Online]. Available: <https://github.com/mlco2/codecarbon>
- [85] W. Pan, C. Hu, G. Huang, W.-q. Dai, and W. Pan, “Energy footprint: Concept, application and modeling,” *Ecological Indicators*, vol. 158, p. 111459, 2024.
- [86] G. Pinto, F. Castor, and K. Liu, “A comprehensive study on the energy efficiency of java’s thread-safe collections,” *IEEE Transactions on Software Engineering*, vol. 43, no. 12, pp. 1151–1173, 2017.
- [87] kliu20, “kliu20/jRAPL at gh-pages,” 2014. [Online]. Available: <https://github.com/kliu20/jRAPL/tree/gh-pages?tab=readme-ov-file>
- [88] R. Redelmeier, *cpuburn: CPU burn-in software for Linux*, 2011. [Online]. Available: <https://packages.debian.org/sid/cpuburn>

- [89] —, “burnMMX(1) — cpuburn — Debian jessie — Debian Manpages,” Jun. 2011. [Online]. Available: <https://manpages.debian.org/jessie/cpuburn/burnMMX.1.en.html>
- [90] H. Garcia-Molina, J. D. Ullman, and J. Widom, *Database Systems: The Complete Book*, 2nd ed. Pearson Prentice Hall, 2009.
- [91] F. W. Sears, M. W. Zemansky, H. D. Young, R. A. Freedman, and A. L. Ford, *Física universitaria*. Pearson, 2004, vol. 2.
- [92] R. L. Burden and J. D. Faires, *Numerical Analysis*, 9th ed. Boston, MA: Cengage Learning, 2010.
- [93] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008. [Online]. Available: <https://nlp.stanford.edu/IR-book/>
- [94] S. Chaudhuri, *An Overview of Query Optimization in Relational Systems*, 1998.
- [95] C. J. Date, *Database Design and Relational Theory: Normal Forms and All That Jazz*. O’Reilly Media, 2012.
- [96] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 6th ed. McGraw-Hill, 2010.
- [97] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018.
- [98] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA: Pearson, 2014.
- [99] J. M. Hellerstein, M. Stonebraker, and J. Hamilton, “Architecture of a database system,” *Foundations and Trends® in Databases*, vol. 1, no. 2, pp. 141–259, 2007.
- [100] G. Graefe, “Query evaluation techniques for large databases,” *ACM Computing Surveys (CSUR)*, vol. 25, no. 2, pp. 73–169, 1993.
- [101] R. p. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Hoboken, NJ: Pearson, 2015.
- [102] C. Niemann, “Energy-efficient data management in embedded and server systems,” Doctoral dissertation, Technische Universität Dortmund, 2012.
- [103] W. Lang, J. M. Patel, and J. F. Naughton, “On energy management in a database cluster,” in *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, 2012, pp. 817–828.
- [104] S. S. Shapiro and M. B. Wilk, “An analysis of variance test for normality (complete samples),” *Biometrika*, vol. 52, no. 3/4, pp. 591–611, 1965.
- [105] J. P. Royston, “Algorithm as 181: The w test for normality,” *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, vol. 31, no. 2, pp. 176–180, 1982.
- [106] C. Spearman, “The proof and measurement of association between two things,” *The American Journal of Psychology*, vol. 15, no. 1, pp. 72–101, 1904.

- [107] H. Hotelling and M. R. Pabst, “Rank correlation and tests of significance involving no assumption of normality,” *The Annals of Mathematical Statistics*, vol. 7, no. 1, pp. 29–43, 1936.
- [108] M. G. Kendall, “A new measure of rank correlation,” *Biometrika*, vol. 30, no. 1/2, pp. 81–93, 1938.
- [109] —, *Rank Correlation Methods*, 1st ed. London: Charles Griffin & Company, 1948.
- [110] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*, 4th ed. Springer, 2020.
- [111] Transaction Processing Performance Council (TPC), “Tpc benchmark h (decision support) standard specification,” TPC, Tech. Rep., 2021. [Online]. Available: <http://www.tpc.org/tpch/>
- [112] Intel, “Running Average Power Limit Energy Reporting CVE-2020-8694,...” 2022. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/advisory-guidance/running-average-power-limit-energy-reporting.html>
- [113] Hubblo, “Introduction - Scaphandre documentation,” Sep. 2025. [Online]. Available: <https://hubblo-org.github.io/scaphandre-documentation/index.html>
- [114] Free Software Foundation, *GNU Coreutils online help*, 2024. [Online]. Available: <https://www.gnu.org/software/coreutils/>
- [115] F. S. Foundation, *GNU sed, a stream editor*, 2020. [Online]. Available: <https://www.gnu.org/software/sed/manual/sed.html>
- [116] S. Dolan, *jq: A command-line JSON processor*, 2023. [Online]. Available: <https://jqlang.github.io/jq/>
- [117] G. Van Rossum and F. L. Drake, *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace, 2009.
- [118] R Core Team, *R: A Language and Environment for Statistical Computing*, R Foundation for Statistical Computing, Vienna, Austria, 2023. [Online]. Available: <https://www.R-project.org/>
- [119] Posit Team, *RStudio: Integrated Development Environment for R*, Posit Software, PBC, Boston, MA, 2024. [Online]. Available: <http://www.posit.co/>
- [120] F. S. Foundation, *GNU Wget manual*, 2023. [Online]. Available: <https://www.gnu.org/software/wget/manual/wget.html>
- [121] The PostgreSQL Global Development Group, *PostgreSQL 16 Documentation - Chapter 25. Routine Database Maintenance Tasks: The Autovacuum Daemon*, 2023. [Online]. Available: <https://www.postgresql.org/docs/current/routine-vacuuming.html#AUTOVACUUM>
- [122] D. A. Skoog, D. M. West, F. J. Holler, and S. R. Crouch, *Fundamentals of Analytical Chemistry*, 9th ed. Belmont, CA: Cengage Learning, 2013.

- [123] R. Jain, *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. John Wiley & Sons, 1991.
- [124] D. C. Montgomery, *Design and Analysis of Experiments*, 9th ed. John Wiley & Sons, 2017.
- [125] Citus Data, Microsoft, *Citus Documentation - Query Processing*, 2024. [Online]. Available: https://docs.citusdata.com/en/stable/develop/reference_processing.html
- [126] —, *Citus Documentation - Query Performance Tuning*, 2024. [Online]. Available: https://docs.citusdata.com/en/stable/performance/performance_tuning.html
- [127] Prometheus Authors, *Best practices: Instrumentation - High-resolution metrics*, The Prometheus Project, 2024. [Online]. Available: <https://prometheus.io/docs/practices/instrumentation/#high-resolution-metrics>
- [128] —, *Configuration: Scrape Config*, The Prometheus Project, 2024. [Online]. Available: https://prometheus.io/docs/prometheus/latest/configuration/configuration/#scrape_config
- [129] M. Stonebraker, “The case for shared nothing,” *IEEE Database Engineering Bulletin*, vol. 9, no. 1, pp. 4–9, 1986.
- [130] D. DeWitt and J. Gray, “Parallel database systems: The future of high performance database systems,” *Communications of the ACM*, vol. 35, no. 6, pp. 85–98, 1992.
- [131] M. S. Gharajeh, “Security Issues and Privacy Challenges of NoSQL Databases,” in *NoSQL: Database for Storage and Retrieval of Data in Cloud*, 1st ed., G. C. Deka, Ed. Boca Raton, FL : CRC Press, Taylor & Francis Group, [2016] |Includes bibliographical references and index.: Chapman and Hall/CRC, May 2017, pp. 271–290. [Online]. Available: <https://www.taylorfrancis.com/books/9781498784375/chapters/10.1201/9781315155579-15>

Apéndice A. TPC-H

A.1. Diseño lógico de la base de datos

Dado el diseño conceptual, se muestra el diseño lógico en modelo relacional [33] con las simplificaciones consideradas en la asignatura CI-3311 (Sistemas de Bases de Datos I):

- Region(regionkey, name, comment).
- Nation(nationkey, name, region
regionkey, comment).
- Part(partkey, name, mfgr, brand, type, size, container, retailprice, comment).
- Customer(custkey, name, address, nation
nationkey, phone, acctbal, mktsegment, comment).
- Supplier(suppkey, name, address, nation
nationkey, phone, acctbal, comment).
- Orders(orderkey, customer
custkey, orderstatus, totalprice, orderpriority, clerk, shippriority, comment).
- Lineitem(orders
orderkey, linenumber, partsupp
partkey, supplier, quantity, extendedprice, discount, tax, returnflag, linestatus, shipdate, commitdate, receiptdate, shipinstruct, shipmode, comment).
- Partsupp(part
partkey, supplier
suppkey, availqty, supplycost, comment).

A.2. Consultas

A continuación se presenta una lista con una breve descripción conceptual de cada consulta del *benchmark* TPC-H [33].

1. Informe del resumen de precios: informa sobre la cantidad de negocios facturados, enviados y devueltos.
2. Proveedor de costo mínimo: encuentra el proveedor que se debe seleccionar para realizar un pedido de una pieza determinada en una región específica.

3. Prioridad de envío: recupera los 10 pedidos no enviados con el mayor valor.
4. Comprobación de prioridad de pedidos: determina el desempeño del sistema de prioridad de pedidos y proporciona una evaluación de la satisfacción del cliente.
5. Volumen de proveedores locales: indica el volumen de ingresos generados a través de proveedores locales.
6. Pronóstico de cambios en los ingresos: cuantifica el aumento de ingresos que se habría producido al eliminar ciertos descuentos a nivel de toda la empresa en un rango porcentual determinado en un año determinado.
7. Volumen de envíos: determina el valor de las mercancías enviadas entre ciertos países para facilitar la renegociación de los contratos de envío.
8. Cuota de mercado nacional: determina cómo ha cambiado la cuota de mercado de un país determinado dentro de una región específica a lo largo de dos años para un tipo de pieza determinado.
9. Medida de utilidad por tipo de producto: determina la utilidad obtenida de una línea de piezas determinada, desglosada por país del proveedor y año.
10. Informe de devoluciones: identifica a los clientes que podrían tener problemas con las piezas que se les envían.
11. Identificación de stock importante: encuentra el subconjunto más importante del stock de los proveedores en un país determinado.
12. Modos de envío y prioridad de pedidos: determina si la selección de modos de envío más económicos afecta negativamente a los pedidos de prioridad crítica, al provocar que los clientes reciban más piezas después de la fecha límite.
13. Distribución de clientes: busca relaciones entre los clientes y el tamaño de sus pedidos.
14. Efecto de promoción: monitoriza la respuesta del mercado a una promoción, como anuncios de televisión o una campaña especial.
15. Proveedor principal: identifica al proveedor principal para recompensarlo, aumentar su negocio o identificarlo para un reconocimiento especial.
16. Relación piezas/proveedor: determina cuántos proveedores pueden suministrar piezas con atributos determinados. Puede utilizarse, por ejemplo, para determinar si hay suficientes proveedores para piezas con gran demanda.
17. Ingresos por pedidos de pequeña cantidad: determina la pérdida promedio de ingresos anuales si no se surtieran pedidos de pequeñas cantidades de ciertas piezas. Esto puede reducir los gastos generales al concentrar las ventas en envíos más grandes.
18. Cliente de gran volumen: clasifica a los clientes según si han realizado un pedido de gran cantidad. Los pedidos de gran cantidad se definen como aquellos cuya cantidad total supera un cierto nivel.

19. Ingresos descontados: informa los ingresos brutos descontados atribuidos a la venta de piezas seleccionadas, gestionadas de una manera específica. Esta consulta es un ejemplo de código que podría generarse programáticamente mediante una herramienta de minería de datos.
20. Promoción potencial de piezas: identifica a los proveedores de un país en particular que han seleccionado piezas que podrían ser candidatas a una oferta promocional.
21. Proveedores con pedidos en espera: identifica a ciertos proveedores que no pudieron enviar las piezas requeridas a tiempo.
22. Oportunidad de venta global: identifica zonas geográficas donde hay clientes con probabilidades de realizar una compra.

```

1 /* TPC-H Q97 */
2 SELECT
3   o_year,
4   SUM(CASE
5     WHEN nation = 'BRAZIL'
6     THEN volume
7     ELSE 0
8   ) / SUM(volume) AS mkt_share
9 FROM (
10   SELECT
11     EXTRACT(YEAR FROM o_orderdate) AS o_year,
12     l_extendedprice * (1 - l_discount) AS volume,
13     n2.n_name AS nation
14   FROM
15     part,
16     supplier,
17     lineitem,
18     orders,
19     customer,
20     nation n1,
21     nation n2
22   WHERE
23     /* JOIN lineitem on */ s_suppkey = l_suppkey
24     /* JOIN orders on */ o_orderkey = l_orderkey
25     /* JOIN customer on */ c_custkey = o_custkey
26     /* JOIN nation on */ n1.nationkey = n2.nationkey
27     /* Filtro complejo */
28     AND (
29       (n1.n_name = 'FRANCE' AND n2.n_name = 'GERMANY')
30       OR (n1.n_name = 'GERMANY' AND n2.n_name = 'FRANCE')
31     )
32     AND l_shipdate BETWEEN '1995-01-01' AND '1996-12-31'
33 ) AS shipping
34 GROUP BY
35   o_year,
36   cust_nation,
37   l_year;

```

```

1 /* TPC-H Q88 */
2 SELECT
3   o_year,
4   SUM(CASE
5     WHEN nation = 'BRAZIL'
6     THEN volume
7     ELSE 0
8   ) / SUM(volume) AS mkt_share
9 FROM (
10   SELECT
11     EXTRACT(YEAR FROM o_orderdate) AS o_year,
12     l_extendedprice * (1 - l_discount) AS volume,
13     n2.n_name AS nation
14   FROM
15     part,
16     supplier,
17     lineitem,
18     orders,
19     customer,
20     nation n1,
21     nation n2,
22     region
23   WHERE
24     /* JOIN lineitem on */ p_partkey = l_partkey
25     /* JOIN supplier on */ s_suppkey = l_suppkey
26     /* JOIN orders on */ o_orderkey = l_orderkey
27     /* JOIN customer on */ c_custkey = o_custkey
28     /* JOIN nation on */ n1.nationkey = n2.nationkey
29     /* JOIN region on */ n1.n_regionkey = r.regionkey
30     AND r.name = 'AMERICA'
31     /* JOIN nation on */ n1.nationkey = n2.nationkey
32     AND o_orderdate BETWEEN '1995-01-01' AND '1996-12-31'
33     AND p_type = 'ECONOMY-ANODIZED-STEEL'
34 ) AS all_nations
35 GROUP BY
36   o_year,
37   o_year;

```

```

1 /* TPC-H Q99 */
2 SELECT
3   nation,
4   o_year,
5   SUM(amount) AS sum_profit
6 FROM (
7   SELECT
8     n.name AS nation,
9     EXTRACT(YEAR FROM o_orderdate) AS o_year,
10    l_extendedprice * (1 - l_discount) - ps_supplycost * l_quantity AS amount
11  FROM
12    part,
13    supplier,
14    lineitem,
15    partsupp,
16    orders,
17    nation
18  WHERE
19    /* JOIN on supplier */ s_suppkey = l_suppkey
20    /* JOIN on partsupp */ AND ps_suppkey = l_suppkey
21    /* JOIN on partsupp */ AND ps_partkey = l_partkey
22    /* JOIN on lineitem */ AND p_partkey = l_partkey
23    /* JOIN on orders */ AND o_orderkey = l_orderkey
24    /* JOIN on nation */ AND s_nationkey = n.nationkey
25    AND p.name LIKE 'green'
26 ) AS profit
27 GROUP BY
28   nation,
29   o_year
30 ORDER BY
31   nation,
32   o_year DESC;

```

Figura A.1: Consultas inherentemente pesadas

En la figura A.1 se muestran las operaciones inherentemente costosas para el gestor y para la consulta. Sólo las tablas **nation** (veinticinco filas), **region** (cinco filas) y **supplier** (10 000S filas con S igual al número de gigabytes de la base de datos)

```

1 /* TPC-H Q92 */
2 SELECT
3   o_year,
4   SUM(CASE
5     WHEN nation = 'EUROPE'
6     THEN volume
7     ELSE 0
8   ) / SUM(volume) AS mkt_share
9 FROM (
10   SELECT
11     EXTRACT(YEAR FROM o_orderdate) AS o_year,
12     l_extendedprice * (1 - l_discount) AS volume,
13     n2.n_name AS nation
14   FROM
15     part,
16     supplier,
17     lineitem,
18     orders,
19     customer,
20     nation n1,
21     nation n2
22   WHERE
23     /* JOIN lineitem on */ s_suppkey = l_suppkey
24     /* JOIN orders on */ o_orderkey = l_orderkey
25     /* JOIN customer on */ c_custkey = o_custkey
26     /* JOIN nation on */ n1.nationkey = n2.nationkey
27     /* Filtro complejo */
28     AND (
29       (n1.n_name = 'FRANCE' AND n2.n_name = 'GERMANY')
30       OR (n1.n_name = 'GERMANY' AND n2.n_name = 'FRANCE')
31     )
32     AND l_shipdate BETWEEN '1995-01-01' AND '1996-12-31'
33 ) AS shipping
34 GROUP BY
35   o_year,
36   cust_nation,
37   l_year;

```

```

1 /* TPC-H Q86 */
2 SELECT
3   SUM(l_extendedprice * l_discount) AS revenue
4 FROM
5   lineitem
6 WHERE
7   l_shipdate >= '1994-01-01'
8   AND l_shipdate <= '1995-01-01'
9   AND l_discount BETWEEN 0.05 AND 0.07
10  AND l_quantity < 24

```

```

1 /* TPC-H Q19 */
2 SELECT
3   SUM(l_extendedprice * (1 - l_discount)) AS revenue
4 FROM
5   lineitem /* by part # para co-locación */
6   part
7 WHERE
8   (
9     p_partkey = l_partkey
10    /* Filtros varios
11   */
12   )
13   (
14     p_partkey = l_partkey
15     /* Filtros varios
16   */
17   )
18   (
19     p_partkey = l_partkey
20     /* Filtros varios
21   */

```

Figura A.3: Consultas 2, 6, 19

```

1  /* TPC-H Q17 */
2  SELECT
3    SUM(l_extendedprice) / 7.0 AS avg_yearly
4  FROM
5    lineitem_by_part,
6    part
7  WHERE
8    p_partkey = l_partkey
9    AND p_brand = 'Brand#23'
10   AND p_container = 'MED BOX'
11   AND l_quantity < (
12     SELECT
13       0.2 * AVG(l_quantity)
14     FROM
15       lineitem_by_part
16     WHERE
17       l_partkey = p_partkey
18   )

```

```

1  /* TPC-H Q20 */
2  SELECT
3    ....,
4  FROM
5    ....,
6  WHERE
7    s_suppkey IN (
8      SELECT
9        ps_suppkey
10     FROM
11       partsupp
12     WHERE
13       ps_partkey IN (
14         SELECT
15           p_partkey
16         FROM
17           part
18         WHERE
19           p_name LIKE 'forest%'
20       )
21     AND ps_availqty > (
22       SELECT
23         0.5 * SUM(l_quantity)
24       FROM
25         lineitem_by_part
26       WHERE
27         l_partkey = ps_partkey
28         AND l_suppkey = ps_suppkey
29         AND l_shipdate >= DATE('1994-01-01')
30         AND l_shipdate < DATE('1995-01-01')
31     )
32   )
33   ....,
34  ORDER BY
35    s_name

```

Figura A.2: Consultas Q17 y Q20

```

1  /* TPC-H Q11 */
2  SELECT
3    ps_partkey,
4    SUM(ps_supplycost * ps_availqty) AS value
5  FROM
6    partsupp,
7    supplier,
8    nation
9  WHERE
10     ps_suppkey = s_suppkey -- JOIN de tablas co-locadas
11     AND s_nationkey = n_nationkey -- JOIN de tablas co-locadas
12     AND n_name = 'GERMANY'
13  GROUP BY
14     ps_partkey HAVING
15     SUM(ps_supplycost * ps_availqty) > (
16       -- Consulta anidada: correlacionada
17       SELECT
18         SUM(ps_supplycost * ps_availqty) * 0.0001
19       FROM
20         partsupp,
21         supplier,
22         nation
23       WHERE
24         ps_suppkey = s_suppkey
25         AND s_nationkey = n_nationkey
26         AND n_name = 'GERMANY'
27     )
28  ORDER BY
29     value DESC

```

```

1  /* TPC-H Q18 */
2  SELECT
3    c_name,
4    c_custkey,
5    o_orderkey,
6    o_orderdate,
7    o_totalprice,
8    SUM(l_quantity)
9  FROM
10     customer,
11     orders o,
12     orders by_customer oc,
13     lineitem
14  WHERE
15     o_orderkey IN ( -- Consulta anidada: correlacionada
16       SELECT
17         l_orderkey
18       FROM
19         lineitem
20       GROUP BY
21         l_orderkey HAVING SUM(l_quantity) > 300
22     )
23     AND c_custkey = oc_o_custkey -- JOIN de tablas co-locadas
24     AND o_orderkey = l_orderkey -- JOIN de tablas co-locadas
25     AND o_orderkey = oc_o_custkey -- JOIN de tablas co-locadas
26  GROUP BY
27     c_name,
28     c_custkey,
29     o_orderkey,
30     o_orderdate,
31     o_totalprice
32  ORDER BY
33     o_totalprice DESC,
34     o_orderdate
35  LIMIT 100;

```

```

1  /* TPC-H Q22 */
2  SELECT
3    c_ntrycode,
4    COUNT(*) AS numcust,
5    SUM(c_acctbal) AS totacctbal
6  FROM (
7    SELECT
8      SUBSTRING(c_phone FROM 1 FOR 2) AS c_ntrycode,
9      c_acctbal
10   FROM
11     customer
12   WHERE
13     SUBSTRING(c_phone FROM 1 FOR 2) IN
14     ('13','31','23','29','30','18','17')
15     AND c_acctbal > ( -- Consulta anidada: correlacionada
16       SELECT
17         AVG(c_acctbal)
18       FROM
19         customer
20       WHERE
21         c_acctbal > 0.00
22         AND SUBSTRING(c_phone FROM 1 FOR 2) IN
23         ('13','31','23','29','30','18','17')
24     )
25     AND NOT EXISTS ( -- Consulta anidada: correlacionada
26       SELECT
27         *
28       FROM
29         orders_by_customer
30       WHERE
31         o_custkey = c_custkey
32     )
33 ) AS custsale
34 GROUP BY
35     c_ntrycode
36  ORDER BY
37     c_ntrycode

```

Figura A.4: Consultas 11, 18, 22

```

1  /* TPC-H Q01 */
2  SELECT
3    l_returnflag,
4    l_linestatus,
5    SUM(l_quantity) AS sum_qty,
6    SUM(l_extendedprice) AS sum_base_price,
7    SUM(l_extendedprice * (1 - l_discount)) AS sum_disc_price,
8    SUM(l_extendedprice * (1 - l_discount) * (1 + l_tax)) AS sum_charge,
9    AVG(l_quantity) AS avg_qty,
10   AVG(l_extendedprice) AS avg_price,
11   AVG(l_discount) AS avg_disc,
12   COUNT(*) AS count_order
13  FROM
14     lineitem
15  WHERE
16     l_shipdate <= date '1998-09-01' - INTERVAL '90 days'
17  GROUP BY
18     l_returnflag,
19     l_linestatus
20  ORDER BY
21     l_returnflag,
22     l_linestatus;

```

```

1  /* TPC-H Q01 Reescrita */
2  WITH precalculos AS (
3    SELECT
4      l_returnflag,
5      l_linestatus,
6      l_quantity,
7      l_extendedprice,
8      l_discount,
9      l_extendedprice * (1 - l_discount) AS descuento,
10     l_extendedprice * (1 - l_discount) * (1 + l_tax) AS cargos
11   FROM
12     lineitem
13   WHERE
14     l_shipdate <= '1998-09-01'
15 )
16 SELECT
17   l_returnflag,
18   l_linestatus,
19   SUM(l_quantity) AS sum_qty,
20   SUM(l_extendedprice) AS sum_base_price,
21   SUM(descuento) AS sum_disc_price,
22   SUM(cargos) AS sum_charge,
23   AVG(l_quantity) AS avg_qty,
24   AVG(l_extendedprice) AS avg_price,
25   AVG(l_discount) AS avg_disc,
26   COUNT(*) AS count_order
27  FROM
28     precalculos
29  GROUP BY
30     l_returnflag,
31     l_linestatus
32  ORDER BY
33     l_returnflag,
34     l_linestatus;

```

```

1  /* TPC-H Q05 */
2  SELECT
3    n_name,
4    SUM(l_extendedprice * (1 - l_discount)) AS revenue
5  FROM
6    customer,
7    orders,
8    lineitem,
9    supplier,
10   nation,
11   region
12  WHERE
13     c_custkey = o_custkey
14     AND l_orderkey = o_orderkey
15     AND l_suppkey = s_suppkey
16     AND c_nationkey = s_nationkey
17     AND s_nationkey = n_nationkey
18     AND n_regionkey = r_regionkey
19     AND r_name = 'ASIA'
20     AND o_orderdate >= '1994-01-01'
21     AND o_orderdate <= '1995-01-01'
22  GROUP BY
23     n_name
24  ORDER BY
25     revenue DESC

```

Figura A.5: Consultas 1 (original, izquierda y reescrita, centro), 5 y 21

```

1  /* TPC-H Q13 */
2  SELECT
3    c_count,
4    COUNT(*) AS custdist
5  FROM ( -- Consulta anidada: correlacionada
6    SELECT
7      c_custkey,
8      COUNT(o_orderkey)
9    FROM
10     customer LEFT OUTER JOIN orders by_customer ON
11     c_custkey = o_custkey AND o_comment NOT LIKE '%special%requests%'
12   GROUP BY
13     c_custkey
14 ) AS c_orders (c_custkey, c_count)
15 GROUP BY
16     c_count
17  ORDER BY
18     custdist DESC,
19     c_count DESC

```

```

1  /* TPC-H Q13 Reescrita */
2  WITH ordenes AS (
3    SELECT
4      c_custkey,
5      COUNT(o_orderkey)
6    FROM
7      customer LEFT OUTER JOIN orders by_customer ON
8      c_custkey = o_custkey AND o_comment NOT LIKE '%special%requests%'
9    GROUP BY
10     c_custkey
11 )
12 SELECT
13   c_count,
14   COUNT(*) AS custdist
15  FROM
16     ordenes
17  GROUP BY
18     c_count
19  ORDER BY
20     custdist DESC,
21     c_count DESC;

```

Figura A.6: Consulta 13

```

1  /* TPC-H Q16 */
2  SELECT
3    p_brand,
4    p_type,
5    p_size,
6    COUNT(DISTINCT ps_suppkey) AS supplier_cnt
7  FROM
8    partsupp,
9    part
10 WHERE
11   p_partkey = ps_partkey # JOIN de tablas co-locadas
12   AND p_brand <> 'Brand#45'
13   AND p_type NOT LIKE 'MEDIUM POLISHED%'
14   AND p_size IN (49, 14, 23, 45, 19, 3, 36, 9)
15   AND ps_suppkey NOT IN ( # Consulta anidada correlacionada
16     SELECT
17       s_suppkey
18     FROM
19       supplier
20     WHERE
21       s_comment LIKE '%Customer%Complaints%'
22   )
23 GROUP BY
24   p_brand,
25   p_type,
26   p_size
27 ORDER BY
28   supplier_cnt DESC,
29   p_brand,
30   p_type,
31   p_size;

```

```

1  /* TPC-H Q16 Reescrita */
2  WITH proveedores AS (
3    SELECT
4      s_suppkey
5    FROM
6      supplier
7    WHERE
8      s_comment LIKE '%Customer%Complaints%'
9  )
10 SELECT
11   p_brand,
12   p_type,
13   p_size,
14   COUNT(DISTINCT ps_suppkey) AS supplier_cnt
15 FROM
16   partsupp,
17   part
18 WHERE
19   p_partkey = ps_partkey # JOIN de tablas co-locadas
20   AND p_brand <> 'Brand#45'
21   AND p_type NOT LIKE 'MEDIUM POLISHED%'
22   AND p_size IN (49, 14, 23, 45, 19, 3, 36, 9)
23   AND ps_suppkey NOT EXISTS (
24     SELECT
25       1
26     FROM
27       proveedores p
28     WHERE
29       p.s_suppkey = ps_suppkey
30   )
31 GROUP BY
32   p_brand,
33   p_type,
34   p_size
35 ORDER BY
36   supplier_cnt DESC,
37   p_brand,
38   p_type,
39   p_size;

```

Figura A.7: Consulta 16

```

WHERE
    s_suppkey = l1.l_suppkey
    AND o_orderkey = l1.l_orderkey
    AND o_orderstatus = 'F'
    AND l1.l_receiptdate > l1.l_commitdate
    AND EXISTS (
        SELECT
            *
        FROM
            lineitem l2
        WHERE
            l2.l_orderkey = l1.l_orderkey
            AND l2.l_suppkey <> l1.l_suppkey
    )
    AND NOT EXISTS (
        SELECT
            *
        FROM
            lineitem l3
        WHERE
            l3.l_orderkey = l1.l_orderkey
            AND l3.l_suppkey <> l1.l_suppkey
            AND l3.l_receiptdate > l3.l_commitdate
    )
    AND s_nationkey = n_nationkey

```

Figura A.8: JOIN en Q21

<pre> 1 /* TPC-H Q03 */ 2 SELECT 3 ..l_orderkey, 4 ..SUM(l_extendedprice*(1-..l_discount)) AS revenue, 5 ..o_orderdate, 6 ..o_shippriority 7 FROM 8 ..customer, 9 ..orders, 10 ..lineitem 11 WHERE 12 ..c_mktsegment = 'BUILDING' 13 AND c_custkey = o_custkey # JOIN de tablas co-locadas 14 AND l_orderkey = o_orderkey # JOIN de tablas co-locadas 15 AND o_orderdate < '1995-03-15' 16 AND l_shipdate > '1995-03-15' 17 GROUP BY 18 ..l_orderkey, 19 ..o_orderdate, 20 ..o_shippriority 21 ORDER BY 22 ..revenue DESC, 23 ..o_orderdate 24 LIMIT 10; </pre>	<pre> 1 /* TPC-H Q04 */ 2 SELECT 3 ..o_orderpriority, 4 ..COUNT(*) AS order_count 5 FROM 6 ..orders 7 WHERE 8 ..o_orderdate >= '1993-07-01' 9 AND o_orderdate < '1993-10-01' 10 AND EXISTS (# Consulta anidada correlacionada 11 ..SELECT 12 ..* 13 FROM 14 ..lineitem 15 WHERE 16 ..l_orderkey = o_orderkey # JOIN de tablas co-locadas 17 ..AND l_commitdate < l_receiptdate 18) 19 GROUP BY 20 ..o_orderpriority 21 ORDER BY 22 ..o_orderpriority; </pre>
--	--

Figura A.9: Consultas 3 y 4

```

1  /* TPC-H Q10 */
2  SELECT
3    c_custkey,
4    c_name,
5    SUM(l_extendedprice * (1 - l_discount)) AS revenue,
6    c_acctbal,
7    n_name,
8    c_address,
9    c_phone,
10   c_comment
11 FROM
12   customer,
13   orders,
14   lineitem,
15   nation
16 WHERE
17   c_custkey = o_custkey # JOIN de tablas NO co-locadas
18   AND l_orderkey = o_orderkey # JOIN de tablas co-locadas
19   AND o_orderdate >= '1993-10-01'
20   AND o_orderdate < '1994-01-01'
21   AND l_returnflag = 'R'
22   AND c_nationkey = n_nationkey # JOIN de tablas co-locadas
23 GROUP BY
24   c_custkey,
25   c_name,
26   c_acctbal,
27   c_phone,
28   n_name,
29   c_address,
30   c_comment
31 ORDER BY
32   revenue DESC
33 LIMIT 20;

```

```

1  /* TPC-H Q14 */
2  SELECT
3    100.00 * SUM(CASE
4      WHEN p_type LIKE 'PROMO%'
5      THEN l_extendedprice * (1 - l_discount)
6      ELSE 0
7    END) / SUM(l_extendedprice * (1 - l_discount)) AS promo_revenue
8 FROM
9   lineitem,
10  part
11 WHERE
12   l_partkey = p_partkey # JOIN de tablas NO co-locadas
13   AND l_shipdate >= '1995-09-01'
14   AND l_shipdate < '1995-10-01';
15

```

Figura A.10: Consultas 10 y 14

Apéndice B. Instalación de herramientas

B.1. Scaphandre

```
1 sudo apt install -y cargo pkg-config libssl-dev
2 curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
3 # Cuando pregunte, elegir la opcion por defecto (1)
4 source $HOME/.cargo/env
5 rustup install 1.64.0
6 rustup default 1.64.0 # Esto hara que corra la version 1.64.0 globalmente.
7 git clone https://github.com/hubblo-org/scaphandre.git #Codigo fuente
8 cd scaphandre && cargo build --release && cd .. # compilar el codigo
9 # Anadir la ruta de scaphandre a PATH para su uso en cualquier directorio
10 echo 'export PATH=$HOME/scaphandre/target/release:$PATH' >> ~/.bashrc
11 source ~/.bashrc
12 # Esto lo volvera ejecutable como superusuario (sudo)
13 sudo cp $HOME/scaphandre/target/release/scaphandre /usr/local/bin/
```

B.2. Prometheus

La instalación es directa y vía apt:

```
1 sudo apt install prometheus -y
```

Se debe modificar el archivo `/etc/prometheus/prometheus.yml` de forma que se vea así:

```
1 # Sample config for Prometheus.
2
3 global:
4   scrape_interval:      15s # ... Default is every 1 minute.
5   evaluation_interval: 15s # Evaluate rules ... The default is every 1 minute.
6   # scrape_timeout is set to the global default (10s).
7
8   ...
9 scrape_configs:
10  ...
11
12  - job_name: 'citus_energy'
13    # scrape_interval: 1s
14    scrape_timeout: 10s
15    static_configs:
16      - targets: ['159.90.9.10:8080', '159.90.9.11:8080', '159.90.9.12:8080',
17                  '159.90.9.13:8080']
```

Para, finalmente, reiniciar el servicio mediante `systemctl`:

```
1 promtool check config /etc/prometheus/prometheus.yml
2 sudo systemctl restart prometheus
3 sudo systemctl status prometheus
```

La herramienta `promtool` permite verificar la sintaxis del archivo de configuración editado para garantizar que el reinicio del servicio sea exitoso. Sin embargo, como medida adicional, se sugiere verificar el estado del servicio con el último paso.

B.3. Jq

Este es un procesador de archivos JSON de CLI. Su instalación también es directa mediante `apt-get`:

```
1 sudo apt-get install jq
```

Su función es la de tomar las métricas recolectadas por Prometheus y procesarlas en entradas legibles para humanos.

B.4. Dbgen de TPC-H

Es el generador de datos aleatorios para el *benchmark* TPC-H. Se obtiene todo el lote mediante descarga directa en su sitio oficial¹. La descarga requiere que el usuario llene un formulario en señal de aceptación de los términos y condiciones de uso. El formulario puede llenarse de forma segura con los datos de la afiliación a la Universidad Simón Bolívar, o a otra institución de ser preciso. Este formulario hará que se envíe un correo electrónico con el enlace de descarga a la dirección que el usuario suministró.

Una vez descargado el archivo se deben seguir los siguientes pasos para completar la instalación solamente en uno de los computadores:

```
1 unzip ~/Downloads/<archivo>.zip -d ~ && mv ~/TPC-H\ V3.0.1 ~/TPC-H
2 cd ~/TPC-H/dbgen
3 nano makefile.suite
4 # En este paso se debe navegar por el archivo para encontrar las lineas:
5 # CC = gcc
6 # DATABASE = SQLSERVER
7 # MACHINE = LINUX
8 # WORKLOAD = TPCH
9 make -f makefile.suite
```

B.5. Configuración de clústeres

Para la instalación de un clúster de bases de datos relacionales como PostgreSQL se necesitan extensiones de estos gestores que funcionen como *middlewares* y versiones de los mismos gestores que sean compatibles. El *middleware* de PostgreSQL es Citus y el proceso de instalación es:

```
1 sudo apt update
2 sudo apt install -y build-essential git curl postgresql-16 libkrb5-dev \
3 postgresql-server-dev-16 libcurl4-openssl-dev liblz4-dev libzstd-dev
4
```

¹https://www.tpc.org/tpc_documents_current_versions/current_specifications5.asp

```

5 git clone https://github.com/citusdata/citus.git && cd citus
6 git checkout v12.1.1
7
8 ./configure PG_CONFIG=/usr/lib/postgresql/16/bin/pg_config
9
10 make
11 sudo make install
12
13 sudo -u postgres psql -c "CREATE DATABASE <base-de-datos>;"
14 sudo -u postgres psql -d <base-de-datos> -c "CREATE EXTENSION citus;"

```

El nombre <base-de-datos> es libre para el usuario con la única condición de que no haya sido usado previamente.

Cuando la creación ha sido exitosa, se le debe indicar a Citus quién es el nodo administrador (*manager*) y quiénes son los nodos trabajadores (*workers*). Para ello, primero, hay que conectar los nodos del clúster mediante el archivo de configuración `/etc/postgresql/16/main/pg_hba.conf`. En este archivo hay que añadir líneas al final de él, una por cada ordenador a utilizar:

```

1 host      all             postgres      <IP1>/32      trust
2 host      all             postgres      <IP2>/32      trust
3 host      all             postgres      <IP3>/32      trust
4 host      all             postgres      <IP4>/32      trust

```

Cada <IP> corresponde a cada ordenador y esto permite que todos los ordenadores puedan enviarse resultados parciales de las consultas de ser requerido. Luego, dentro de `psql` se debe indicar la identidad de cada nodo a Citus:

```

1 sudo -u postgres psql -d <base-de-datos> -c "SELECT citus_set_coordinator_host
2 ('<IP1>');"
3 sudo -u postgres psql -d <base-de-datos> -c "SELECT * from citus_add_node('<IP2
4 >', 5432);"
5 sudo -u postgres psql -d <base-de-datos> -c "SELECT * from citus_add_node('<IP3
6 >', 5432);"
7 sudo -u postgres psql -d <base-de-datos> -c "SELECT * from citus_add_node('<IP4
8 >', 5432);"

```

Esto finaliza la configuración del clúster.

Apéndice C. Medias y desviaciones estándar del consumo energético por nodos

C.1. Consumo nodo coordinador

Tabla C.1: Sin diseño

Carga (GB) Consulta	1	2	5	10	20	50
Q1	0,0(2)	0(5)	$0(1) \times 10^1$	$0(2) \times 10^1$	$0(2) \times 10^1$	$0(2) \times 10^1$
Q2	0,001(1)	0,002(7)	0,00(1)	0,00(1)	0,0(5)	0(7)
Q3	0,1(6)	0,0(1)	0(2)	1(3)	7(9)	$2(1) \times 10^1$
Q4	0,004(2)	0,003(8)	0,00(2)	0(2)	0(3)	0(4)
Q5	0(1)	0(2)	$0(1) \times 10^1$	1(5)	4(7)	$2(1) \times 10^1$
Q6	0,00(1)	0,00(2)	0,00(1)	0(2)	0(2)	0(2)
Q7	0(1)	0,6(9)	$0(1) \times 10^1$	1(3)	3(3)	$1(1) \times 10^1$
Q8	0(4)	3(7)	$1(2) \times 10^1$	$1(2) \times 10^1$	$3(2) \times 10^1$	$1,4(3) \times 10^2$
Q9	0(9)	$1(2) \times 10^1$	$2(2) \times 10^1$	$5(2) \times 10^1$	$1,1(3) \times 10^2$	$2,9(5) \times 10^2$
Q10	0,2(3)	0,1(5)	1(1)	3(3)	14(5)	$7(1) \times 10^1$
Q11	0,002(3)	0,002(4)	0,00(1)	0,0(2)	0(1)	0(2)
Q12	0,003(2)	0,00(1)	0,01(1)	0(2)	0(2)	0(2)
Q13	0,002(3)	0,00(1)	0,004(9)	0(6)	0(9)	$0(2) \times 10^1$
Q14	0,0(4)	0,0(2)	0(2)	0(2)	5(2)	9(6)
Q15	0,010(6)	0,12(1)	1(4)	4(3)	11(5)	36(9)
Q16	0,008(3)	0,014(5)	1,9(1)	7,1(2)	23(7)	$7(1) \times 10^1$
Q17	0(7)	$0(1) \times 10^1$	$0(2) \times 10^1$	0(2)	0(2)	$0(6) \times 10^1$
Q18	0,002(4)	0,003(4)	0,00(1)	0,00(2)	0(2)	0(9)
Q19	0,0(4)	0(1)	0,00(1)	4(3)	6(4)	27(8)
Q20	$0(1) \times 10^1$	$0(1) \times 10^1$	2(4)	0(4)	0(6)	0(9)
Q21	0,002(4)	0,00(1)	$0(2) \times 10^1$	0(3)	0(4)	0(6)
Q22	0,002(1)	0,003(1)	0,006(7)	0,04(8)	0,01(4)	0,1(4)

Tabla C.2: Con diseño

Carga (GB) Consulta	1	2	5	10	20	50
Q1	0,0(2)	0,01(2)	0(9)	0(9)	$0(1) \times 10^1$	0(8)
Q2	0,001(3)	0,002(2)	0,002(6)	0,00(2)	0,00(1)	0(3)
Q3	0,0(5)	0,1(6)	0(2)	$0(1) \times 10^1$	$0(1) \times 10^1$	$4(2) \times 10^1$
Q4	0,001(4)	0,001(4)	0,01(8)	0,0(5)	0(4)	0(9)
Q5	0,1(9)	0,1(6)	0(1)	$0(2) \times 10^1$	4(8)	$5(1) \times 10^1$
Q6	0,001(2)	0,001(3)	0,002(3)	0,002(9)	0,002(8)	0,0(2)
Q7	0,6(5)	0,7(8)	1(3)	$0(1) \times 10^1$	$0(2) \times 10^1$	$5(2) \times 10^1$
Q8	0(3)	$0(1) \times 10^1$	$1(2) \times 10^1$	$1(2) \times 10^1$	$3(2) \times 10^1$	$5(3) \times 10^1$
Q9	0(7)	$1(2) \times 10^1$	$2(2) \times 10^1$	$5(2) \times 10^1$	$1,1(2) \times 10^2$	$6(8) \times 10^1$
Q10	0,4(6)	0,3(4)	0,7(7)	3(1)	13(9)	$5(2) \times 10^1$
Q11	0,001(2)	0,001(2)	0,026(9)	0,00(2)	0,07(8)	0,01(7)
Q12	0,001(2)	0,002(5)	0,00(1)	0,001(9)	0(4)	0(4)
Q13	0,01(8)	0,1(4)	0,005(9)	$0(2) \times 10^4$	$1(2) \times 10^1$	$0(4) \times 10^1$
Q14	0,1(1)	0(4)	0,1(4)	2(1)	4(3)	3(9)
Q15	0,006(2)	0,0(6)	1,2(5)	4(1)	13(8)	4(8)
Q16	0,007(2)	0,01(1)	2,48(5)	6,3(3)	18946(5)	$0(2) \times 10^1$
Q17	0,001(4)	$0(1) \times 10^1$	0,001(3)	0,001(3)	0,002(9)	0,002(6)
Q18	0,002(6)	0,00(1)	0,001(7)	0,00(2)	0,00(2)	0(3)
Q19	0,001(2)	0,001(4)	0,003(7)	0,00(1)	0,00(1)	0,001(4)
Q20	0,00(2)	0,003(3)	0,00(1)	0,01(2)	0,01(3)	0(2)
Q21	0,00(1)	0,00(2)	0,01(6)	0(2)	0(3)	0(3)
Q22	0,002(2)	0,0018(8)	0,02(7)	0,0(1)	0,03(7)	0,02(6)

C.2. Consumo nodo trabajador 1

Tabla C.3: Sin diseño

Carga (GB) Consulta	1	2	5	10	20	50
Q1	0,1(2)	2(5)	$3(1) \times 10^1$	$9(2) \times 10^1$	$2,2(2) \times 10^2$	$5,9(2) \times 10^2$
Q2	0,012(1)	0,016(7)	0,02(1)	0,01(1)	0,1(5)	27(7)
Q3	0,5(6)	0,3(1)	1(2)	22(3)	51(9)	$1,5(1) \times 10^2$
Q4	0,025(2)	0,045(8)	0,05(2)	13(2)	32(3)	82(4)
Q5	0(1)	1(2)	$1(1) \times 10^1$	43(5)	85(7)	$1,1(1) \times 10^2$
Q6	0,04(1)	0,04(2)	0,04(1)	6(2)	15(2)	45(2)
Q7	0(1)	0,9(9)	$1(1) \times 10^1$	23(3)	40(3)	$1,10(1) \times 10^2$
Q8	0(4)	0(7)	$3(2) \times 10^1$	$9(2) \times 10^1$	$1,8(2) \times 10^2$	$4,3(3) \times 10^2$
Q9	0(9)	$4(2) \times 10^1$	$1,2(2) \times 10^2$	$2,1(2) \times 10^2$	$4,1(3) \times 10^2$	$1,01(5) \times 10^3$
Q10	0,4(3)	0,4(5)	2(1)	18(3)	34(5)	$1,0(1) \times 10^2$
Q11	0,014(3)	0,014(4)	0,02(1)	0,1(2)	0(1)	17(2)
Q12	0,042(2)	0,05(1)	0,04(1)	10(2)	22(2)	59(2)
Q13	0,017(3)	0,01(1)	0,023(9)	2(6)	14(9)	$8(2) \times 10^1$
Q14	0,5(4)	0,4(2)	2(2)	10(2)	20(2)	56(6)
Q15	0,026(6)	0,05(1)	4(4)	22(3)	43(5)	116(9)
Q16	0,010(3)	0,012(5)	0,0(1)	0,2(2)	4(7)	$2(1) \times 10^1$
Q17	0(7)	$1,4(1) \times 10^2$	$1,06(2) \times 10^3$	8(2)	27(2)	$2,6(6) \times 10^2$
Q18	0,024(4)	0,025(4)	0,02(1)	0,02(2)	2(2)	15(9)
Q19	0,4(4)	0(1)	0,02(1)	19(3)	35(4)	93(8)
Q20	$0(1) \times 10^1$	$2,3(1) \times 10^2$	4(4)	15(4)	40(6)	116(9)
Q21	0,046(4)	0,04(1)	$1,61(2) \times 10^3$	19(3)	42(4)	129(6)
Q22	0,011(1)	0,013(4)	0,2(4)	0,1(3)	2(3)	6(6)

Tabla C.4: Con diseño

Carga (GB) Consulta	1	2	5	10	20	50
Q1	0,1(2)	0,05(2)	8(9)	47(9)	$1,0(1) \times 10^2$	289(8)
Q2	0,007(3)	0,009(2)	0,007(6)	0,03(2)	0,02(1)	16(3)
Q3	0,6(5)	0,5(6)	1(2)	$1(1) \times 10^1$	$5(1) \times 10^1$	$1,6(2) \times 10^1$
Q4	0,008(4)	0,017(4)	0,03(8)	0,1(5)	19(4)	77(9)
Q5	0,6(9)	0,5(6)	1(1)	$2(2) \times 10^1$	133(8)	$9(1) \times 10^1$
Q6	0,008(2)	0,009(3)	0,005(3)	0,012(9)	0,012(8)	0,1(2)
Q7	0,6(5)	0,8(8)	2(3)	$1(1) \times 10^1$	$3(2) \times 10^1$	$1,3(2) \times 10^2$
Q8	0(3)	$1(1) \times 10^1$	$4(2) \times 10^1$	$9(2) \times 10^1$	$1,7(2) \times 10^2$	$4,5(3) \times 10^2$
Q9	0(7)	$4(2) \times 10^1$	$1,2(2) \times 10^2$	$2,0(2) \times 10^2$	$4,1(2) \times 10^2$	$9,82(8) \times 10^2$
Q10	0,5(6)	0,4(4)	1,2(7)	3(1)	11(9)	$5(2) \times 10^1$
Q11	0,006(2)	0,008(2)	0,017(9)	0,01(2)	0,04(8)	0,05(7)
Q12	0,010(2)	0,041(5)	0,01(1)	0,010(9)	3(4)	53(4)
Q13	0,04(8)	0,1(4)	0,022(9)	$0(2) \times 10^4$	$2(2) \times 10^1$	$4(4) \times 10^1$
Q14	0,4(1)	0(4)	0,5(4)	1(1)	3(3)	11(9)
Q15	0,010(2)	0,1(6)	0,4(5)	2(1)	12(8)	26(8)
Q16	0,010(2)	0,01(1)	0,03(5)	0,3(3)	23(5)	$3(2) \times 10^1$
Q17	0,006(4)	$1,4(1) \times 10^2$	0,005(3)	0,004(3)	0,011(9)	0,018(6)
Q18	0,020(6)	0,02(1)	0,006(7)	0,01(2)	0,01(2)	1(3)
Q19	0,006(2)	0,006(4)	0,013(7)	0,01(1)	0,01(1)	0,021(4)
Q20	0,01(2)	0,009(3)	0,01(1)	0,01(2)	0,02(3)	0(2)
Q21	0,05(1)	0,05(2)	0,04(6)	14(2)	34(3)	87(3)
Q22	0,009(2)	0,014(4)	0,04(8)	1(2)	1(2)	10(8)

C.3. Consumo nodo trabajador 2

Tabla C.5: Sin diseño

Carga (GB) Consulta	1	2	5	10	20	50
Q1	0,1(4)	4(7)	$3(1) \times 10^1$	$1,0(1) \times 10^2$	$2,3(2) \times 10^2$	$5,9(3) \times 10^2$
Q2	0,013(2)	0,019(6)	0,02(2)	0,01(1)	0(1)	25(8)
Q3	0,6(7)	0,3(1)	2(3)	$2(1) \times 10^1$	59(9)	$1,4(2) \times 10^2$
Q4	0,027(3)	0,047(7)	0,04(1)	9(4)	33(5)	81(7)
Q5	0(1)	1(2)	10(8)	$4(2) \times 10^1$	94(7)	103(8)
Q6	0,04(1)	0,04(1)	0,1(2)	6(4)	15(1)	48(5)
Q7	0(1)	2(4)	5(6)	13(8)	47(7)	$1,1(1) \times 10^2$
Q8	0(3)	10(8)	$2(1) \times 10^1$	$8(2) \times 10^1$	$1,9(2) \times 10^1$	$4,1(4) \times 10^2$
Q9	$0(1) \times 10^1$	$3(2) \times 10^1$	$1,0(3) \times 10^2$	$2,1(3) \times 10^2$	$4,6(2) \times 10^1$	$9,4(6) \times 10^2$
Q10	0,6(6)	0,7(7)	2(2)	$1(1) \times 10^1$	32(5)	$1,0(1) \times 10^2$
Q11	0,014(3)	0,014(5)	0,03(2)	0,0(1)	0,03(2)	$1(1) \times 10^1$
Q12	0,040(2)	0,05(1)	0,1(5)	10(3)	21(2)	66(5)
Q13	0,017(2)	0,02(1)	0,04(1)	1(3)	15(9)	$9(2) \times 10^1$
Q14	0,5(5)	0,7(6)	1(1)	12(3)	22(3)	54(7)
Q15	0,03(1)	0,3(9)	5(5)	$2(2) \times 10^1$	53(7)	$1,5(2) \times 10^1$
Q16	0,012(3)	0,02(2)	0,04(7)	1(2)	$1(1) \times 10^1$	$4(2) \times 10^1$
Q17	0(9)	$1,5(1) \times 10^2$	$1,25(2) \times 10^3$	15(5)	62(6)	212(8)
Q18	0,025(6)	0,028(6)	0,03(1)	0,03(3)	4(4)	14(8)
Q19	0,4(4)	0(1)	0,03(1)	15(5)	35(4)	100(8)
Q20	$0(1) \times 10^1$	$2,3(1) \times 10^2$	3(3)	9(2)	15(7)	54(4)
Q21	0,045(6)	0,05(1)	$1,66(2) \times 10^3$	$2(1) \times 10^1$	45(5)	127(9)
Q22	0,011(1)	0,012(5)	1(2)	0,1(2)	0,2(6)	11(5)

Tabla C.6: Con diseño

Carga (GB) Consulta	1	2	5	10	20	50
Q1	0,1(1)	3(5)	10(9)	$4(2) \times 10^1$	$1,0(2) \times 10^2$	$2,6(2) \times 10^2$
Q2	0,008(2)	0,009(2)	0,011(9)	0,03(2)	0,03(1)	12(1)
Q3	0,5(3)	0,5(3)	1(1)	9(7)	$5(2) \times 10^1$	$1,4(2) \times 10^2$
Q4	0,009(2)	0,019(5)	0,02(2)	4(2)	1(2)	40(5)
Q5	0,7(7)	0,6(8)	2(2)	$1(1) \times 10^1$	$5(3) \times 10^1$	$1,0(1) \times 10^2$
Q6	0,009(2)	0,010(3)	0,02(1)	0,02(1)	0,015(7)	0,2(5)
Q7	0,9(8)	0,7(9)	4(6)	6(9)	$3(1) \times 10^1$	$1,2(3) \times 10^2$
Q8	0(3)	$1(1) \times 10^1$	$3(1) \times 10^1$	$8(2) \times 10^1$	$1,8(2) \times 10^2$	$5,3(3) \times 10^2$
Q9	0(9)	$3(1) \times 10^1$	$1,1(3) \times 10^2$	$1,9(2) \times 10^2$	$4,4(3) \times 10^2$	$9(1) \times 10^2$
Q10	0,5(4)	0,4(3)	1,4(8)	3(2)	7(4)	$4(2) \times 10^1$
Q11	0,008(2)	0,009(1)	0,05(4)	0,01(2)	0,014(7)	0,025(5)
Q12	0,011(3)	0,033(7)	0,01(2)	0,010(9)	3(3)	32(7)
Q13	0,04(7)	0,1(6)	0,03(1)	$2(9) \times 10^3$	$2(1) \times 10^1$	$9(2) \times 10^1$
Q14	0,6(7)	0(5)	0,5(3)	1,1(7)	1,9(9)	10(7)
Q15	0,010(3)	0,1(5)	1(1)	2(1)	$2(1) \times 10^1$	$6(3) \times 10^1$
Q16	0,011(3)	0,011(3)	0,02(3)	1(1)	1(2)	$4(2) \times 10^1$
Q17	0,006(4)	$1,5(1) \times 10^1$	0,006(4)	0,006(3)	0,01(1)	0,018(6)
Q18	0,023(7)	0,03(1)	0,008(8)	0,01(2)	0,03(5)	0(2)
Q19	0,007(3)	0,006(2)	0,018(9)	0,01(1)	0,02(2)	0,023(5)
Q20	0,01(1)	0,009(3)	0,01(1)	0,01(2)	0,003(2)	3(6)
Q21	0,046(9)	0,04(1)	1(3)	$1(1) \times 10^1$	33(4)	87(5)
Q22	0,011(2)	0,014(3)	0,02(2)	0,3(9)	0,1(4)	9(5)

Apéndice D. Consumo nodos trabajadores

En las figuras siguientes (D.1 a 5.10) se presentarán los diagramas de barras para el consumo energético de cada consulta con cada volumen de datos. Luego se dará un análisis global de las tendencias observadas en cada gráfica. En el apéndice C se muestran las tablas con las medias y las desviaciones estándar.

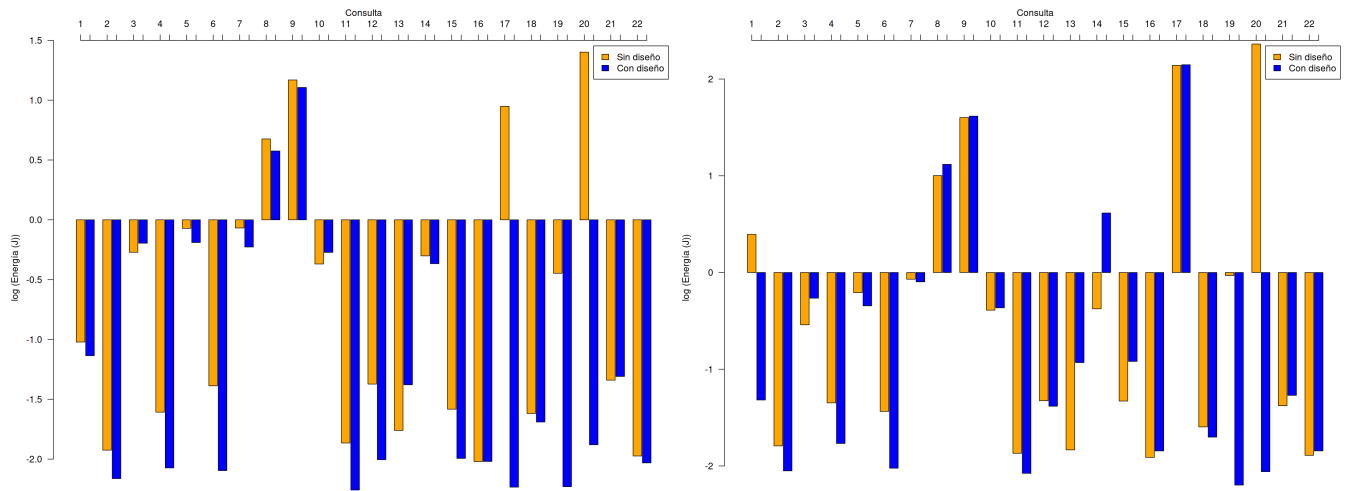


Figura D.1: Consumo del trabajador uno por consulta 1 GB (izquierda) y 2 GB (derecha)

En la figura D.1 se puede apreciar la siguiente división de consultas según si el consumo es poco, moderado o alto: para un gigabyte, consumen poco las consultas 1, 2, 3, 4, 5, 6, 7, 10, 11, 12, 13, 14, 15, 16, Q17(idx), 18, 19, Q20(idx), 21 y 22; mientras que para dos gigabytes son Q1(CTE), 2, 3, 4, 5, 6, 7, 10, 11, 12, 13, Q14, 15, 16, 18, 19, Q20(idx), 21, 22. Las que presentan consumo moderado son: la 8 para un gigabyte y Q1, Q8, Q14(idx) para dos gigabytes. Las de consumo alto son: 9, Q17 y Q20 para un giga y Q8(idx), 9, 17 y Q20 para dos.

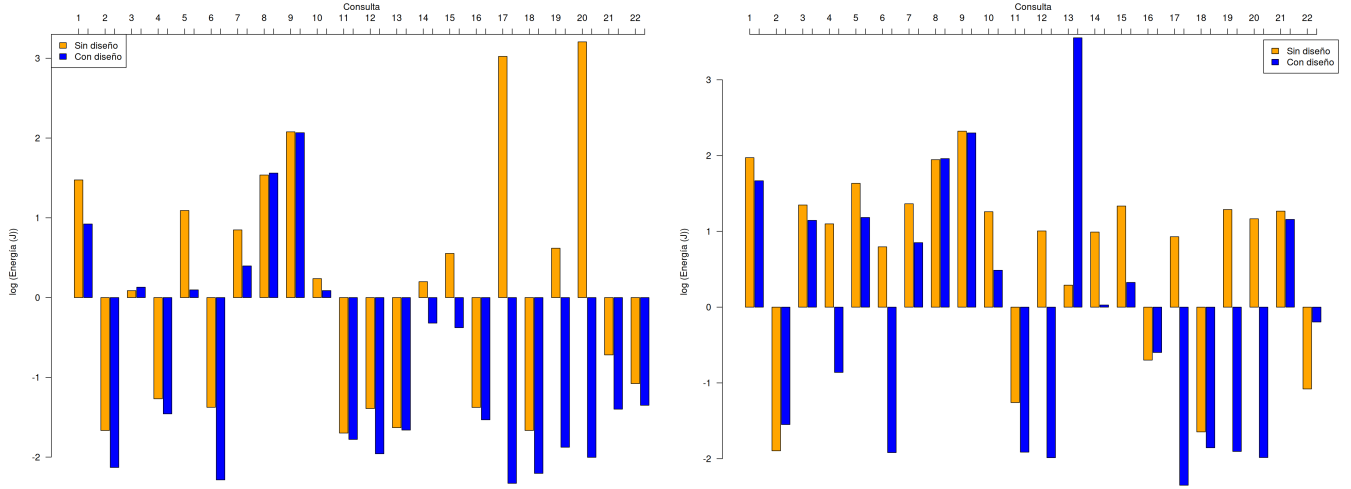


Figura D.2: Consumo del trabajador uno por consulta 5 GB (izquierda) y 10 GB (derecha)

De la figura D.2 se tiene la clasificación de consultas como se sigue: para cinco gigabytes, consumen poco las consultas 2, 4, 6, 11, 12, 13, Q14(idx), Q15(idx), 16, Q17(idx), 18, Q19(*), Q20(idx) y 22 mientras que para diez gigabytes son 2, Q4(idx), Q6(idx), 11, Q12(idx), 16, Q17(idx), 18, Q19(*), Q20(idx) y 22. Las consultas de consumo moderado son: Q1(CTE), 3, Q5(idx), 7, 10, Q13, Q14, Q19 para cinco gigabytes mientras que para diez gigabytes son Q6, Q7(idx), Q10(idx), Q12, Q13, 14, Q15(idx) y Q17. Finalmente, las de consumo alto son: Q1, Q5, 8, 9, Q17 y Q20 para cinco gigabytes y 1, 3, Q4, 5, Q7, 8, 9, Q10, Q13(CTE), Q15, Q19, Q20 y 21 para diez.

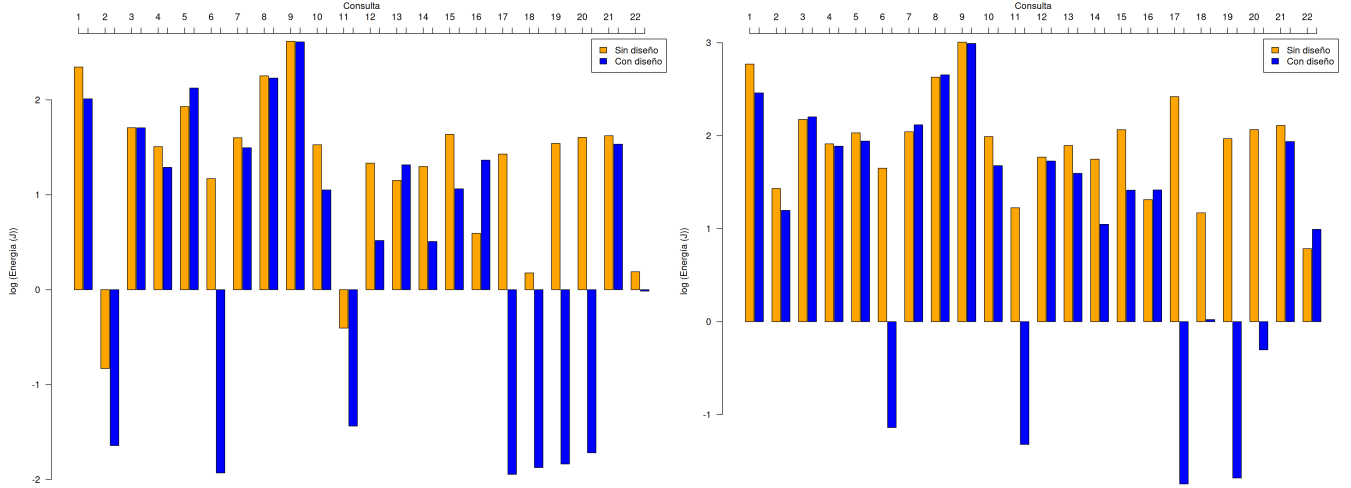


Figura D.3: Consumo del trabajador uno por consulta 20 GB (izquierda) y 50 GB (derecha)

De la figura D.3 se sigue la siguiente clasificación de las consultas: para veinte gigabytes, consumen poco las consultas 2, Q6(idx), 11, Q17(idx), Q18(*), Q19(*) y Q20(idx) y para cincuenta gigabytes son Q6(idx), Q11(idx), Q17(idx), Q19(*) y Q20(idx). Para las que exhiben consumo moderado son: Q12(idx), Q14(idx), Q15(idx), Q16, Q18 y 22 para veinte gigabytes mientras que para cincuenta gigabytes son Q14(idx), Q18(*) y 22. Por último, las consultas de consumo alto son: 1, 3, 4, 5, Q6, 7, 8, 9, 10, Q12, 13, Q14, Q15, Q16(CTE), Q17, Q18, Q19, Q20 y 21 para veinte gigabytes y para cincuenta gigabytes se incluyen además Q11, Q12(idx), Q15(idx) y Q16.

Análogamente para el segundo trabajador, en la figura D.4 se tiene la clasificación: 1, 2, 3, 4, 5, 6, 7, 10, 11, 12, 13, 14, 15, 16, Q17(idx), 18, 19, Q20(idx), 21 y 22 son las que consumen poco con un gigabyte mientras que con dos son 2, 3, 4, Q5(idx), 6, Q7(idx), 10, 11, 12, 13, Q14, 15, 16, 18, 19, Q20(idx), 21 y 22. Las de consumo moderado son 8 para un gigabyte y 1, Q5, Q7, Q8 y Q14(idx) para dos. Finalmente, las de consumo alto son: 9, Q17 y Q20 para un giga mientras que para dos son Q8(idx), 9, 17 y Q20.

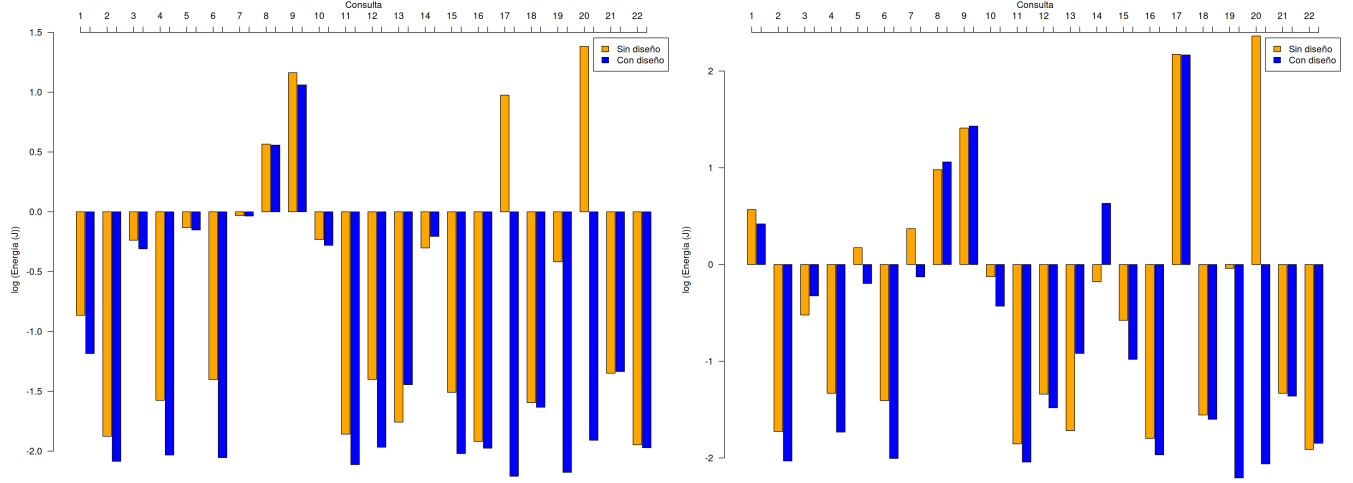


Figura D.4: Consumo del trabajador dos por consulta 1 GB (izquierda) y 2 GB (derecha)

De la figura D.5 se desprende la siguiente clasificación: las que consumen poco son 2, 4, 6, 11, 12, 13, Q14(idx), Q15(idx), 16, Q17(idx), 18, Q19(*), Q20(idx), 21 y 22 para cinco gigabytes y para diez gigabytes son 2, Q6(idx), 11, Q12(idx), Q16(CTE), Q17(idx), 18, Q19(*), Q20(idx) y 22. Las consultas de consumo energético moderado son Q1(CTE), 3, 5, 7, 10, Q14, Q15 y Q19 para cinco gigabytes y con diez gigabytes son Q3(idx), 4, Q6, Q7(idx), 10, Q12, Q13, 14, Q15(idx), Q16 y Q20. Y las de consumo alto son Q1, 8, 9, Q17 y Q20 para cinco gigabytes y para diez son 1, Q3, 5, Q7, 8, 9, Q13(CTE), Q15, Q17, Q19 y 21.

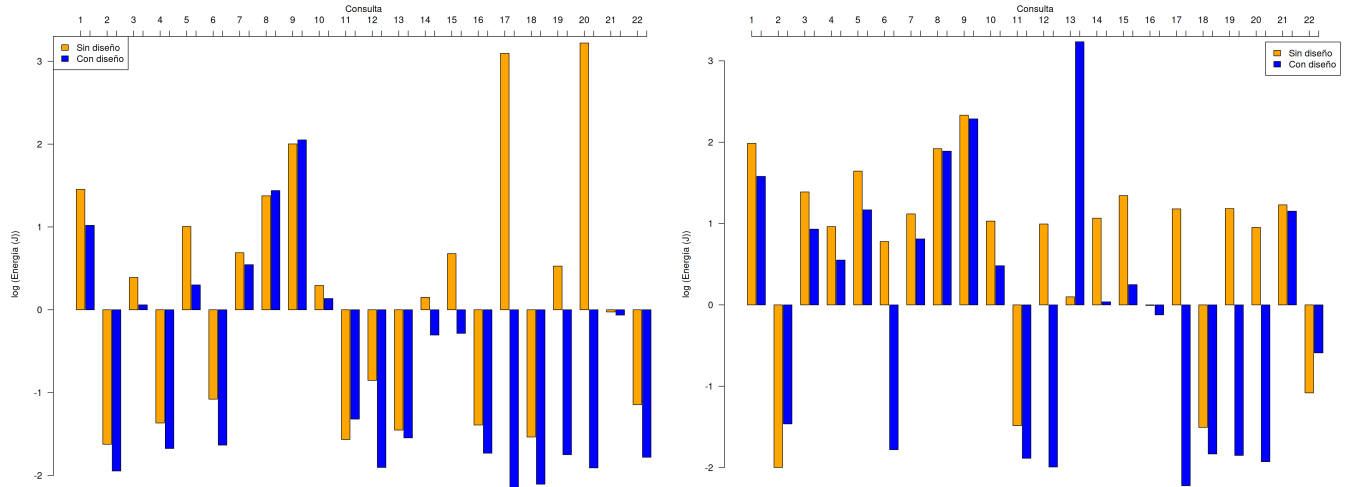


Figura D.5: Consumo del trabajador dos por consulta 5 GB (izquierda) y 10 GB (derecha)

En la figura D.6 se observa la clasificación: 2, Q4(idx), Q6(idx), 11, Q17(idx), Q18(*), Q19(*),

Q20(idx) y 22 son las consultas que consumen poca energía para la carga de veinte gigabytes y para cincuenta gigabytes son Q6(idx), Q11(idx), Q17(idx), Q18(*) y Q19(*). Las de consumo moderado son Q10(idx), Q12(idx), Q14(idx), 16 y Q18 con veinte gigabytes y Q14(idx), Q20(idx) y 22 para cincuenta. Finalmente, las consultas de consumo alto son 1, 3, Q4, 5, Q6, 7, 8, 9, Q10, Q12, 13, Q14, 15, Q17, Q19, Q20 y 21 para veinte gigabytes y para cincuenta son 1, 2, 3, 4, 5, Q6, 7, 8, 9, 10, Q11, 12, 13, Q14, 15, 16, Q17, Q18, Q19, Q20 y 21.

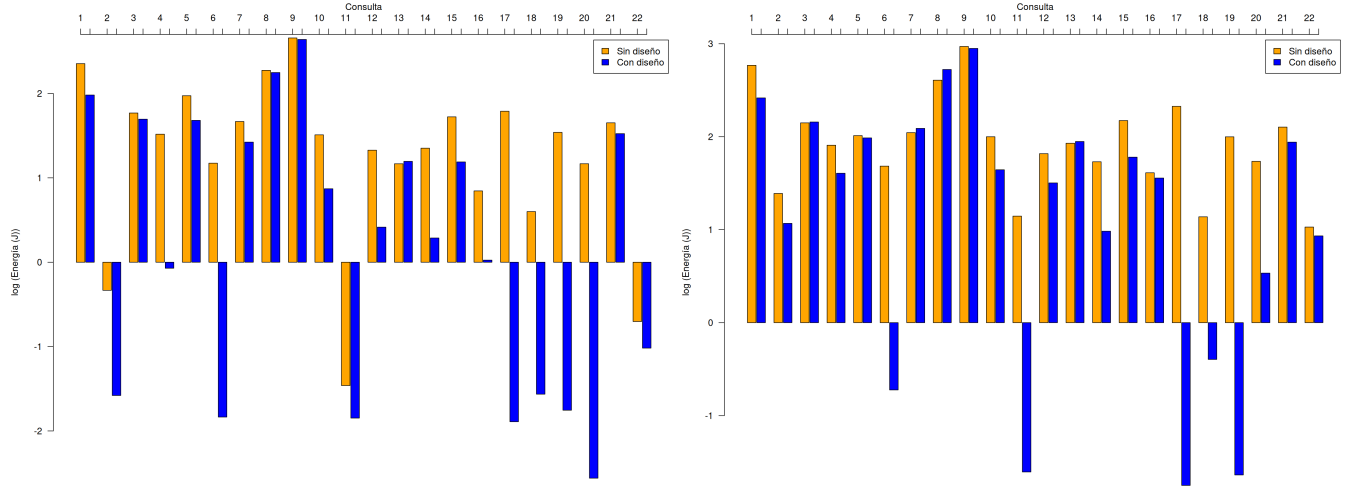


Figura D.6: Consumo del trabajador dos por consulta 20 GB (izquierda) y 50 GB (derecha)

Apéndice E. Gráficos de dispersión sin diseño

E.1. Consumo neto

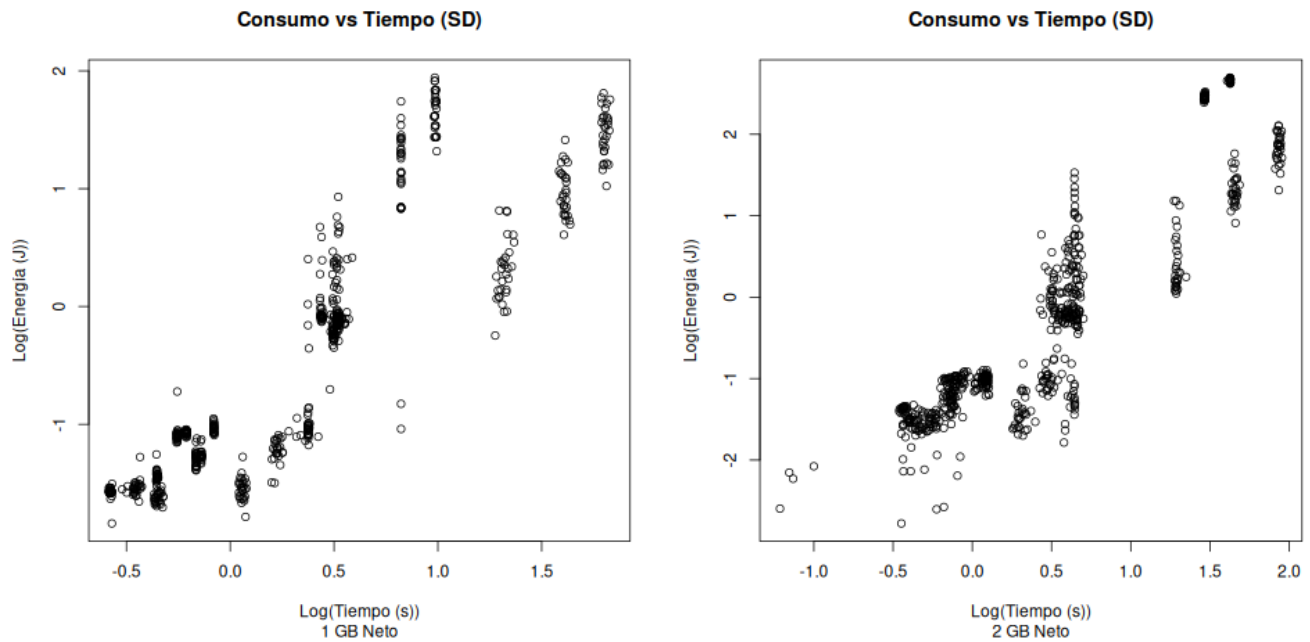


Figura E.1: Gráficos de dispersión de consumo neto para 1 GB y 2 GB

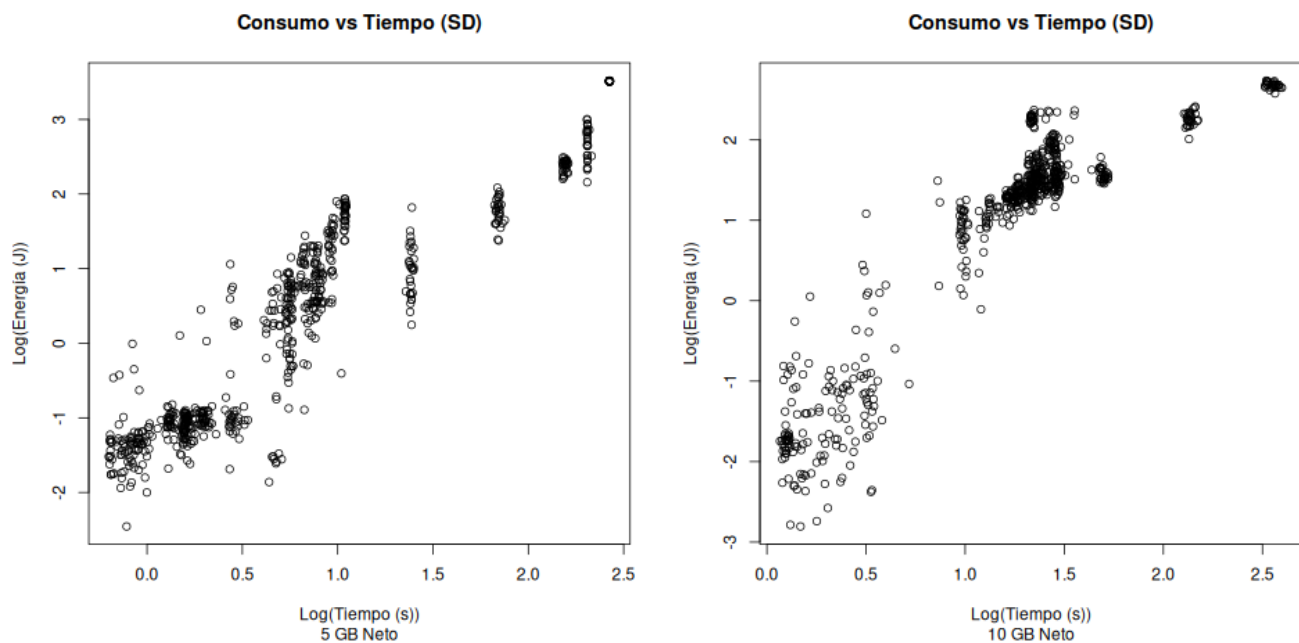


Figura E.2: Gráficos de dispersión de consumo neto para 5 GB y 10 GB

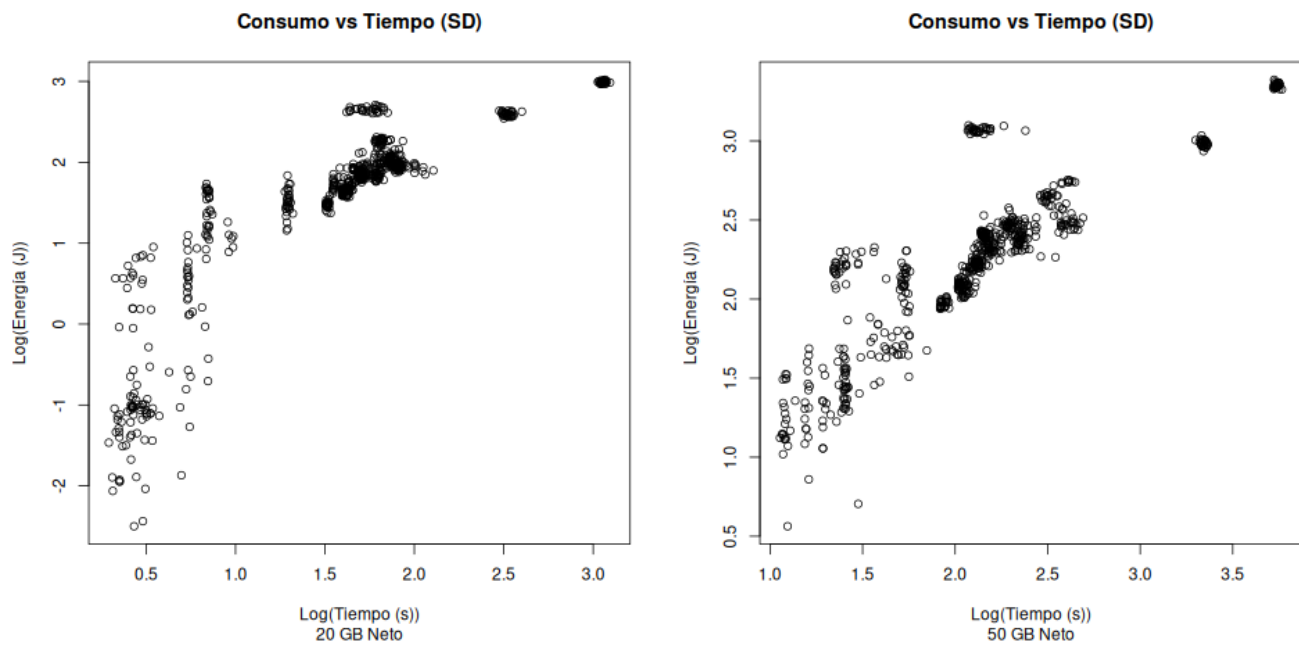


Figura E.3: Gráficos de dispersión de consumo neto para 20 GB y 50 GB

E.2. Consumo trabajador 1

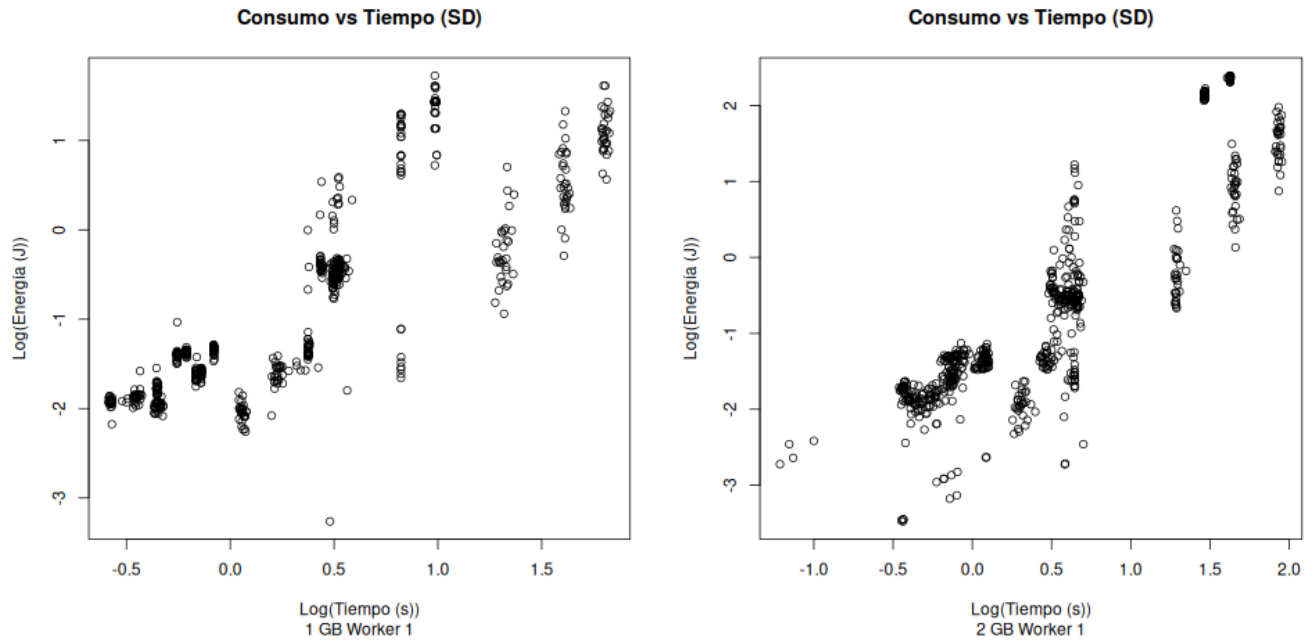


Figura E.4: Gráficos de dispersión de consumo del trabajador 1 para 1 GB y 2 GB

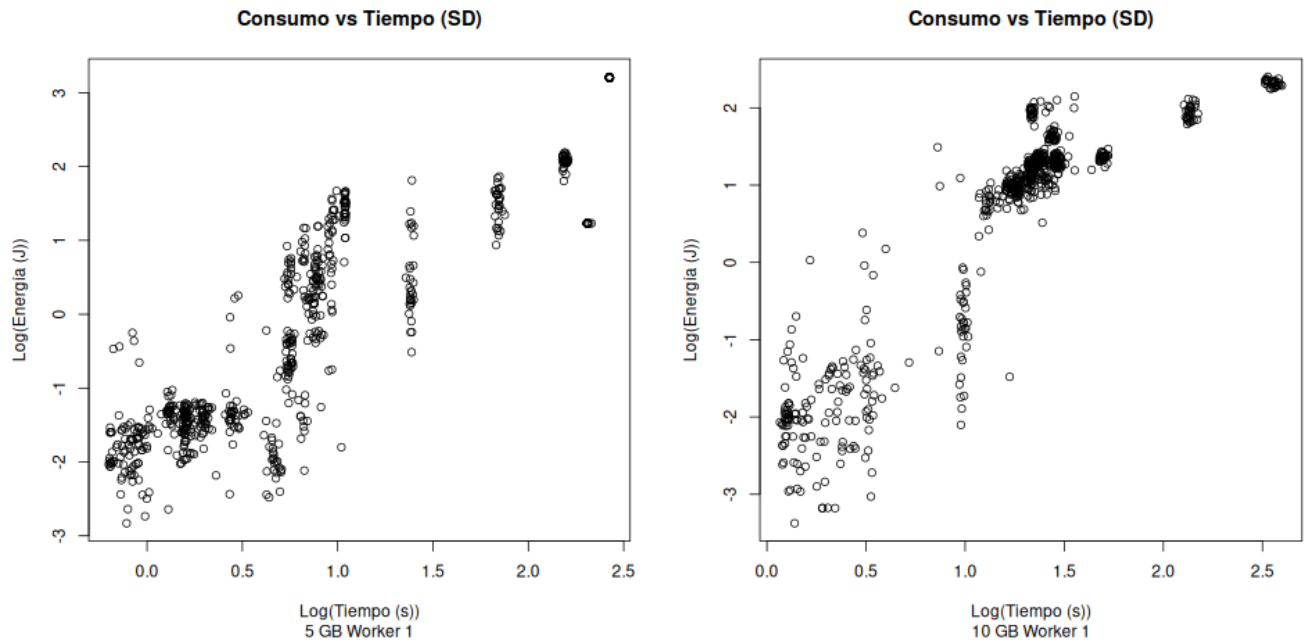


Figura E.5: Gráficos de dispersión de consumo del trabajador 1 para 5 GB y 10 GB

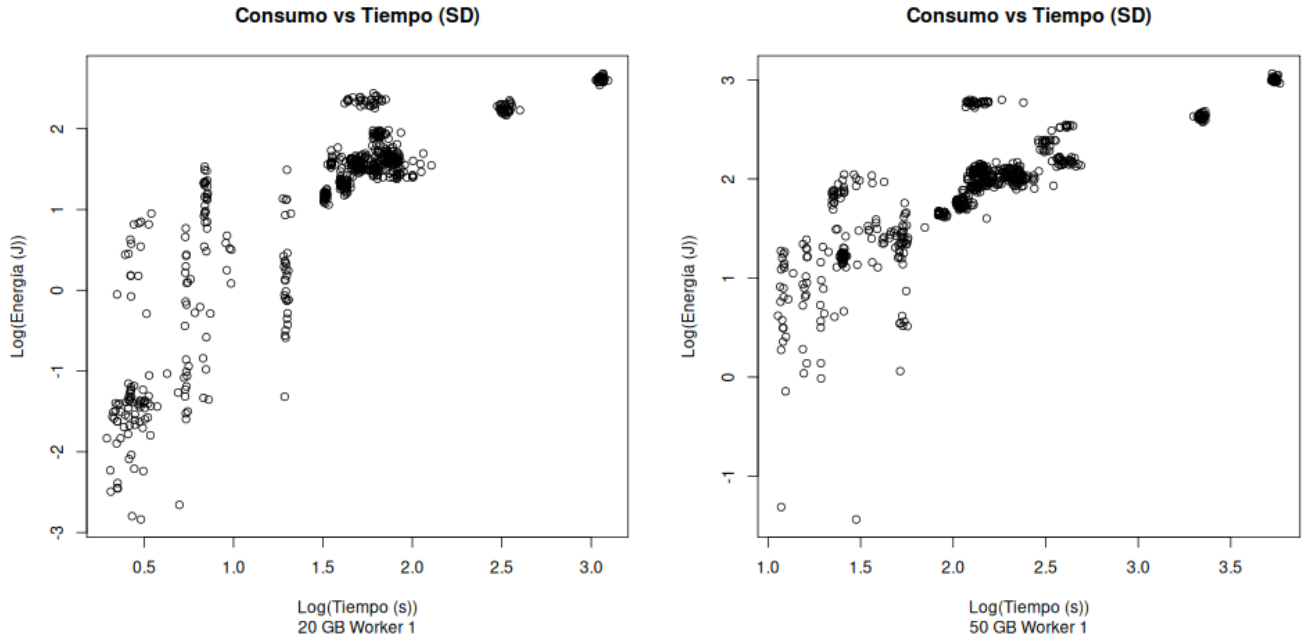


Figura E.6: Gráficos de dispersión de consumo del trabajador 1 para 20 GB y 50 GB

E.3. Consumo trabajador 2

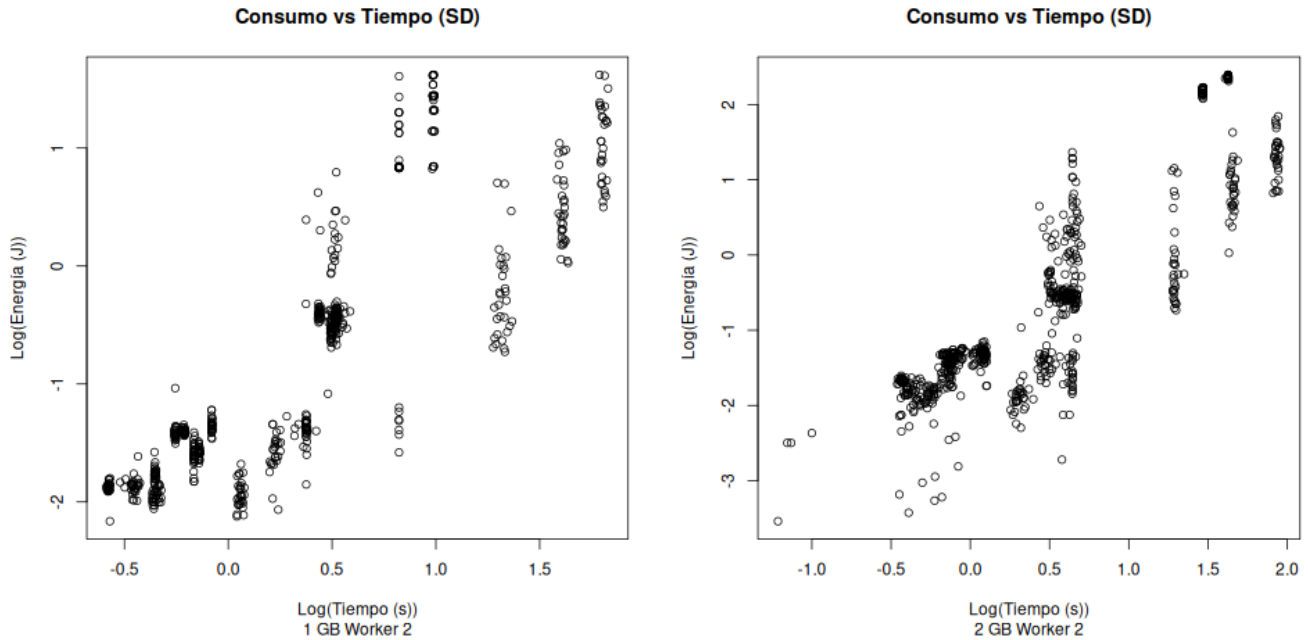


Figura E.7: Gráficos de dispersión de consumo del trabajador 2 para 1 GB y 2 GB

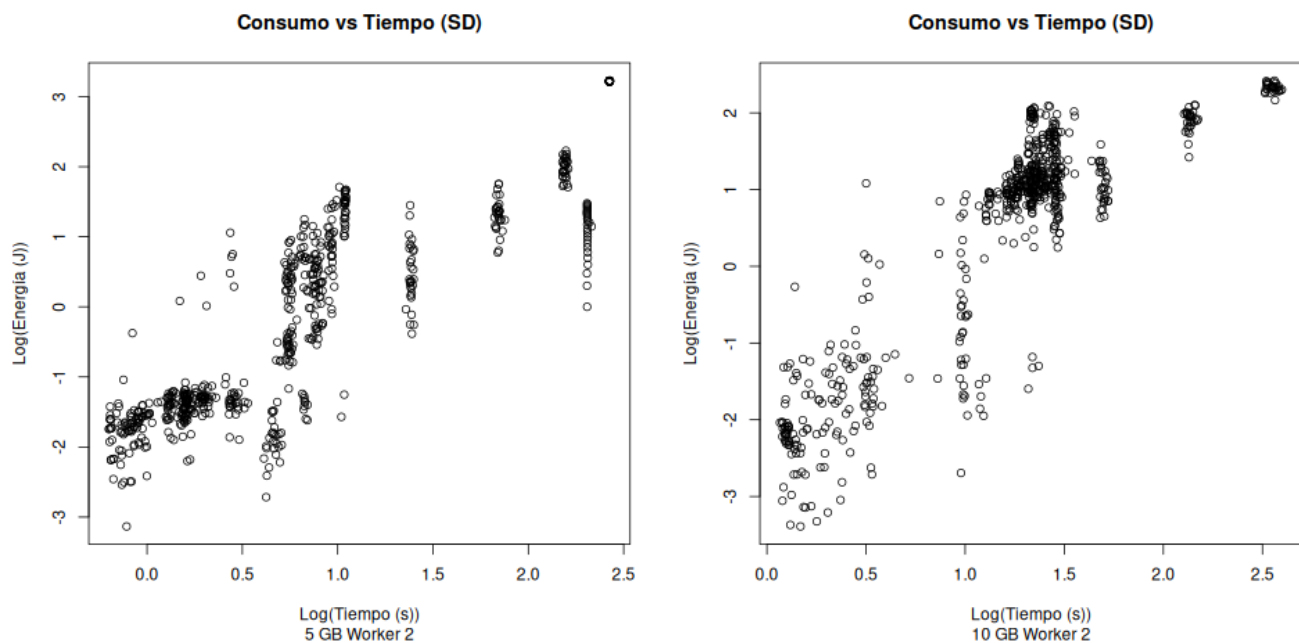


Figura E.8: Gráficos de dispersión de consumo del trabajador 2 para 5 GB y 10 GB

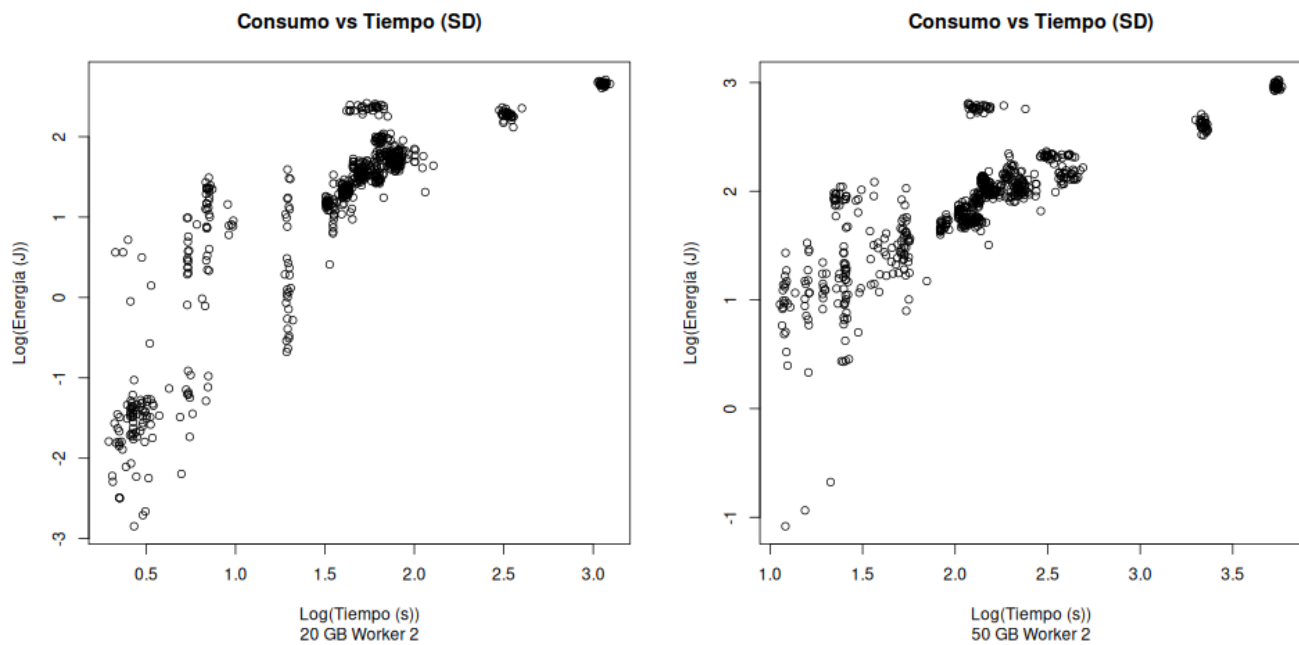


Figura E.9: Gráficos de dispersión de consumo del trabajador 2 para 20 GB y 50 GB

E.4. Consumo coordinador

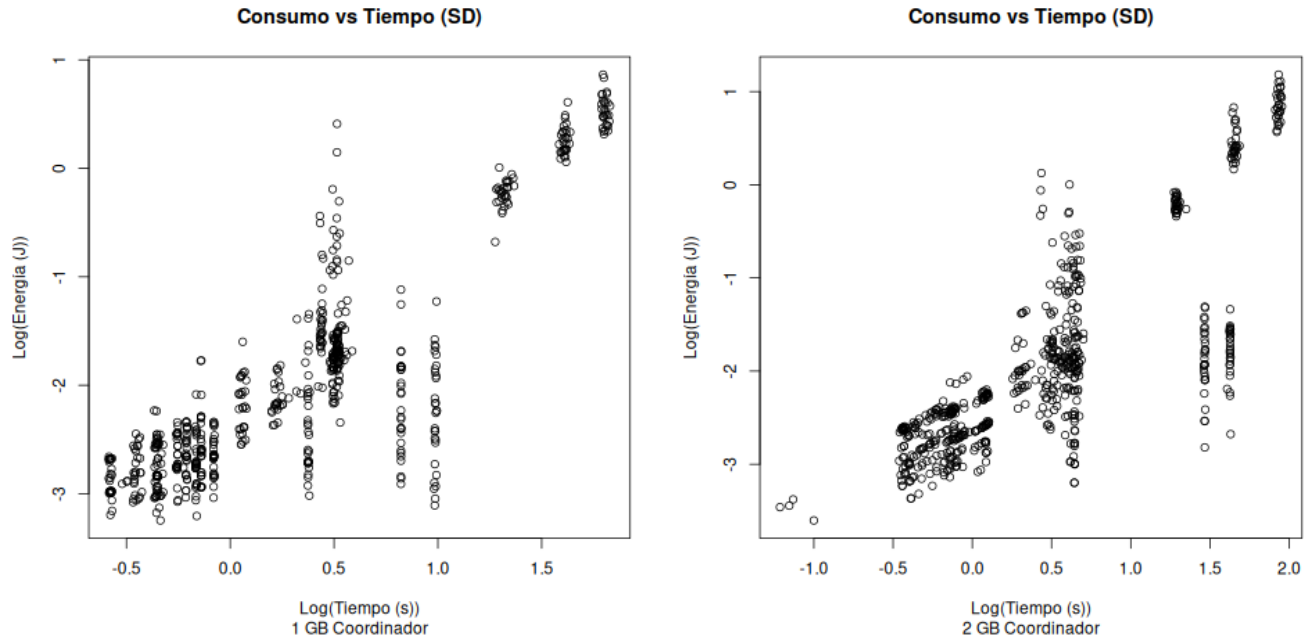


Figura E.10: Gráficos de dispersión de consumo del coordinador para 1 GB y 2 GB

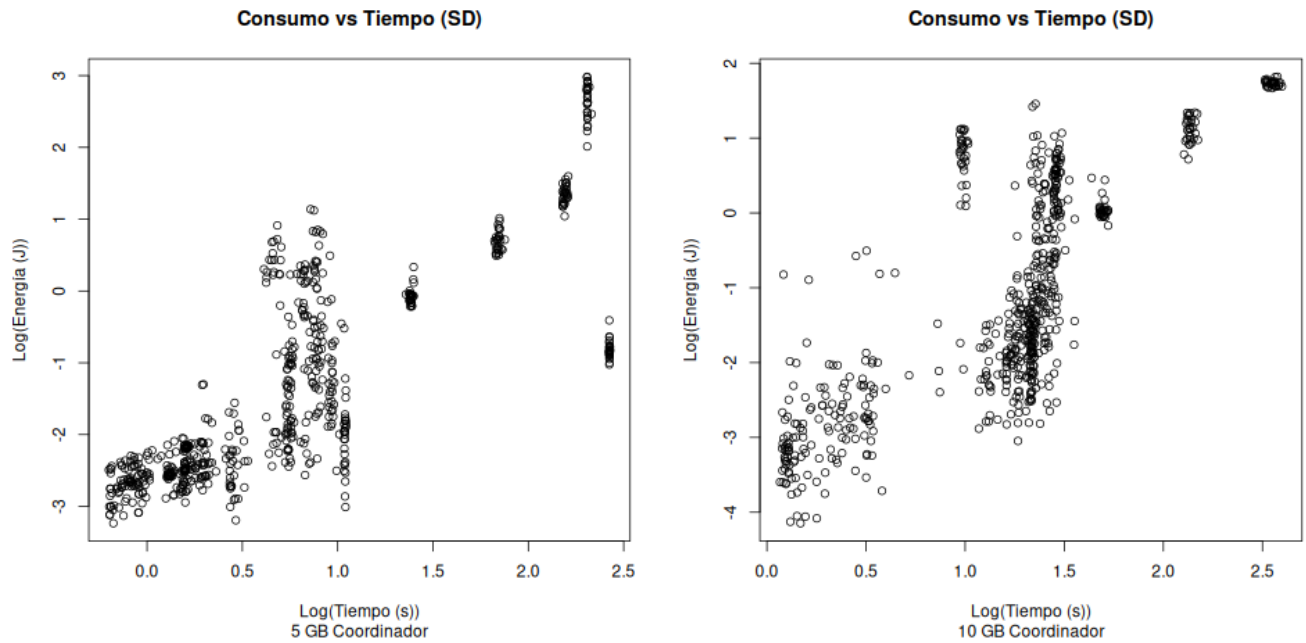


Figura E.11: Gráficos de dispersión de consumo del coordinador para 5 GB y 10 GB

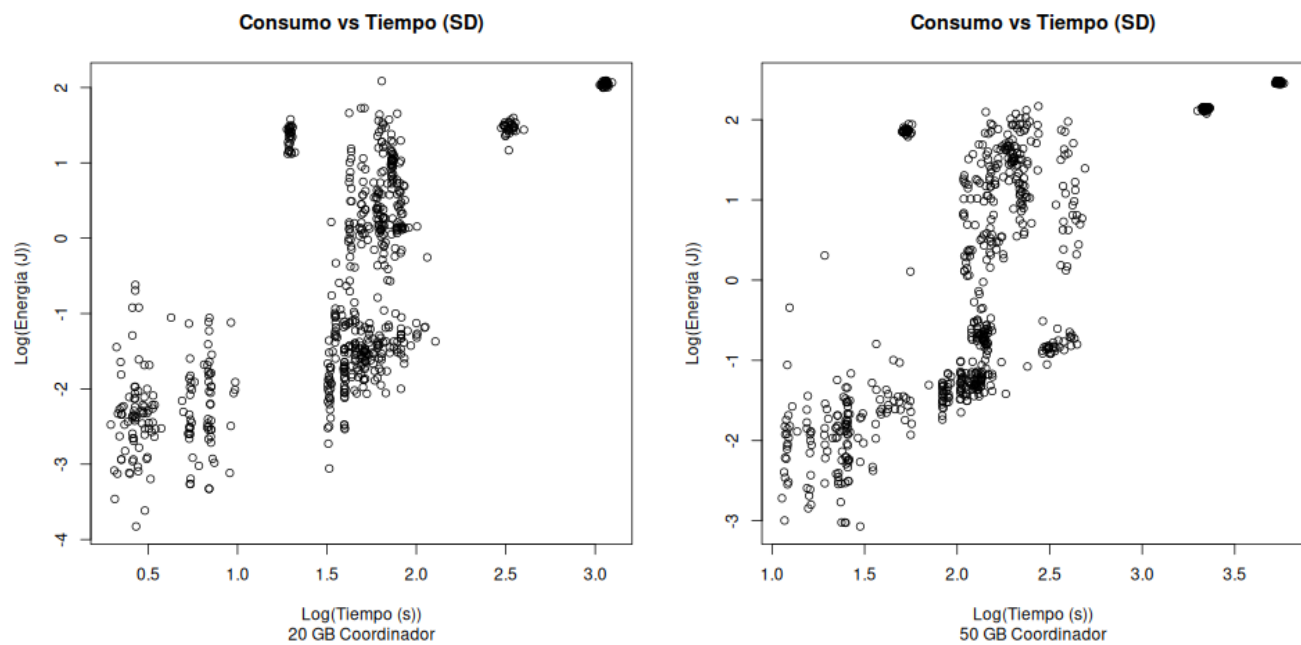


Figura E.12: Gráficos de dispersión de consumo del coordinador para 20 GB y 50 GB

Apéndice F. Gráficos de dispersión con diseño

F.1. Consumo neto

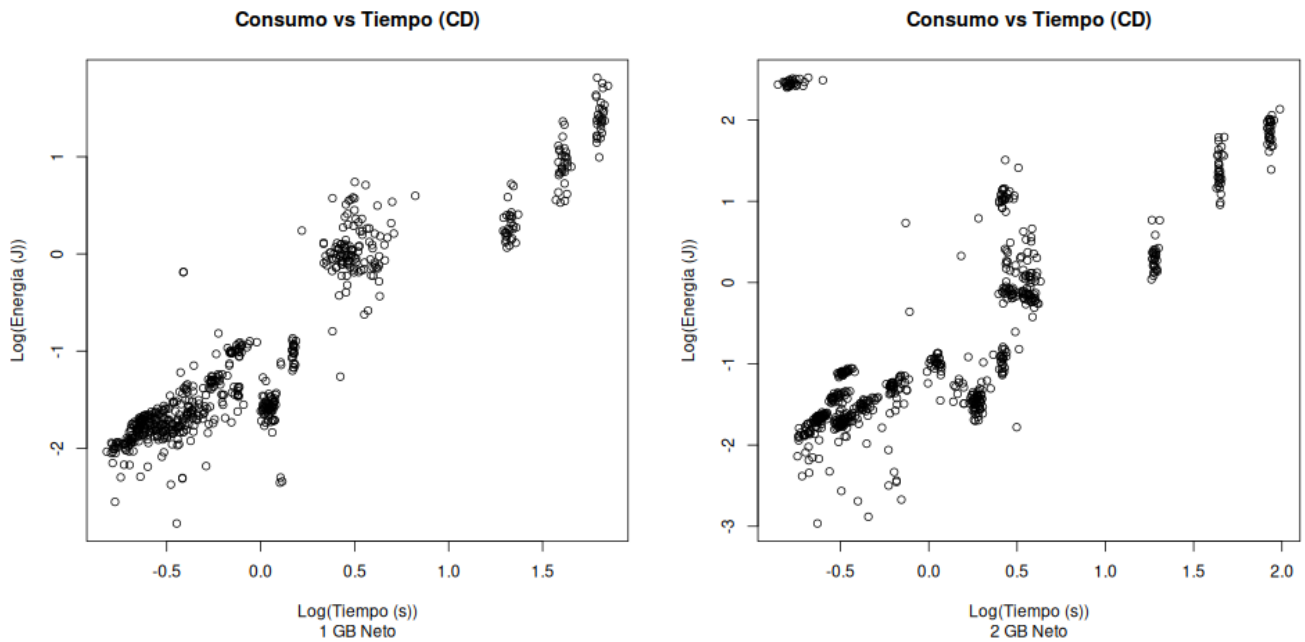


Figura F.1: Gráficos de dispersión de consumo neto para 1 GB y 2 GB

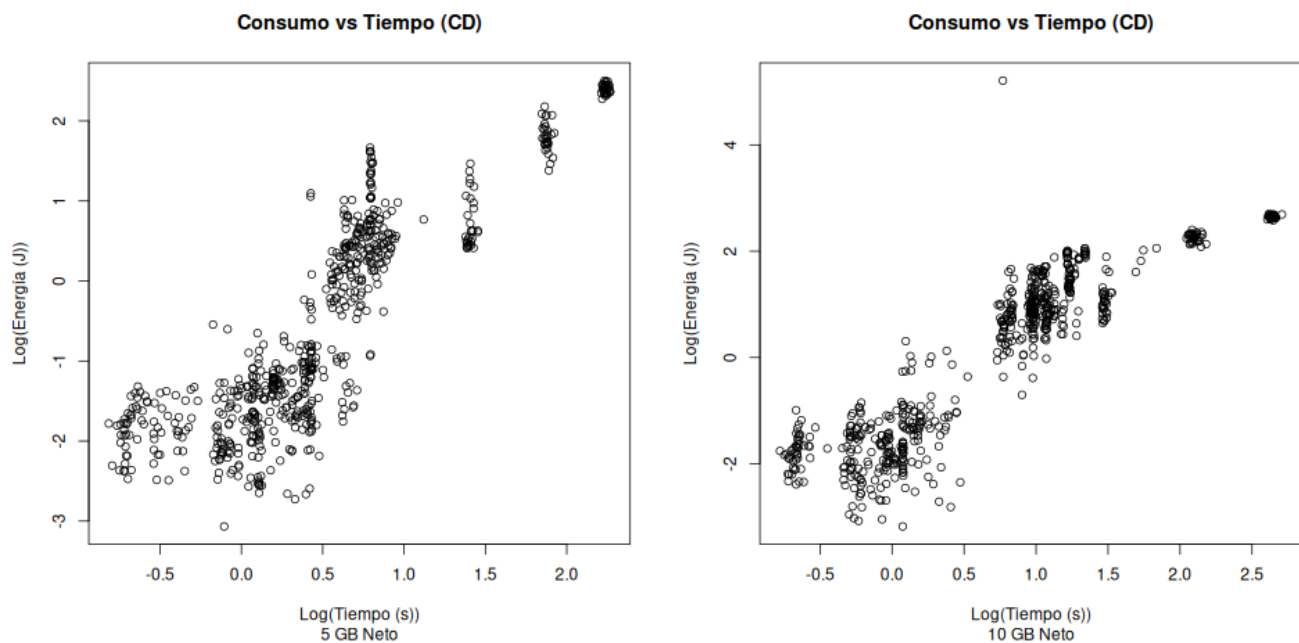


Figura F.2: Gráficos de dispersión de consumo neto para 5 GB y 10 GB

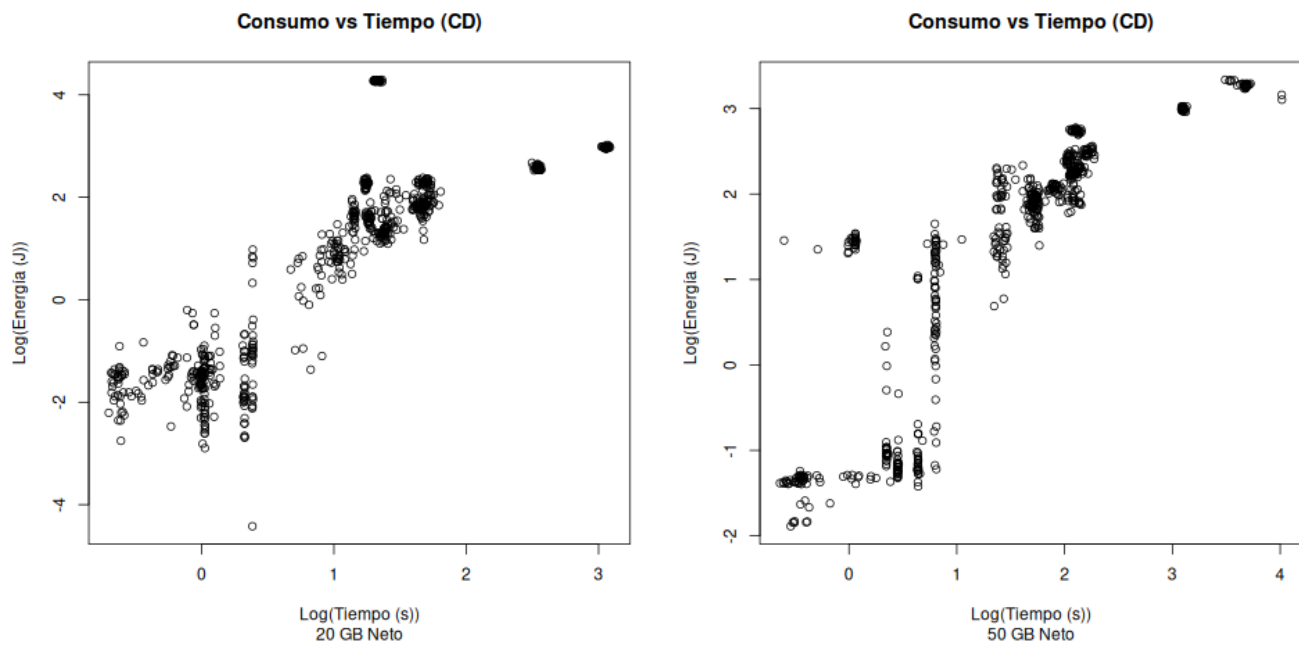


Figura F.3: Gráficos de dispersión de consumo neto para 20 GB y 50 GB

F.2. Consumo trabajador 1

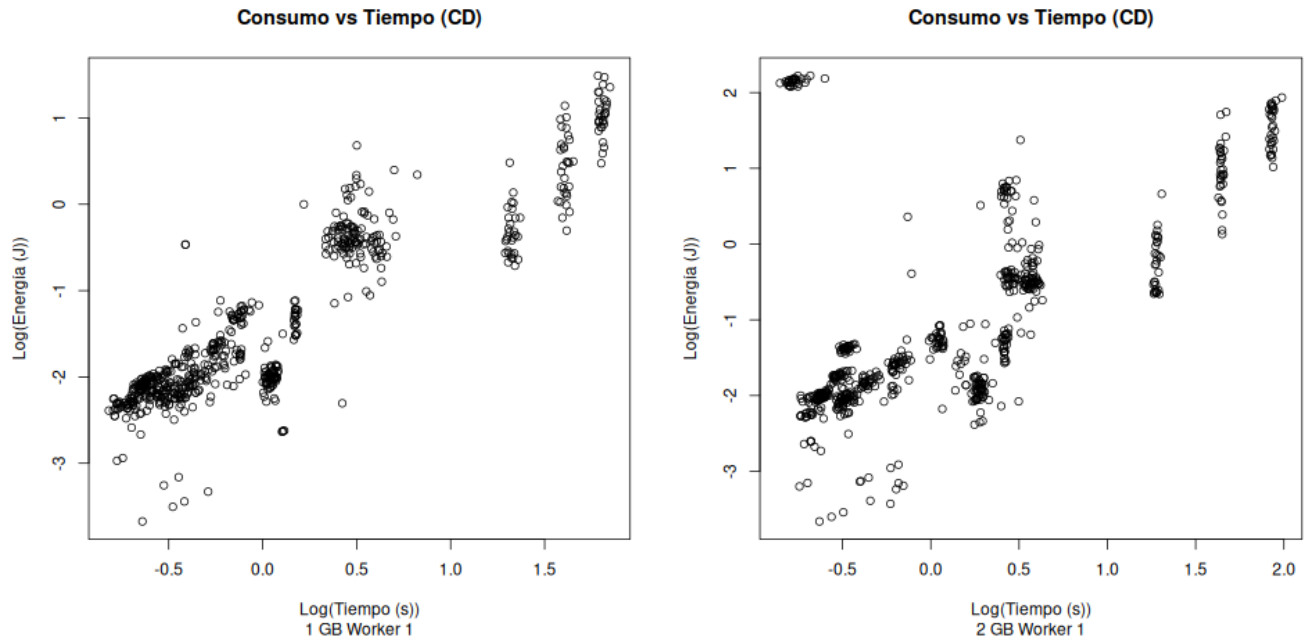


Figura F.4: Gráficos de dispersión de consumo del trabajador 1 para 1 GB y 2 GB

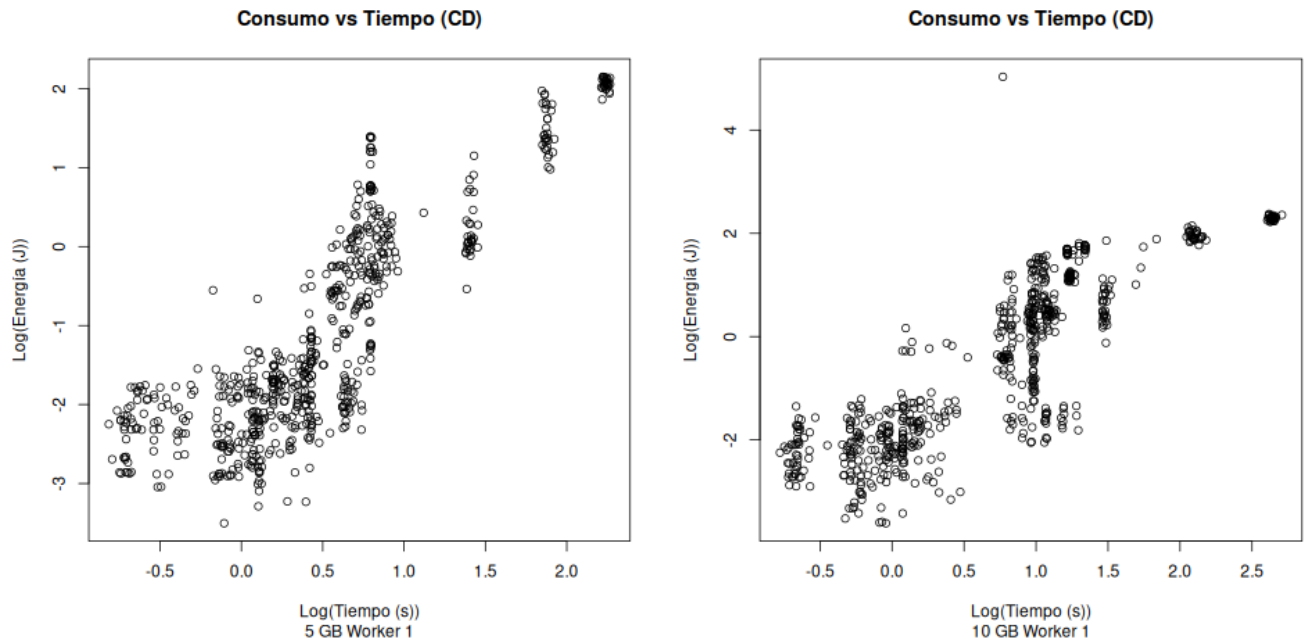


Figura F.5: Gráficos de dispersión de consumo del trabajador 1 para 5 GB y 10 GB

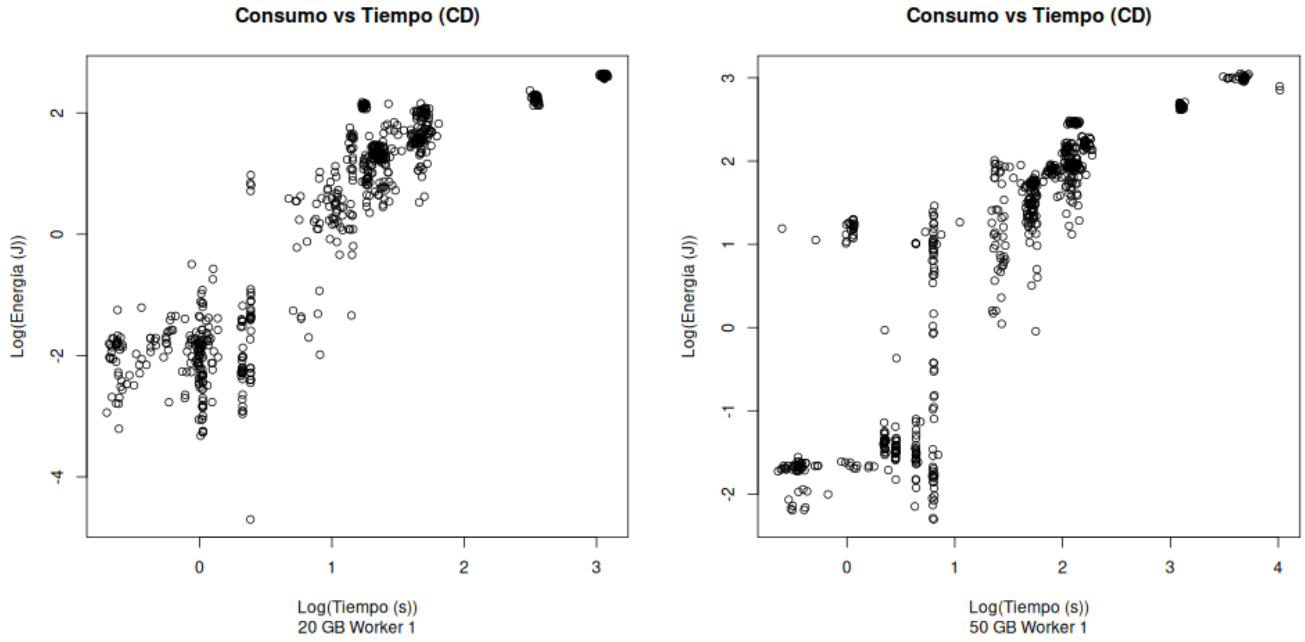


Figura F.6: Gráficos de dispersión de consumo del trabajador 1 para 20 GB y 50 GB

F.3. Consumo trabajador 2

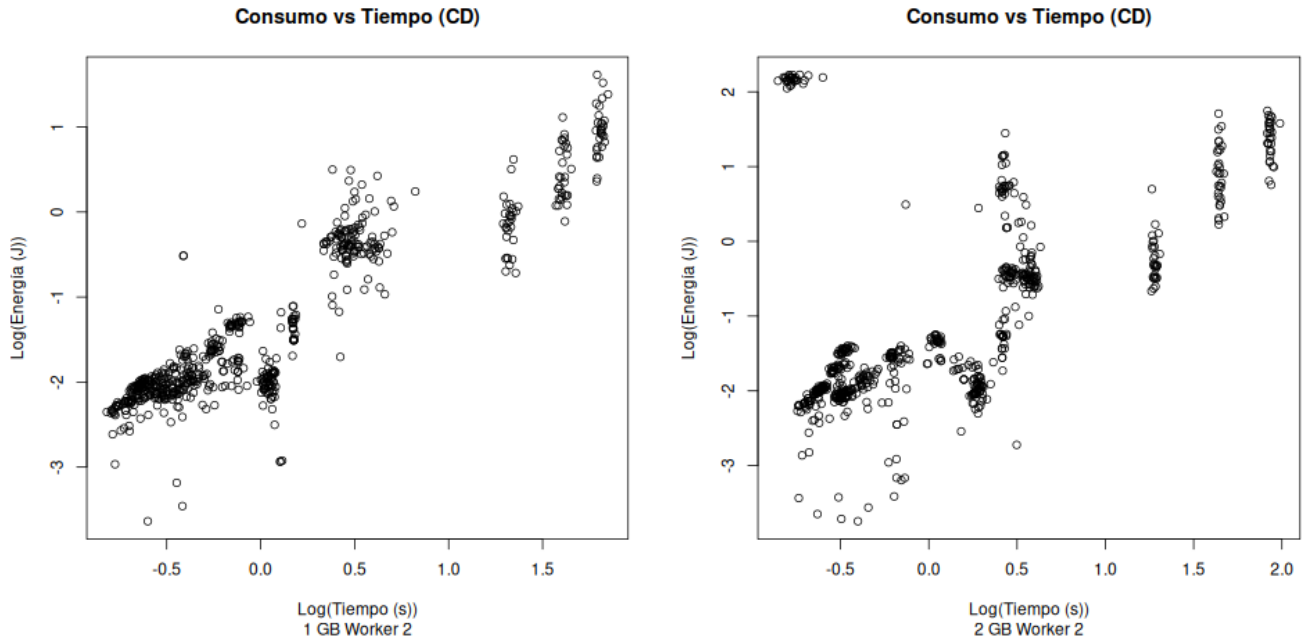


Figura F.7: Gráficos de dispersión de consumo del trabajador 2 para 1 GB y 2 GB

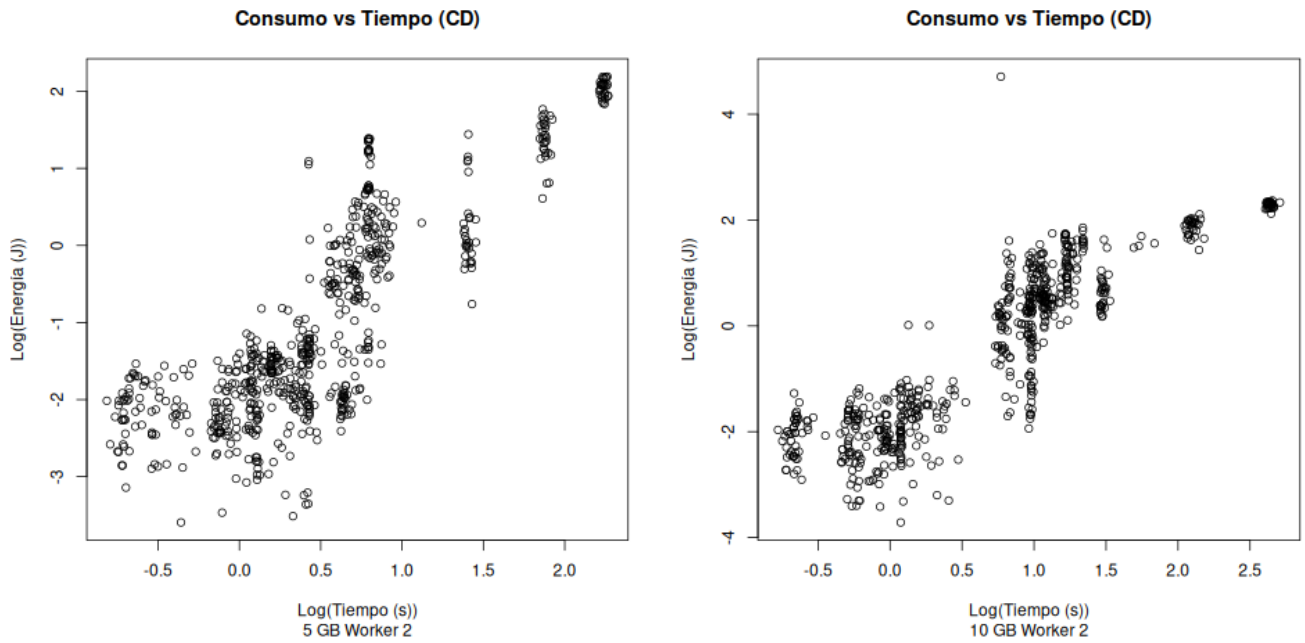


Figura F.8: Gráficos de dispersión de consumo del trabajador 2 para 5 GB y 10 GB

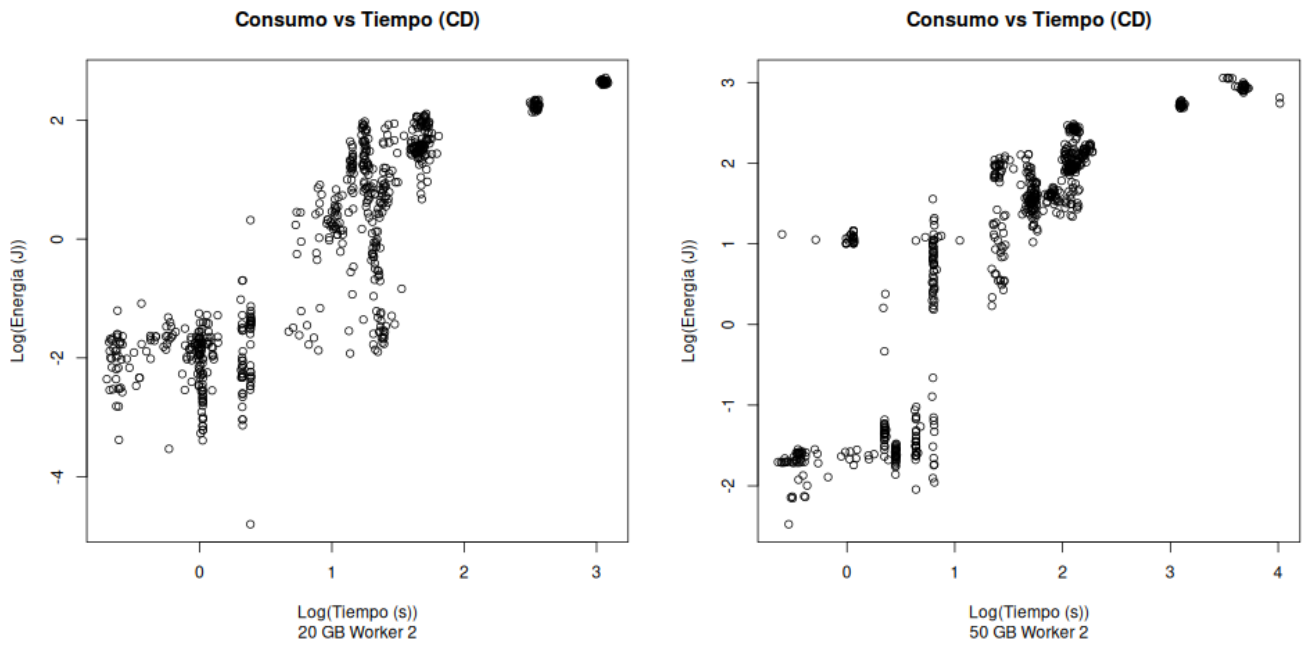


Figura F.9: Gráficos de dispersión de consumo del trabajador 2 para 20 GB y 50 GB

F.4. Consumo coordinador

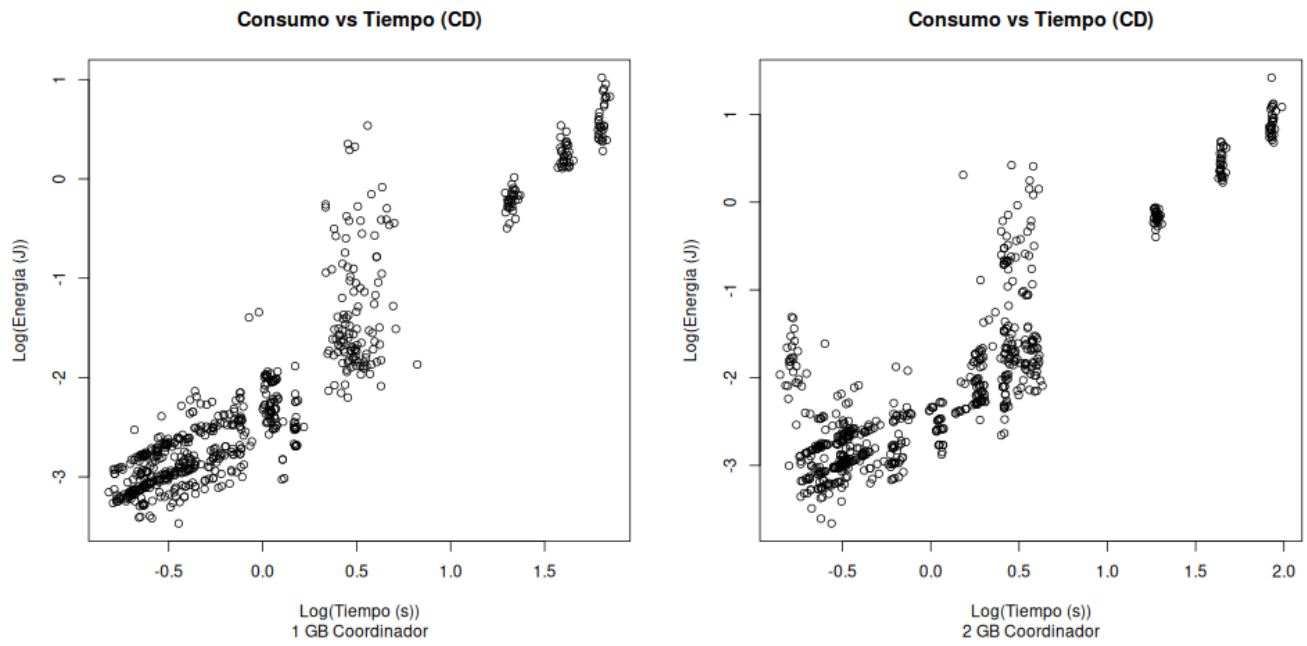


Figura F.10: Gráficos de dispersión de consumo del coordinador para 1 GB y 2 GB

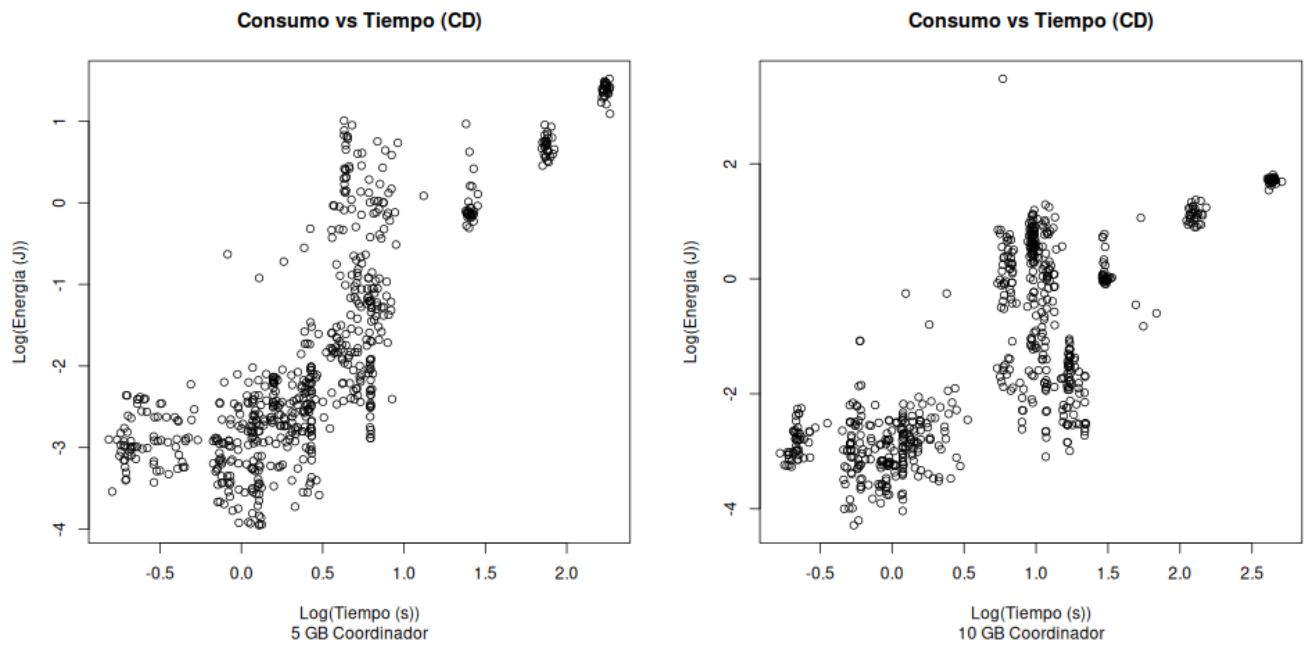


Figura F.11: Gráficos de dispersión de consumo del coordinador para 5 GB y 10 GB

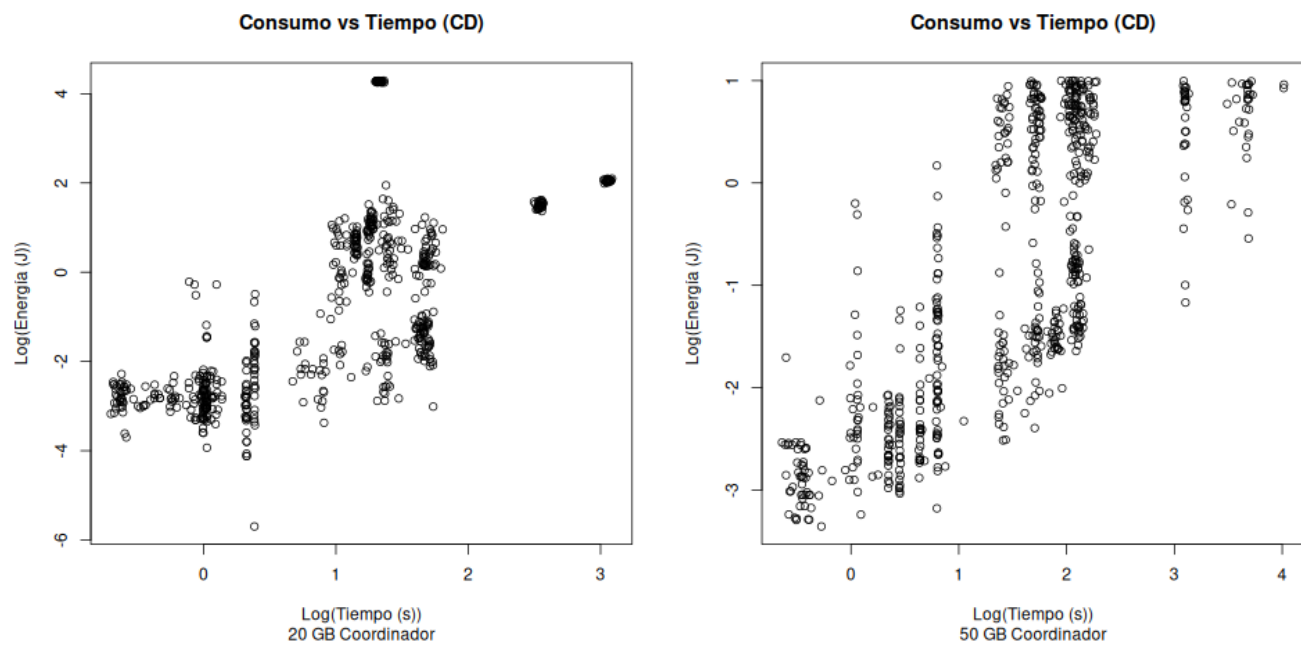


Figura F.12: Gráficos de dispersión de consumo del coordinador para 20 GB y 50 GB